

Speech and Video Processing (CS30033)

Complete Study Guide — Module I & Module II (up to HMM-based Speech Recognition)

Compiled from SVP Lesson Plan (Autumn 2026)

Contents

Preface	5
I MODULE I — SPEECH PROCESSING CONCEPTS	6
1 Introduction to Speech Processing	7
1.1 What is Speech? What is Processing?	7
1.2 Speech Transmission vs. Speech Processing	7
1.3 The Speech Processing Model	8
1.4 Applications of Digital Speech Processing	8
2 Human Speech Production and Perception	10
2.1 The Speech Chain	10
2.2 Speech Production System	10
2.3 Excitation Sources	11
2.4 Production of Different Speech Sound Categories	11
2.5 Speech Perception System	12
3 Acoustic-Phonetic Representation of Speech Sounds	13
3.1 Phonemes	13
3.2 IPA and ARPAbet	13
3.3 Vowels	13
3.4 Formant Positions for Vowels	14
3.5 F1-F2 Cluster and Centroid	14
3.6 Diphthongs and Semi-vowels	14
3.7 Consonants: Nasals, Fricatives, and Stops	15
3.8 Chapter 1-3 : Practice / Exam Questions	16
4 Digital Representation of Speech Signals	23
4.1 Basic Signal Concepts	23
4.1.1 Continuous-Time vs. Discrete-Time Signals	23
4.2 Digitization: The Big Picture	24
4.3 Sampling	24
4.4 The Nyquist-Shannon Sampling Theorem	25
4.5 Quantization	25
4.5.1 Quantization Step Size (QSS)	25
4.5.2 Quantization Error	26
4.6 Bit Rate and File Storage	26
4.7 Worked Numerical Examples — Chapter 4	27
4.8 Chapter 4 : Practice / Exam Questions	30
5 Time-Domain and Frequency-Domain Analysis; LTI Systems	33

5.1	Time-Domain vs. Frequency-Domain Representation	33
5.2	Discrete-Time Systems and LTI Systems	33
5.2.1	(a) Linearity — the Principle of Superposition	34
5.2.2	(b) Time Invariance	34
5.3	Worked Examples — LTI Properties	35
6	Convolution in the Time Domain	36
6.1	Impulse Decomposition	36
6.2	Impulse Response and Convolution	36
6.2.1	Properties of Convolution	37
6.3	Limitations of Time-Domain Convolution	37
6.4	Worked Numerical Examples — Convolution	38
7	Pole-Zero Modeling of Speech	40
7.1	Why Pole-Zero Modeling?	40
7.2	The Z-Transform (Brief)	40
7.3	Significance of Pole and Zero Locations	41
7.4	Pole-Zero Modeling for Speech	41
7.5	Worked Numerical Examples — Pole-Zero Modeling	42
7.6	Chapter 5–7 : Practice / Exam Questions	43
8	Frequency Analysis of Speech Signals: The Fourier Transform	47
8.1	Exponential Representation of Signals — Euler’s Formula	47
8.2	The Fourier Transform	47
8.2.1	Properties of the Fourier Transform	48
8.3	Discrete-Time Fourier Transform (DTFT)	49
8.4	Discrete Fourier Transform (DFT)	49
8.5	Worked Numerical Examples — DFT	51
8.6	Spectral Leakage	52
8.7	Complexity of the DFT, and the Fast Fourier Transform (FFT)	53
8.7.1	Even-Odd Decomposition (Radix-2 FFT)	53
8.7.2	Butterfly Network	54
8.8	Chapter 8 : Practice / Exam Questions	55
9	Spectral Analysis Models: Filter Banks and Linear Prediction	58
9.1	The Spectral Envelope	58
9.2	Bank-of-Filters (BOF) Model	58
9.2.1	Types of Digital Filters	58
9.2.2	Complete BOF Model Structure	59
9.3	The Mel Filter — a Non-Uniform, Perceptually Motivated Filter Bank	60
9.3.1	How to Implement a Mel Filter Bank	60
9.4	Applications of the Filter Bank Model	61
10	Linear Predictive Coding (LPC)	63
10.1	The Core Idea of Linear Prediction	63
10.2	Predicted Sample and Prediction Error	63
10.3	Prediction Order	64
10.4	All-Pole Interpretation of LPC	64
10.5	Estimating LPC Coefficients: Autocorrelation Method	65
10.5.1	Durbin’s Recursion (Levinson-Durbin Algorithm)	65

10.6 Typical LPC Model Parameters	66
10.7 Chapter 9–10 : Practice / Exam Questions	67

II MODULE II — FEATURE EXTRACTION AND SPEECH RECOGNITION 71

11 Feature Extraction: An Overview 72	72
11.1 What is a Feature?	72
11.2 Why Do We Need Feature Extraction?	72
11.3 Types of Features	73
11.3.1 Static Features	73
11.3.2 Dynamic Features	74
11.4 Chapter 11 : Practice / Exam Questions	75
12 Cepstral Analysis 77	77
12.1 What is the Cepstrum?	77
12.1.1 New Terminology in the Cepstral Domain	77
12.1.2 Why Does This Work? — The Core Insight	77
12.2 Steps to Compute the Cepstrum	78
12.2.1 Master Formula	78
12.3 Principle of Cepstral Analysis — Why the Logarithm?	78
12.4 Real Cepstrum	79
12.4.1 Steps to Calculate the Real Cepstrum	79
12.5 Complex Cepstrum	79
12.5.1 Steps to Calculate the Complex Cepstrum	80
12.6 Cepstral Analysis for the Speech Signal — Source-Filter Separation	80
13 Mel-Frequency Cepstral Coefficients (MFCC) 82	82
13.1 Motivation	82
13.2 The Seven Steps to Extract MFCC	82
13.2.1 Step 1 — A/D Conversion	82
13.2.2 Step 2 — Pre-emphasis	82
13.2.3 Step 3 — Framing and Windowing	82
13.2.4 Step 4 — Discrete Fourier Transform (DFT)	83
13.2.5 Step 5 — Mel Filterbank	83
13.2.6 Step 6 — Log	84
13.2.7 Step 7 — IDFT (Inverse Discrete Fourier Transform)	84
13.3 Applications of MFCC	84
14 Linear Prediction Cepstral Coefficients (LPCC) 85	85
14.1 What is LPCC?	85
14.2 Steps to Extract LPCC	85
14.2.1 Step 1 — Pre-emphasis	85
14.2.2 Step 2 — Frame Blocking	85
14.2.3 Step 3 — Windowing	86
14.2.4 Step 4 — Autocorrelation Analysis	86
14.2.5 Step 5 — LPC Analysis	86
14.2.6 Step 6 — LPC Parameter Conversion	86
14.2.7 Step 7 — Parameter Weighting and Temporal Derivatives	87

14.3Applications of LPCC	87
14.4MFCC vs. LPCC — Comparison	87
14.5Chapter 12-13 : Practice / Exam Questions	89
15Dynamic Features and Normalization	93
15.1Delta (Δ) Coefficients	93
15.2Delta-Delta ($\Delta\Delta$) Coefficients	93
15.3Normalization	94
15.3.1Why Normalize?	95
15.3.2Mean and Variance Normalization	95
15.3.3Cepstral Mean Normalization (CMN)	96
16Vector Quantization (VQ)	97
16.1Motivation for Vector Quantization	97
16.2Vector Quantization Terminology	97
16.3Distance Measure — Euclidean Distance	97
16.4Codebook Generation (Brief Overview — LBG Algorithm)	98
16.5Applications of Vector Quantization	98
16.6Chapter 15-16 : Practice / Exam Questions	100
17Pattern Matching Approaches	102
17.1What is Pattern Matching in Speech Recognition?	102
17.2Categories of Pattern Matching Approaches	102
17.2.1Template Matching	102
17.3Distance Measures	103
17.4Why Does Simple Matching Fail for Speech?	103
18Hidden Markov Model (HMM) Based Speech Recognition	104
18.1What is a Hidden Markov Model?	104
18.2Parameters of an HMM	104
18.3Left-to-Right (Bakis) HMM for Speech	105
18.4The Three Classic Problems of HMM	105
18.5The Forward Algorithm	105
18.6The Viterbi Algorithm	107
18.7The Baum-Welch Algorithm (Training)	108
18.8Continuous and Discrete HMM	109
18.9Chapter 17-18 : Practice / Exam Questions	110

Preface

This study guide has been prepared strictly from the **SVP Lesson Plan (Autumn 2026)** reference material, covering everything up to **Module II: HMM-based Speech Recognition**, exactly as per your syllabus. Every topic listed in the lesson plan is covered in detail, every numerical from the slides is worked out step-by-step, and a complete question bank (with detailed answers) is provided at the end of each chapter.

This edition has been rewritten in **plain, simple language** — as if a friend who understands the subject well is explaining it to you over a cup of coffee. Wherever a concept might feel abstract, you'll find a **real-life example** (your phone, WhatsApp calls, Alexa, YouTube, Zoom, etc.) to anchor it, along with **diagrams** to help you visualize what's happening.

How to use this guide:

- Read the plain-language explanation and the real-life example first — don't worry about the formula right away.
- Look at the diagram, then re-read the formula — it should click much faster now.
- Work through every numerical yourself before reading the given solution.
- Attempt the "Practice / Exam Questions" at the end of each chapter without looking at the answer first.
- Formulas that are highly likely to be asked directly are boxed for quick revision.

Part I

MODULE I — SPEECH PROCESSING CONCEPTS

Chapter 1

Introduction to Speech Processing

1.1 What is Speech? What is Processing?

Think about the last time you spoke to Alexa, Siri, or Google Assistant. You said something like “play some music,” and within a second, it understood you and responded. Ever wondered what actually happens in that one second? That entire journey — from the sound leaving your mouth to a machine understanding it — is what this subject is about.

Speech is simply the natural way humans talk to each other. In technical terms, it is the process of producing and transmitting spoken language through sound waves, in order to communicate information, ideas, and emotions. Every time you speak, your voice signal is actually carrying **three kinds of information at once**, even though you never think about it consciously:

- **Speaker information** — who is speaking (your own voice sounds different from your friend’s, even saying the exact same word).
- **Language information** — what language, accent, or dialect is being used.
- **Message information** — the actual words/meaning you are trying to convey.

Processing, in plain terms, just means “doing something useful with data.” Here, it means analyzing, transforming, or manipulating a signal to pull out useful information or achieve some goal.

Put the two words together and you get **Speech Processing**: the study of speech signals and the techniques used to analyze, recognize, compress, or reproduce them. This is exactly what powers voice assistants, automatic transcription, voice-based translation, and voice biometric security.

Real-life scenario: When you send a voice note on WhatsApp, that app is doing *speech processing*: it samples your voice, compresses it, and stores it as a tiny file. When you say “Hey Google, what’s the weather,” Google Assistant is doing an even deeper form of speech processing: it converts your voice into text, understands your intent, fetches the weather, and speaks the answer back to you. Same subject, two different depths of use.

Why do we even need Speech Processing? Because computers do not naturally understand sound the way we do. We need methods to **analyze, enhance, compress, recognize, and synthesize** speech so that a machine can “listen” and “talk back” — this is the whole foundation of voice assistants, transcription apps, real-time translators, and voice-based security systems.

1.2 Speech Transmission vs. Speech Processing

People often mix these two up, but they solve **completely different problems**. Let's use a simple analogy: a **courier company** vs. a **reader**.

- A courier company (transmission) just wants to deliver your parcel safely and quickly — it does not care what is inside the parcel.
- A reader (processing) actually opens the letter and understands what it says.

Speech Transmission (e.g., a normal phone call):

- Your voice is captured by a microphone and converted into an electrical signal.
- The signal is compressed using codecs (like AMR or Opus) so it takes up less bandwidth.
- The compressed signal travels over the phone network (GSM, VoLTE, or VoIP).
- **Goal:** simply deliver your voice clearly to the other person — the network makes **no attempt to understand** what you are saying.

Digital Speech Processing (e.g., Alexa or Google transcription):

- Raw audio is converted into numerical features such as MFCC or LPC (covered in later chapters).
- The system identifies phonemes, words, and meaning from those features.
- Used in voice assistants, automatic transcription, biometrics, and machine translation.
- **Goal:** actually understand, analyze, or act upon the *content* of your speech.

Exam one-liner: Speech transmission cares about *delivering* the voice signal faithfully; speech processing cares about *understanding/extracting information from* the voice signal.

1.3 The Speech Processing Model

Imagine you're teaching a robot to understand speech from scratch. What steps would it need to go through? Roughly six stages — this is exactly the general model of a speech-processing system:

Information Source → Measurement → Signal Representation

→ Signal Transformation → Feature Extraction → Interpretation of Information

1. **Information Source:** This is simply the human speaker who wants to say something.
2. **Measurement:** A microphone captures the raw sound wave and turns it into an electrical signal — this is the “recording” step.
3. **Signal Representation:** The continuous (analog) electrical signal is converted into a discrete, digital form your computer can actually store — this involves **sampling** and **quantization** (Chapter 2).
4. **Signal Transformation:** The signal is cleaned up — removing noise, filtering, and slicing it into small chunks (framing/windowing).

5. **Feature Extraction:** The cleaned waveform is converted into a small set of numbers that capture what matters about the sound (e.g., MFCC, LPC — Module II).
6. **Interpretation of Information:** Finally, those numbers are used to figure out which phoneme, word, or speaker it was.

Real-life scenario: This is literally what happens, in order, every single time you say “OK Google.” Your phone’s microphone measures the sound (Step 2), digitizes it (Step 3), cleans and slices it (Step 4), extracts features (Step 5), and finally a recognition model interprets those features as the words “OK Google” (Step 6) — all in a fraction of a second.

1.4 Applications of Digital Speech Processing

- **Smart Homes:** Voice assistants (Alexa, Siri, Google Assistant) controlling appliances, lighting, entertainment.
- **Education:** Real-time captioning/transcription, pronunciation and fluency checking in language-learning apps (like Duolingo’s speaking exercises).
- **Healthcare:** Recording doctor-patient conversations, prescription assistance, speech-aid devices for the disabled.
- **Media:** Live captioning for broadcast and streaming (YouTube’s auto-captions, Netflix subtitles).
- **Business/Commerce:** Speech-driven customer service chatbots (the automated voice that handles your bank’s customer care call).
- **Automotive:** Voice commands for navigation, infotainment, hands-free calling.
- **Security:** Voice biometric authentication — some banking apps let you log in just by saying a phrase, because your voice is as unique as a fingerprint.

Modern real-time platforms include **Deepgram** (ultra-low-latency speech-to-text with diarization/sentiment analysis), **AssemblyAI** (real-time transcription with speaker labelling), **OpenAI Realtime** (speech recognition + conversational AI), and **Google Cloud Speech-to-Text** (125+ languages).

Chapter 2

Human Speech Production and Perception

2.1 The Speech Chain

Have you ever thought about how many steps happen between you *thinking* “I’m hungry” and your friend *understanding* you said it? Linguists call this journey the “**speech chain**.” It has six broad steps:

1. A speaker **forms a linguistic message** in the brain (you decide what to say).
2. The brain sends **motor commands** to the vocal organs (mouth, tongue, lips) to produce the intended sounds.
3. The vocal organs **generate an acoustic (sound) wave** — this is speech **production**.
4. The acoustic wave **travels through air** to a listener.
5. The listener’s ear captures the sound wave and converts it into neural signals — this is speech **perception**.
6. The listener’s brain **decodes** the neural signals back into a linguistic message (your friend understands “I’m hungry”).



Figure 2.1: The Speech Chain: from a thought in your brain to understanding in someone else’s

Real-life scenario: This is exactly what happens on every phone call. You think of a sentence, your mouth produces the sound, it becomes an electrical signal, travels through the network, comes out of your friend’s phone speaker as sound again, and their ears and brain decode it. Speech technology essentially tries to insert a “ma-

chine” somewhere into this same chain — either to carry the signal (transmission) or to understand it (processing).

2.2 Speech Production System

You don’t need to be a biology expert for this course, but it helps enormously to have a simple mental picture of “how sound comes out of a human.” Think of your vocal system as an **assembly line** for sound — air goes in one end, and a fully-shaped speech sound comes out the other end.

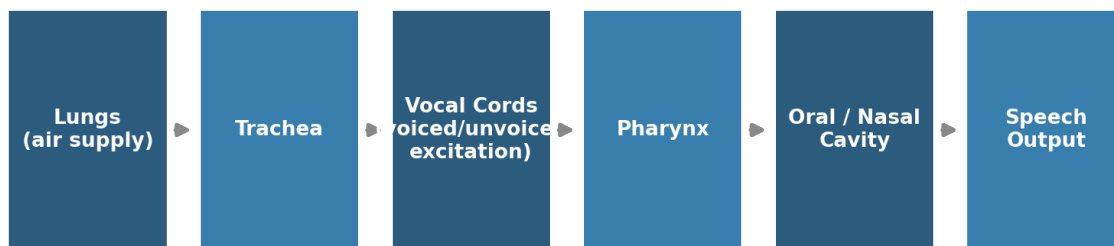


Figure 2.2: Your vocal tract as an assembly line for sound

1. Lungs (Air Source): Just like a bagpipe or an accordion needs air pumped into it to make sound, your lungs provide the airflow needed for speech. Your diaphragm pushes air upward through the trachea (windpipe).

2. Vocal Cords / Vocal Folds (Sound Source): Air from the lungs passes through the vocal cords located in the larynx (the “voice box” — you can feel it as your Adam’s apple).

- When the vocal cords **vibrate**, you get **voiced sounds** (e.g., vowels, /b/, /d/, /g/, /z/...). Try saying “zzzz” with your finger on your throat — you’ll feel the buzz.
- When the vocal cords are **held open** (not vibrating), air passes through with turbulence, producing **unvoiced sounds** (e.g., /s/, /f/, /p/, /t/, /k/...). Try “ssss” — no buzz this time.

3. Pharyngeal Cavity: The sound next enters the throat cavity, which acts like the first “echo chamber,” starting to shape the sound.

4. Oral Cavity: The tongue, lips, teeth, and jaw work like an equalizer, changing the airflow to produce all the different sounds we associate with speech. **Most speech sounds are shaped here.**

5. Velum (Soft Palate): Think of the velum as a **traffic valve** that decides whether air exits via the mouth or the nose.

- **Velum raised:** the “nose door” is closed, so air is forced out only through the mouth → **oral sounds**.
- **Velum lowered:** the “nose door” opens too, letting air into the nasal cavity → **nasal sounds** such as /m/, /n/, /ŋ/ (try pinching your nose while saying “mmmm” — it sounds completely different or gets blocked, proving air was going through your nose).

6. Nasal Cavity: An extra echo chamber used only for nasal sounds, before air finally exits through the nostrils.

7. Speech Output: The final shaped sound exits through the mouth and/or nose as the speech you hear.

2.3 Excitation Sources

The “excitation” is what actually sets the air molecules vibrating before the vocal tract shapes the sound into something recognizable — think of it as the “raw energy” that the vocal tract then sculpts, the way a guitar string’s vibration (excitation) is shaped by the guitar body (filter) into a musical note. There are three types:

- **Voiced excitation:** produced by the periodic vibration of the vocal folds → gives **voiced speech** (quasi-periodic, buzzy).
- **Unvoiced excitation:** produced by forcing air through a constriction (creating turbulent, noise-like airflow) → gives **unvoiced speech** (hissy, random).
- **Mixed excitation:** voiced and unvoiced excitation happening together (e.g., voiced fricatives like /z/, /v/, where you get both a buzz and a hiss at once).

2.4 Production of Different Speech Sound Categories

Sound type	Manner of Production
Vowels	Oral cavity wide open, tongue hump position varies, glottis vibrates
Unvoiced consonants	Complete closure (constriction) in the vocal tract, no vocal-fold vibration
Voiced consonants	Constriction in vocal tract and glottal (vocal-fold) vibration
Nasal sounds	Nasal cavity is coupled in (velum lowered)
Fricatives	Partial closure, narrow opening causing turbulent airflow

2.5 Speech Perception System

If speech production is the “speaker” side, speech perception is the “listener” side — how your ear and brain turn sound waves back into meaning. The human ear (the “receiver” of the speech chain) has **three main regions**:

- **Outer Ear:** Works like a satellite dish — it collects the speech sound waves from the air and funnels them down the ear canal toward the eardrum.
- **Middle Ear:** Amplifies the sound and converts the air-pressure variations into **mechanical vibrations** using three tiny bones (ossicles) — like a mechanical lever system boosting a weak signal.
- **Inner Ear:** Converts the mechanical vibrations into **electrical nerve impulses** (via the cochlea), which travel to the brain through the auditory nerve — this is where sound finally becomes “data” your brain can use.
- **Brain:** Decodes these nerve impulses and interprets the message to derive meaning — the same way a phone’s speech-recognition app decodes a digital signal into words.

The audible frequency range for a typical human is approximately **20 Hz to 20 kHz**. This single fact motivates several engineering choices discussed later (e.g., the 44.1 kHz sampling rate used in audio CDs is chosen so the Nyquist frequency, 22.05 kHz, safely covers this entire audible range).

Chapter 3

Acoustic-Phonetic Representation of Speech Sounds

3.1 Phonemes

Think about the words “bat” and “pat.” They sound almost identical, except for one tiny sound at the start — yet that one small change completely changes the meaning of the word. That smallest “building block” sound that can change a word’s meaning is called a **phoneme**.

Formally: a **phoneme** is the **smallest unit of sound in a language that changes the meaning of a word** when substituted with another.

Important nuance for exams: **the exact number of phonemes in a language is not fixed** — it is a matter of linguistic judgement and varies between linguists. For example, in English, some linguists count 44 phonemes, others count 42, 45, or 46, depending on whether certain sound variations are treated as separate phonemes or merely as variants (“allophones”) of the same phoneme.

3.2 IPA and ARPAbet

If every language has its own way of writing sounds, how do speech scientists working across different languages talk about sounds consistently? They use a shared, standardized “alphabet of sounds”:

- **IPA (International Phonetic Alphabet):** A standardized system of symbols that can represent the sounds of *any* spoken language in the world, precisely and unambiguously. It is the global “alphabet of sounds,” similar to how music notation is a universal way to write down any melody regardless of language.
- **ARPAbet:** A version of the same idea, but designed to be typed on a plain keyboard (ASCII text) instead of needing special IPA symbols. Because it is easy for a computer to store and process, it was widely used in early speech recognition and speech synthesis systems, especially in the United States.

3.3 Vowels

Vowels are the “open-mouth” sounds — produced with an open vocal tract, letting air flow freely (say “aaa,” “eee,” “ooo” and notice your mouth stays open the whole time).

They are:

- Typically **voiced** (vocal cords vibrate).
- Classified by **tongue position** (height and backness/frontness), **lip rounding**, and **tenseness**.
- Generally **longer in duration** compared to consonants.
- Very reliably recognized by both humans and machines — try reading “th_ qu_ ck br_ wn f_ x” (vowels removed) versus “-e -ui- -o-n -o-” (consonants removed): vowels carry a surprising amount of intelligibility, which is why they play such a big role in speech recognition.

3.4 Formant Positions for Vowels

Every vowel has its own unique “acoustic fingerprint,” created by resonances in your vocal tract called **formants** — these are frequency peaks that shape the quality (timbre) of a vowel and let us tell one vowel apart from another (this is exactly why “ee” sounds different from “oo,” even said at the same pitch).

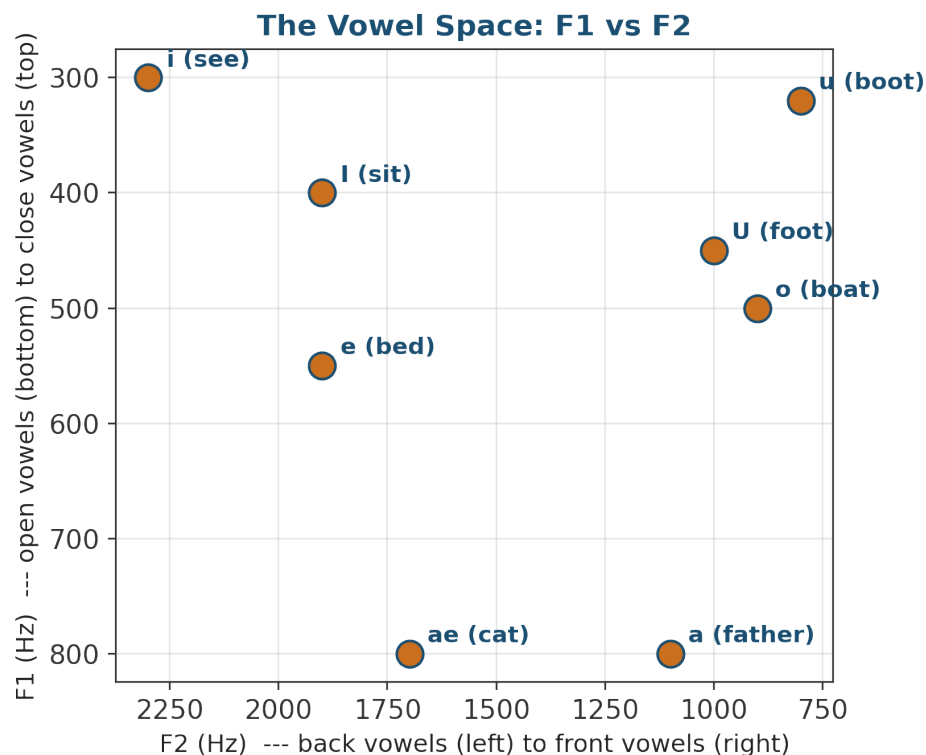


Figure 3.1: The vowel space: every vowel sits at its own characteristic F1-F2 location

F1 and F2 are the two most important formants for vowel identity:

- **F1** varies *inversely* with tongue height.
 - High tongue position (close vowels, e.g., /i/, /u/) → **Low F1**
 - Low tongue position (open vowels, e.g., /a/, /æ/) → **High F1**

- **F2** reflects tongue frontness/backness.
 - Front vowels (/i/, /e/, /æ/) → **High F2**
 - Back vowels (/u/, /o/, /open-o/) → **Low F2**

Higher formants (F3, F4) add finer detail related to timbre and speaker identity, rather than vowel identity itself.

Why is F1 related to vowel openness? Think of your mouth as a resonating tube, like a bottle. Blow across a bottle that's more "open" (bigger cavity) and you get one pitch of resonance; a "closer" cavity gives another. A low tongue position creates a large oral cavity opening, which resonates at a higher F1; a high tongue position constricts the cavity, lowering F1.

Why is F2 useful for distinguishing front and back vowels? F2 depends on where along the vocal tract the tongue makes its main constriction. A front-tongue constriction (like in "ee") leaves a short cavity behind it, which resonates at a high F2; a back-tongue constriction (like in "oo") leaves a long cavity behind it, resonating at a low F2 — same idea as a shorter vs. longer flute producing different pitches.

3.5 F1-F2 Cluster and Centroid

- **F1-F2 Cluster:** If you asked 100 different people to say the vowel "ee," their F1-F2 values would all be *slightly* different (due to differences in accent, vocal-tract size, etc.), but they'd all cluster together in roughly the same region of the chart above. This scattered grouping of measured F1-F2 values for one vowel, across many speakers, is called the F1-F2 cluster.
- **F1-F2 Centroid:** The **average** (central) F1 and F2 position of that cluster — i.e., the single "typical" point that best represents that vowel. It's a handy reference for vowel identification, comparing pronunciation across speakers, and building automatic speech/vowel recognition systems.

3.6 Diphthongs and Semi-vowels

A **diphthong** is a *gliding* vowel sound where the tongue moves smoothly from one vowel position to another **within the same syllable**. It starts at one vowel target and ends at another.

Examples in English:

- /aɪ/ as in "bite" (glides from /a/ towards /ɪ/)
- /eɪ/ as in "make" (glides from /e/ towards /ɪ/)
- /ɔɪ/ as in "boy" (glides from /ɔ/ towards /ɪ/)
- /aʊ/ as in "house" (glides from /a/ towards /ʊ/)
- /oʊ/ as in "boat" (glides from /o/ towards /ʊ/)

Semi-vowels (also called **glides**) are sounds that behave **acoustically like vowels** but **function like consonants** in the syllable structure (i.e., they occur at syllable edges, not as the syllable nucleus).

Examples in English:

- /j/ as in “yes” — acoustically similar to the vowel /i/
- /w/ as in “we” — acoustically similar to the vowel /u/

Diphthongs vs. Semi-vowels: A diphthong *is* the vowel nucleus of a syllable and glides between two vowel qualities. A semi-vowel is *not* a syllable nucleus — it is consonant-like in function even though its sound is vowel-like, and it typically appears immediately next to a full vowel (e.g., “yes,” “we”).

3.7 Consonants: Nasals, Fricatives, and Stops

Nasal Consonants: Air passes through the *nose* while the oral cavity is completely blocked.

- /m/ (as in “man”) — bilabial nasal (lips closed)
- /n/ (as in “net”) — alveolar nasal (tongue tip against the alveolar ridge)
- /ŋ/ (as in “sing”) — velar nasal (back of tongue against the velum)

Fricatives: Produced by forcing air through a **narrow constriction**, creating audible turbulence (friction-like hissing noise). Fricatives can be voiced or voiceless.

- Voiceless: /f/ (fun), /s/ (sun), /ʃ/ (shoe)
- Voiced: /v/ (van), /z/ (zoo), /ʒ/ (measure)

Stops (Plosives): Produced by a **complete closure** of the vocal tract, followed by a **sudden release** of the built-up air pressure. Stops can be voiced or voiceless.

- Voiceless: /p/ (pat), /t/ (top), /k/ (cat)
- Voiced: /b/ (bat), /d/ (dog), /g/ (go)

Stops can also be classified by **place of articulation**:

Place of Articulation	Example Consonants
Velar stops	/ka/, /kha/, /ga/, /gha/
Palatal stops	/ch/, /cha/, /j/, /jha/
Alveolar/Retroflex stops	/T/, /Th/, /D/, /Dh/
Dental stops	/t/, /th/, /d/, /dh/
Bilabial stops	/p/, /ph/, /b/, /bh/

3.8 Chapter 1-3 : Practice / Exam Questions

Q1. Define Speech Processing. Write the necessity of speech processing in modern-world applications.

Answer: Speech processing is the study of speech signals and the methods of processing them, involving analysis, synthesis, recognition, and coding of speech for communication and human-computer interaction applications. It is necessary because it enables machines to efficiently analyze, enhance, compress, recognize, and synthesize speech — powering technologies like voice assistants (Alexa, Siri), automatic transcription, real-time translation, biometric voice authentication, and hands-free automotive control, all of which require a machine to “understand” or reproduce human speech rather than just carry it as a raw signal.

Q2. Distinguish between speech transmission and speech processing.

Answer: Speech transmission (e.g., a phone call) is only concerned with capturing a voice signal, compressing/encoding it with a codec (AMR, Opus), and carrying it over a network (GSM/VoLTE/VoIP) so the listener hears it clearly — there is no attempt to interpret the content. Speech processing, in contrast, extracts numerical features (MFCC, LPC) from the raw audio and uses them to recognize phonemes, words, or speaker identity — its goal is to *understand or act on* the speech content, not merely deliver it.

Q3. List out some application areas of digital speech processing.

Answer: Smart homes (voice assistants), education (captioning, pronunciation checking), healthcare (doctor-patient transcription, speech-aid devices), media (live broadcast captioning), business (speech-driven chatbots), automotive (voice navigation/infotainment), and security (voice biometric authentication).

Q4. Explain the stages of human speech generation with a schematic diagram.

Answer: Air from the **lungs** is pushed by the diaphragm through the trachea to the **vocal cords** in the larynx. Vibrating vocal cords create voiced excitation; open vocal cords with turbulent airflow create unvoiced excitation. This sound then enters the **pharyngeal cavity** (first resonator), and is further shaped by the **oral cavity** (tongue, lips, teeth, jaw). The **velum** determines whether air is directed only through the mouth (oral sounds) or also through the **nasal cavity** (nasal sounds like /m/, /n/, /ŋ/). Finally, the shaped sound is radiated as **speech output** from the mouth and/or nose.

Schematic chain: Lungs → Trachea → Vocal Cords → Pharyngeal Cavity → Oral/Nasal Cavity → Speech Output.

Q5. Explain the source-filter model of speech production. How do the vocal cords and the vocal tract act independently in shaping speech sounds?

Answer: The source-filter model treats speech production as two independent stages: (1) an **excitation source** $e[n]$ — either periodic pulses from vibrating vo-

cal cords (voiced) or noise-like turbulence from a constriction (unvoiced) — and (2) a **vocal tract filter** $h[n]$ that shapes the spectral envelope of this source via resonances (formants). The speech signal is the convolution $s[n] = e[n] * h[n]$. Because the source and filter are largely independent, the same excitation (e.g., vocal-cord vibration at a given pitch) can pass through different vocal-tract shapes to produce different vowels, and different excitations (voiced vs. unvoiced) can pass through a similar vocal-tract shape. This independence is exactly why cepstral analysis (Chapter 9) works: taking a logarithm converts this convolution into an addition, allowing the source and filter contributions to be separated.

Q6. What is the difference between voiced and voiceless (unvoiced) phonemes? Provide examples.

Answer: Voiced phonemes are produced with vibration of the vocal cords, giving them a periodic, harmonic structure (e.g., vowels /a/, /i/; consonants /b/, /d/, /g/, /z/, /v/). Voiceless (unvoiced) phonemes are produced without vocal cord vibration — air passes through an open glottis, producing turbulent, noise-like, aperiodic sound (e.g., /p/, /t/, /k/, /s/, /f/, /ʃ/). A simple physical test: place a finger on your throat; you feel a buzzing vibration for voiced sounds but not for voiceless ones.

Q7. Explain the psychoacoustic principles of the Mel-frequency scale. Why does a linear frequency representation fail to capture human auditory perception accurately?

Answer: Human perception of pitch is not linear with respect to physical frequency in Hz — humans perceive equal *ratios* of frequency as equal perceptual *steps* in pitch, especially at higher frequencies, and this psychoacoustic law is what the Mel scale captures: it is nearly linear below about 1 kHz and logarithmic above it. A linear Hz scale would treat the interval from 100 Hz to 200 Hz as perceptually equal to the interval from 10,000 Hz to 10,100 Hz, but humans perceive the first as a much bigger pitch jump than the second. The Mel-scale formula is $m = 2595 \log_{10} \left(1 + \frac{f}{700} \right)$, which compresses the higher-frequency axis to match how densely (or sparsely) the human cochlea actually resolves those frequencies. This is why Mel-based features (like MFCC) outperform linearly-spaced spectral features for speech recognition.

Q8. What is the difference between voiced, unvoiced, and silence regions in the context of human speech? How are voiced and unvoiced sounds produced?

Answer: **Voiced regions** have vibrating vocal cords producing quasi-periodic, high-energy signals (e.g., vowels). **Unvoiced regions** have no vocal-cord vibration; air is forced through a constriction producing noise-like, aperiodic, generally lower-energy signals (e.g., /s/, /f/). **Silence regions** have no significant excitation at all — very low energy, essentially background noise. Voiced sounds are produced by periodic vibration of the vocal folds modulating airflow from the lungs; unvoiced sounds are produced by forcing air through a narrow or complete constriction in the vocal tract without vocal-fold vibration, creating turbulent noise.

Q9. What is the fundamental frequency (F_0) of speech? What factors determine the fundamental frequency of a speaker's voice?

Answer: The fundamental frequency F_0 is the rate (in Hz) at which the vocal cords vibrate during voiced speech; it is perceived as the "pitch" of the voice. It is determined by: the length, mass, and tension of the vocal folds (longer/heavier folds vibrate slower, giving a lower F_0 — this is why adult male voices, on average, have lower F_0 than female or child voices), the subglottal air pressure from the lungs, and voluntary muscular control used for intonation (e.g., raising F_0 at the end of a question).

Q10. Explain the production of plosive, fricative, nasal, and approximant sounds.

Answer: Plosives (stops): complete closure of the vocal tract builds up air pressure, which is then released suddenly (e.g., /p/, /b/, /t/, /d/, /k/, /g/). **Fricatives:** a narrow (but not complete) constriction forces air through at high velocity, creating continuous turbulent noise (e.g., /f/, /v/, /s/, /z/). **Nasals:** the oral cavity is completely blocked but the velum is lowered, directing air through the nasal cavity, which resonates (e.g., /m/, /n/, /ŋ/). **Approximants (including semi-vowels):** articulators approach each other but do not create enough constriction for turbulence — the airflow remains smooth, similar to a vowel (e.g., /w/, /j/, /r/, /l/).

Q11. What are F1, F2, and F3 formants? Why is F1 generally related to vowel openness? Why is F2 useful for distinguishing front and back vowels?

Answer: F1, F2, F3 are the first three resonant-frequency peaks of the vocal tract for a given sound. F1 relates inversely to vowel openness/tongue height because an open vocal tract (low tongue, jaw dropped) has a larger effective cavity volume near the constriction, resonating at a higher F1; a close vowel with a raised tongue narrows this cavity, lowering F1. F2 relates to tongue frontness/backness because it depends on the length of the cavity behind the point of maximum constriction — front-tongue articulation shortens the back cavity (raising F2), while back-tongue articulation lengthens it (lowering F2). Together, F1 and F2 are sufficient to characterize most vowel identity, which is why the "vowel space" is usually plotted as an F1-F2 plane.

Q12. What is a speech signal? Is speech a periodic or aperiodic signal? Why is speech considered non-stationary? What is meant by quasi-stationarity of speech?

Answer: A speech signal is the time-varying acoustic waveform produced by the vocal apparatus, representing air-pressure variations over time. Speech is **neither purely periodic nor purely aperiodic** — voiced segments (vowels) are quasi-periodic (nearly, but not exactly, repeating, because the vocal folds don't vibrate with perfectly identical periods), while unvoiced segments (fricatives, stops) are aperiodic/noise-like. Speech is **non-stationary** because its statistical properties (amplitude, frequency content, energy) change continuously and rapidly over time as the speaker moves between different phonemes. However, over very short time windows (typically 20–30 ms), the vocal tract shape changes slowly enough that

the signal can be treated as approximately constant/periodic — this is called **quasi-stationarity**, and it justifies all short-time (frame-based) analysis techniques used throughout speech processing (framing, windowing, short-time Fourier transform, etc.).

Q13. What is the typical frequency range of human speech? What sampling frequency is required for speech processing? Why is 8 kHz commonly used in telephone speech, and why is 16 kHz commonly used in speech-processing applications?

Answer: Human speech energy is mostly concentrated between roughly 100 Hz and 8 kHz (fundamental frequency around 80–300 Hz plus several formants), though the full audible range for humans is 20 Hz–20 kHz. By the Nyquist theorem, the sampling frequency must be at least twice the highest frequency of interest. **8 kHz** is used in telephone speech because traditional telephone channels are band-limited to about 300–3400 Hz (the range needed for intelligibility, not full quality) — sampling at 8 kHz safely captures up to 4 kHz, matching this “narrowband” telephone channel. **16 kHz** is used in modern speech-processing/recognition applications because it captures up to 8 kHz, preserving much more of the higher formants and fricative energy (which often extends beyond 4 kHz), giving substantially better recognition accuracy — this is called “wideband” speech.

Q14. What is speech amplitude? What is speech intensity?

Answer: Speech amplitude is the instantaneous magnitude of the air-pressure variation (or the corresponding electrical signal from a microphone) at a given time instant — it directly determines the loudness we perceive at that moment. Speech intensity refers to the power of the sound wave (energy per unit time, often measured as amplitude squared, averaged over a short interval or expressed in decibels, dB) — it is a more perceptually meaningful, energy-based measure of loudness than the raw instantaneous amplitude.

Q15. Explain the reasoning behind the 44.1 kHz sampling frequency used by CD-quality microphones/recorders to capture the full spectrum of human hearing.

Answer: The human audible range extends up to about 20 kHz. By the Nyquist-Shannon sampling theorem, to faithfully capture (and later reconstruct) a signal containing frequencies up to f_{max} , we must sample at $f_s \geq 2f_{max}$. For $f_{max} = 20$ kHz, the minimum (Nyquist) rate is 40 kHz. The value 44.1 kHz was chosen (slightly above the bare minimum of 40 kHz) to leave a safety margin/transition band for the practical anti-aliasing filter to roll off before reaching the Nyquist frequency (22.05 kHz), ensuring no audible aliasing distortion while still capturing the entire human hearing range.

Q16. Define phoneme. Identify different types of phonemes.

Answer: A phoneme is the smallest unit of sound in a language that changes the meaning of a word when substituted (e.g., /p/ vs /b/ in “pat”/“bat”). The exact phoneme count is language- and linguist-dependent (English is commonly cited

as having 42-46 phonemes). Broad types of phonemes: **Vowels** (open vocal tract, voiced), **Diphthongs** (gliding vowels), **Semi-vowels/Glides** (vowel-like but function as consonants), and **Consonants**, which are further split into **Nasals** (/m,n,ŋ/), **Fricatives** (/f,v,s,z,ʃ,ʒ/), **Stops/Plosives** (/p,b,t,d,k,g/), and **Approximants** (/w,j,r,l/).

Q17. Define vowels. Write down/explain different types of vowels with suitable examples.

Answer: Vowels are voiced speech sounds produced with an open, unobstructed vocal tract, classified by tongue height (close/mid/open), tongue frontness (front/central/back), and lip rounding (rounded/unrounded). Examples: close front unrounded /i/ (as in "see"), open front unrounded /æ/ (as in "cat"), close back rounded /u/ (as in "boot"), open back /ɑ/ (as in "father"). Vowels are generally longer in duration than consonants and are more reliably recognized by both humans and ASR systems.

Q18. How are diphthongs and semi-vowels different from each other? List at least two examples of each.

Answer: A diphthong is a single vowel-nucleus sound that glides continuously between two vowel targets within one syllable (e.g., /aɪ/ in "bite," /aʊ/ in "house") — it is the syllable's vowel core. A semi-vowel (glide) sounds vowel-like but functions as a consonant, typically appearing at syllable edges rather than as the nucleus (e.g., /j/ in "yes," /w/ in "we").

Q19. Define consonants. List out and explain different types of consonants with suitable examples.

Answer: Consonants are speech sounds produced with some degree of constriction or closure in the vocal tract, obstructing airflow. Types: **Stops/Plosives** — complete closure then sudden release (/p,b,t,d,k,g/); **Fricatives** — narrow constriction creating turbulent noise (/f,v,s,z,ʃ,ʒ/); **Nasals** — oral closure with air routed through the nose (/m,n,ŋ/); **Approximants** — slight constriction without turbulence (/w,j,r,l/); further sub-classified by place of articulation as bilabial, dental, alveolar, palatal, velar stops, etc.

Q20. How is a syllable different from a phoneme? Identify the number of syllables in the word "spectrum."

Answer: A phoneme is the smallest distinctive sound unit; a syllable is a larger unit of pronunciation typically consisting of a vowel (the nucleus) optionally surrounded by consonants, and is the basic rhythmic unit of speech. A single syllable is built from multiple phonemes. The word "spectrum" is pronounced /ˈspek.trəm/, which has **2 syllables**: "spec" and "trum."

Q21. Write the importance of the velum in the speech production system.

Answer: The velum (soft palate) acts as a valve that determines the resonating path of the airflow. When raised, it seals off the nasal cavity, forcing air through the mouth only, producing oral sounds. When lowered, it opens a path into the nasal

cavity, producing nasal sounds (/m/, /n/, /ŋ/) with their characteristic nasal resonance (anti-resonances/zeros in the spectrum). Its precise, rapid movement is essential for correctly distinguishing nasal from oral sounds during continuous speech.

Q22. Identify different types of excitation sources in speech production.

Answer: **Voiced excitation** (periodic vibration of the vocal folds, producing voiced sounds), **Unvoiced excitation** (turbulent airflow through a total or partial constriction, producing unvoiced sounds), and **Mixed excitation** (a combination of both occurring simultaneously, producing mixed sounds such as voiced fricatives /v/, /z/).

Q23. Define F1-F2 Centroid. What purpose does it serve in the acoustic representation of speech sounds?

Answer: The F1-F2 centroid is the average (mean) position of the first and second formant frequencies for a given vowel, computed across many utterances/speakers. It serves as the single most representative reference point for that vowel in the F1-F2 acoustic space, useful for vowel classification, comparing pronunciation across speakers, and building reference templates for automatic speech/vowel recognition systems.

Q24. Define F1-F2 Cluster. What purpose does it serve in the acoustic representation of speech sounds?

Answer: The F1-F2 cluster is the scattered distribution/grouping of many individually measured F1, F2 formant value pairs for a given vowel, collected from many different speakers or utterances. It visually represents the natural variability in how the "same" vowel is pronounced by different people (due to differences in vocal-tract size, accent, etc.), while still forming a recognizable, bounded region distinct from other vowels' clusters — useful for understanding inter-speaker variability and for training statistical vowel classifiers.

Q25. The pronunciation of words differs when spoken in isolation compared to when spoken within continuous sentences. What challenges does this pose for ASR systems?

Answer: In continuous (connected) speech, phonemes are subject to **coarticulation** (a phoneme's pronunciation is influenced by neighboring phonemes), **reduction** (vowels/consonants are pronounced less distinctly or even dropped, e.g., "going to" → "gonna"), and **fast, variable speaking rate and prosody**, none of which occur when words are read in isolation with careful, clear articulation. This makes acoustic models trained on isolated-word data generalize poorly to continuous speech, requires the ASR system to model context-dependent phonemes (e.g., triphones), handle word-boundary ambiguity, and use language models to disambiguate what was actually said.

Q26. Outline the high-level architecture of a modern Automatic Speech Recognition (ASR) system.

Answer: A modern ASR system architecture: (1) **Front-end/Signal processing**

— pre-emphasis, framing, windowing; (2) **Feature extraction** — converting each frame into a feature vector (MFCC/LPCC etc.); (3) **Acoustic model** — maps feature vectors to phoneme/sub-word probabilities (traditionally GMM-HMM, now deep neural networks); (4) **Pronunciation lexicon** — maps words to sequences of phonemes; (5) **Language model** — gives the probability of word sequences, helping disambiguate acoustically similar options; (6) **Decoder/Search** — combines acoustic, lexicon, and language model scores (e.g., via the Viterbi algorithm) to output the most likely word sequence.

Q27. Describe the challenges of ASR due to background noise and room reverberation.

Answer: Background noise (additive noise from traffic, other speakers, machinery) corrupts the clean speech spectrum, reducing the accuracy of feature extraction and lowering recognition accuracy, especially at low signal-to-noise ratios. **Room reverberation** (sound reflecting off walls/objects) smears the signal in time, causing energy from earlier phonemes to overlap with later ones (temporal blurring), which distorts spectral and temporal features and is especially damaging to consonant recognition. Both effects mean models trained on clean, close-talking microphone data generalize poorly to real-world, far-field, noisy environments — motivating techniques like noise-robust features, dereverberation, multi-condition training, and deep-learning-based front-end denoising.

Chapter 4

Digital Representation of Speech Signals

4.1 Basic Signal Concepts

Real-life scenario: Every time you record a voice note, your phone is doing something remarkable: it's converting the continuous sound of your voice — which is really just air pressure wiggling up and down — into a long list of numbers that a computer can store, copy, and send over the internet. This chapter is about exactly how that conversion works.

A **time-series signal** is simply a quantity that changes with time, e.g., $s_1(t) = 10t$, $s_2(t) = 5t^2$. The simplest possible *periodic* (repeating) signal is a **sine wave** — the same smooth up-and-down wiggle you'd see if you shook a rope:

$$x(t) = A \sin(2\pi ft + \phi)$$

where $x(t)$ is the instantaneous value (amplitude) at time t , A is the **amplitude** (how “tall” the wave is, i.e., loudness), f is the **frequency** in Hz (how many wiggles per second, i.e., pitch), t is time in seconds, and ϕ is the **phase angle** (a left-right shift of the wave) in radians.

Here's a genuinely useful fact: **any** natural signal — including your voice — can be approximated, to arbitrary accuracy, as a **sum of many simple sine waves** added together, each with its own frequency, amplitude, and phase:

$$x(t) = \sum_{k=1}^{\infty} A_k \sin(2\pi f_k t + \phi_k)$$

Think of it like mixing paint: you can create almost any color by mixing a handful of primary colors in the right amounts. Similarly, you can build almost any sound by mixing sine waves of different pitches and loudness. This is the seed idea behind the Fourier Transform, discussed in Chapter 8.

4.1.1 Continuous-Time vs. Discrete-Time Signals

- A **continuous-time signal** is defined for *every* instant of time — like the smooth, unbroken sound wave coming out of your mouth: $x(t) = A \sin(\omega_c t + \phi)$, $-\infty < t < \infty$, where ω_c (angular frequency) = $2\pi f$ rad/s.
- A **discrete-time signal** is defined only at specific, evenly-spaced instants of time — like a strobe-light photograph of that same wave, taken at fixed intervals: $x[n] = A \sin(\omega n + \phi)$, $n = 0, \pm 1, \pm 2, \dots$, where ω is the **digital angular frequency** in rad/sample.

Q. Compare and contrast continuous-time and discrete-time signals. How is a continuous-valued signal different from a discrete-valued signal?

Answer: A continuous-time signal is defined at every instant of time (uncountably infinite time axis); a discrete-time signal is defined only at specific sample instants (an integer-indexed sequence). Separately, a *continuous-valued* signal can take any value within a range (infinite precision in amplitude); a *discrete-valued* signal can only take a finite set of allowed amplitude levels. A signal can therefore be continuous-time & continuous-valued (analog), discrete-time & continuous-valued (sampled but not quantized), or discrete-time & discrete-valued (fully digital).

4.2 Digitization: The Big Picture

Recall the microphone from Chapter 1 — it turns the pressure wave from your voice into an electrical signal. This electrical signal can be amplified and viewed on an oscilloscope; the resulting analog waveform is denoted $x_a(t)$ — a continuous-time signal.

The process of converting this analog signal into digital form is called **Analog-to-Digital (A/D) conversion**, performed by an **A/D converter (ADC)** — a chip inside every phone, laptop, and recording device. It involves three sequential sub-steps, like an assembly line:

$$x_a(t) \xrightarrow{\text{Sampler}} x[n] \xrightarrow{\text{Quantizer}} x_q[n] \xrightarrow{\text{Coder}} 01011 \dots$$

1. **Sampling:** Taking “snapshots” of the continuous-time signal at regular time instants (like a strobe light) — covered in the next section.
2. **Quantization:** Rounding each snapshot’s amplitude to the nearest allowed value from a finite set of levels.
3. **Coding:** Writing each rounded value as a b -bit binary number (0s and 1s) so it can actually be stored on a computer.

Block diagram of an A/D converter: $x_a(t) \rightarrow \text{Sampler} \rightarrow x(n) \rightarrow \text{Quantizer} \rightarrow x_q(n) \rightarrow \text{Coder} \rightarrow \text{digital bitstream}$.

4.3 Sampling

Sampling is the process of measuring the amplitude of an analog speech signal at regular time intervals to obtain a discrete-time (digital) representation — exactly like a

strobe light “freezing” a moving object at regular time intervals into a series of still frames. Sampling allows computers to perform operations on speech that would otherwise be impossible on a continuous analog signal.

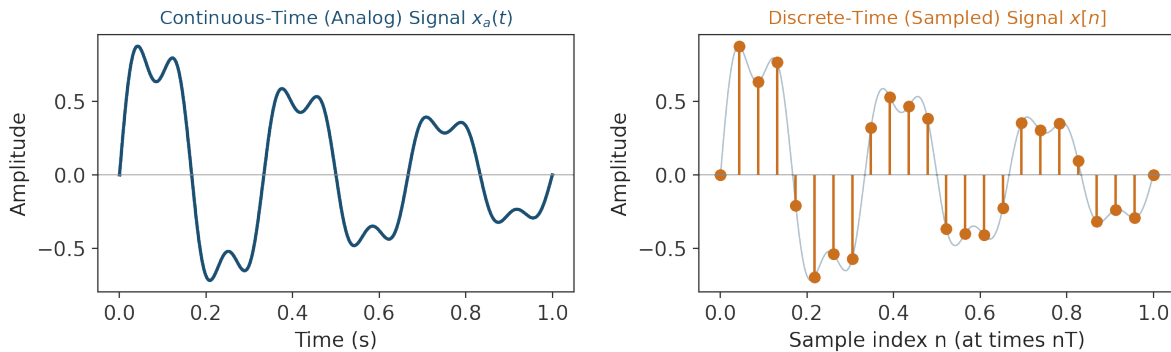


Figure 4.1: Sampling: turning a smooth analog wave into a series of discrete dots

The analog signal $x_a(t)$ is sampled at uniformly spaced time instants, with T being the **sampling interval**. The reciprocal of T , i.e., $1/T$, is called the **sampling frequency**, F_s . The output of the sampler is a series of numbers $x_a(nT)$, simplified in notation as:

$$x[n] = x_a(nT)$$

Sampling Terminologies:

- **Sampling Frequency (F_s):** number of samples taken every second, measured in Hertz (Hz). Common examples: 8000 Hz (telephone), 16000 Hz (speech recognition), 44100 Hz (music CDs).
- **Sampling Interval (T_s):** the time between two consecutive samples:

$$T_s = \frac{1}{F_s}$$

Worked Example: If $F_s = 8000$ Hz, then $T_s = \frac{1}{8000} = 0.000125$ s = 125 μ s — i.e., a new sample is captured every 125 microseconds, 8000 times every second.

4.4 The Nyquist-Shannon Sampling Theorem

This is one of the most important principles in all of digital signal processing — and it answers a very practical question: **“how fast do I need to sample so that I don’t lose any information?”**

“To perfectly reconstruct a continuous-time signal from its samples without distortion, the sampling rate must be at least twice the highest frequency present in the signal.”

$$f_s \geq 2 \times f_{max}$$

This minimum required rate, $2f_{max}$, is called the **Nyquist rate**. Half of the sampling frequency, $f_s/2$, is called the **Nyquist frequency**.

If you sample *too slowly* (below the Nyquist rate), something strange and irreversible happens: high-frequency wiggles in the original signal get mathematically “disguised” and reappear as a completely different, false, lower-frequency wave in your sampled data. This is called **aliasing**, and once it happens, there is no way to tell the difference between real low-frequency content and this fake aliased signal — the information is permanently scrambled.

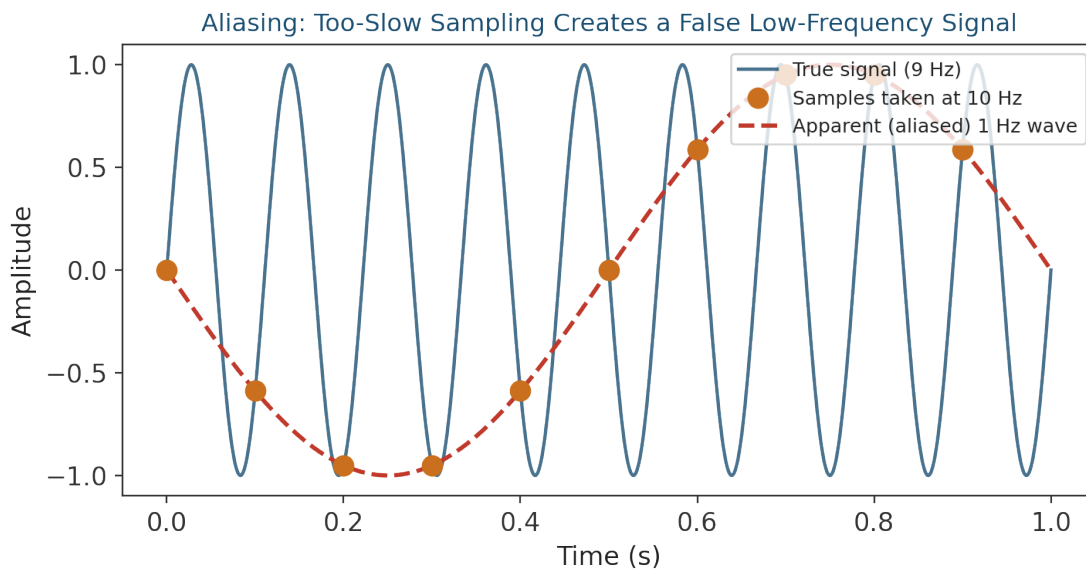


Figure 4.2: Aliasing: sample too slowly, and a fast wave “disguises” itself as a slow one

Real-life scenario: You’ve probably seen aliasing in a movie — a car’s or helicopter’s wheel/blades appear to spin backward or freeze, even though it’s actually spinning forward fast. This happens because the camera “samples” the scene at a fixed frame rate (like 24 frames per second), and if the wheel spins fast enough, it violates the Nyquist condition for the camera’s frame rate — a visual version of exactly the same aliasing that happens with audio sampling.

To prevent aliasing, an **anti-aliasing (low-pass) filter** is applied to the analog signal *before* sampling, to remove any frequency content above the Nyquist frequency, so there’s nothing left that could get disguised.

4.5 Quantization

Sampling only fixes the *time* axis (when we measure). But each measurement still has an amplitude that could, in principle, be any real number (infinite precision) — a computer cannot store infinite precision, so we must also round each sample’s amplitude to one of a fixed set of allowed values. This rounding step is called **quantization**, and it is

necessary because digital systems can only store/represent a finite number of distinct amplitude values (limited by the number of bits available).

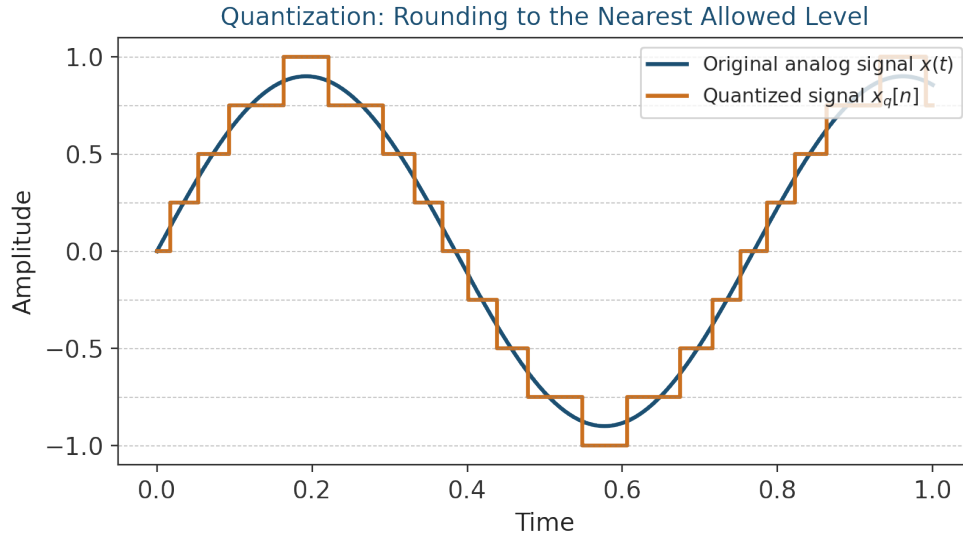


Figure 4.3: Quantization: rounding each sample to the nearest allowed “staircase” level

4.5.1 Quantization Step Size (QSS)

The **quantization step size**, Δ , is the difference in amplitude between two successive quantization levels:

$$\Delta = \frac{V_{max} - V_{min}}{L}$$

where V_{max} = maximum value of the signal, V_{min} = minimum value of the signal, and $L = 2^N$ is the number of quantization levels, with N = number of bits per sample.

4.5.2 Quantization Error

Quantization error is the difference between the original (unquantized) sample value and its quantized (rounded) value:

$$e[n] = x[n] - x_q[n]$$

For a uniform rounding quantizer, this error is always bounded: $-\frac{\Delta}{2} \leq e[n] \leq \frac{\Delta}{2}$.

- **Maximum Quantization Error:** $e_{max} = \frac{\Delta}{2}$

- **Mean Squared Quantization Error** (assuming the error is uniformly distributed over $[-\Delta/2, \Delta/2]$):

$$\sigma_e^2 = \frac{\Delta^2}{12}$$

4.6 Bit Rate and File Storage

Real-life scenario: Ever wondered why a WhatsApp voice note takes up so little storage compared to a music file? It comes down to exactly this formula. Voice notes are typically sampled at a lower rate (e.g., 16 kHz) with fewer bits than CD-quality music (44.1 kHz, 16-bit stereo) — less data captured per second means a smaller file, at the cost of slightly lower fidelity (which is fine for speech, since we don't need music-level quality to understand words).

Once a signal has been sampled (F_s samples/sec) and quantized (N bits/sample), the total data requirements can be computed as:

$$\text{File Size (bits)} = F_s \times N \times T \quad \text{Bit Rate (bps)} = F_s \times N$$

where F_s = sampling frequency (samples per second), N = bits per sample (quantization resolution), and T = duration of recording in seconds.

4.7 Worked Numerical Examples — Chapter 4

Example 1. A speech signal is uniformly quantized using **8 bits per sample** over a range of ± 2 V. Find the quantization step size (Δ). If the sampling frequency is 10 Hz, what is the size of the file for a 10-second recording?

Solution:

Step 1 — Number of levels: $L = 2^N = 2^8 = 256$.

Step 2 — Range: $\text{Range} = V_{\max} - V_{\min} = 2 - (-2) = 4$ V.

Step 3 — Step size:

$$\Delta = \frac{\text{Range}}{L} = \frac{4}{256} = 0.015625 \text{ V} \approx \mathbf{0.0156 \text{ V}}$$

Step 4 — File size: $F_s = 10$ Hz means 10 samples/sec. Each sample = 8 bits = 1 byte.

$$\text{Data per second} = 10 \times 1 \text{ byte} = 10 \text{ bytes/sec}$$

For a 10-second recording: $10 \times 10 = \mathbf{100 \text{ bytes}}$.

Example 2. An 8-bit ADC has an input range of -1 V to $+1$ V. Find the Quantization Step Size, Maximum Quantization Error, and Mean Squared Quantization Error.

Solution:

Step 1: $N = 8 \Rightarrow L = 2^8 = 256$.

Step 2: Voltage range = $1 - (-1) = 2$ V.

Step 3 — QSS:

$$\Delta = \frac{2}{256} = 0.0078125 \text{ V}$$

Step 4 — Maximum Quantization Error:

$$e_{\max} = \frac{\Delta}{2} = \frac{0.0078125}{2} = 0.00390625 \text{ V}$$

Step 5 — Mean Squared Quantization Error:

$$\sigma_e^2 = \frac{\Delta^2}{12} = \frac{(0.0078125)^2}{12} \approx 5.09 \times 10^{-6} \text{ V}^2$$

Example 3 (from Question Bank). A speech signal has a maximum frequency of 3 kHz. It is sampled at 8 kHz using 16-bit uniform quantization. Find the Nyquist rate and check if the sampling frequency is sufficient. If the signal amplitude range is ± 2 V, find the quantization step size and maximum quantization error.

Solution:

Step 1 — Nyquist rate: $2 \times f_{\max} = 2 \times 3 \text{ kHz} = 6 \text{ kHz}$.

Step 2 — Check sufficiency: Given $f_s = 8 \text{ kHz} > 6 \text{ kHz}$ (Nyquist rate) $\Rightarrow$ **sampling frequency is sufficient**, no aliasing will occur.

Step 3 — Quantization step size: $N = 16 \Rightarrow L = 2^{16} = 65536$. Range = $2 - (-2) = 4$

V.

$$\Delta = \frac{4}{65536} \approx 6.104 \times 10^{-5} \text{ V}$$

Step 4 — Max quantization error: $e_{max} = \Delta/2 \approx 3.052 \times 10^{-5} \text{ V}$.

Example 4 (from Question Bank). A speech signal is uniformly quantized using 8 bits per sample over a range of $\pm 2 \text{ V}$. If the sampling frequency is 16 kHz, determine the quantization step size (Δ). Also find the file size for a 5-second recording.

Solution:

$L = 2^8 = 256$; Range = 4 V $\Rightarrow \Delta = 4/256 = 0.015625 \text{ V}$.

File size (bits) = $F_s \times N \times T = 16000 \times 8 \times 5 = 640,000 \text{ bits} = 80,000 \text{ bytes} = 78.125 \text{ KB}$.

Example 5 (from Question Bank). A speech signal has an amplitude range of -1.5 V to $+1.5 \text{ V}$ and is quantized using 10 bits per sample. Sampling frequency is 16 kHz. For a 20-second recording, determine: (a) quantization step size, (b) total number of samples, (c) total number of bits required, (d) approximate file size in KB.

Solution:

(a) $L = 2^{10} = 1024$; Range = $1.5 - (-1.5) = 3 \text{ V}$.

$$\Delta = \frac{3}{1024} \approx 0.00293 \text{ V}$$

(b) Total samples = $F_s \times T = 16000 \times 20 = 320,000 \text{ samples}$.

(c) Total bits = samples $\times$ bits/sample = $320,000 \times 10 = 3,200,000 \text{ bits}$.

(d) File size = $3,200,000/8 = 400,000 \text{ bytes} = 400,000/1024 \approx \mathbf{390.6 \text{ KB}}$.

Example 6 (from Question Bank). A 10-bit ADC converts analog signals in the range -5 V to $+5 \text{ V}$. Determine the Quantization step size, maximum quantization error, and mean squared quantization error.

Solution:

$L = 2^{10} = 1024$; Range = 10 V.

$$\Delta = \frac{10}{1024} \approx 0.009766 \text{ V}$$

$$e_{max} = \frac{\Delta}{2} \approx 0.004883 \text{ V} \quad \sigma_e^2 = \frac{\Delta^2}{12} \approx 7.95 \times 10^{-6} \text{ V}^2$$

Example 7 (from Question Bank). A speech signal is uniformly quantized using 6 bits per sample over a range of $\pm 3 \text{ V}$. Find the quantization step size.

Solution:

$L = 2^6 = 64$; Range = 6 V.

$$\Delta = \frac{6}{64} = 0.09375 \text{ V}$$

Example 8 (from Question Bank). A speech acquisition system samples a signal at 16 kHz using a 10-bit ADC over the input range -1.5 V to $+1.5$ V. Calculate (a) quantization step size, (b) maximum quantization error, (c) mean squared quantization error.

Solution: (Same setup as Example 5(a) but focusing on error terms.)

$$L = 2^{10} = 1024; \text{ Range} = 3 \text{ V} \Rightarrow \Delta = 3/1024 \approx 0.00293 \text{ V.}$$

$$e_{max} = \Delta/2 \approx 0.001465 \text{ V.} \quad \sigma_e^2 = \Delta^2/12 \approx 7.15 \times 10^{-7} \text{ V}^2.$$

Note: the sampling frequency (16 kHz) does not affect the quantization step size or error — those depend only on amplitude range and number of bits. Sampling frequency only affects the number of samples/bit-rate/file size.

Example 9 (Filter-bank compression — from Ch-4 slides, also relevant background for later chapters). A narrowband speech signal is sampled at 8 kHz with 8-bit precision. This signal is processed through a 10-channel filter bank, where each channel output is sampled at 100 Hz and quantized with 6-bit precision. Compute (i) the raw bit rate, (ii) the compressed bit rate, (iii) the compression ratio.

Solution:

(i) *Raw bit rate:* $R_{org} = F_s \times \text{precision} = 8000 \times 8 = 64,000 \text{ bps} = 64 \text{ kbps.}$

(ii) *Compressed bit rate:* Each channel produces $100 \times 6 = 600$ bits/sec. For 10 channels: $R_{compressed} = 10 \times 600 = 6000 \text{ bps} = 6 \text{ kbps.}$

(iii) *Compression ratio:*

$$\frac{R_{org}}{R_{compressed}} = \frac{64}{6} = \mathbf{10.67 : 1}$$

4.8 Chapter 4 : Practice / Exam Questions

Q1. Define digitization. Write the purpose of the analog-to-digital converter in digitization.

Answer: Digitization is the process of converting a continuous-time, continuous-valued (analog) signal into a discrete-time, discrete-valued (digital) signal, via sampling, quantization, and coding. The A/D converter performs exactly this conversion so that a computer — which can only store and process discrete numbers — can capture, analyze, transmit, and reconstruct the original analog speech waveform.

Q2. Define sampling frequency. How is it related to the sampling interval? For a given signal sampled at 16 kHz, determine the sampling interval.

Answer: Sampling frequency F_s is the number of samples taken per second (in Hz). It is the reciprocal of the sampling interval T_s : $T_s = 1/F_s$. For $F_s = 16,000$ Hz: $T_s = 1/16000 = 6.25 \times 10^{-5}$ s = 62.5 μ s.

Q3. Why must the sampling rate not be below twice the highest frequency present in the signal? How is the Nyquist rate related to the Nyquist frequency?

Answer: If the sampling rate is below twice the highest signal frequency, higher frequency components cannot be uniquely represented by the samples — they get “folded back” onto lower frequencies (aliasing), and the original continuous signal can no longer be uniquely reconstructed from its samples. The **Nyquist rate** ($2f_{max}$) is the *minimum* sampling rate required to avoid aliasing for a given signal; the **Nyquist frequency** ($f_s/2$) is *half the actual sampling frequency being used*, representing the highest frequency that can be unambiguously represented at that sampling rate. For alias-free sampling, the signal’s maximum frequency must not exceed the Nyquist frequency.

Q4. Define aliasing. Write its effect and list ways to avoid it in signal processing.

Answer: Aliasing is the distortion that occurs when a signal is sampled below its Nyquist rate, causing high-frequency components to be misrepresented as (indistinguishable from) lower-frequency components in the sampled signal. Effect: permanent, irreversible loss/corruption of information — the original signal cannot be correctly reconstructed. Ways to avoid it: (1) sample at or above the Nyquist rate; (2) apply an anti-aliasing low-pass filter before sampling to remove frequency content above the Nyquist frequency; (3) oversample and then digitally filter/downsample.

Q5. Define quantization. Why is it a significant step in signal processing? Define quantization step size and quantization error. How does quantization step size affect mean squared quantization error?

Answer: Quantization is the process of mapping the continuous range of amplitude values of a sampled signal onto a finite set of discrete levels. It is significant because digital systems have finite storage/precision (a fixed number of bits per sam-

ple) and quantization is what makes true digital (as opposed to merely discrete-time) representation possible — every real-world ADC must quantize. Quantization step size $\Delta = (V_{max} - V_{min})/L$ is the amplitude gap between adjacent quantization levels. Quantization error $e[n] = x[n] - x_q[n]$ is the rounding error introduced by this process. Since mean squared quantization error $\sigma_e^2 = \Delta^2/12$, it is proportional to Δ^2 — halving the step size (i.e., adding one more bit, doubling L) reduces the mean squared error by a factor of 4, i.e., roughly 6 dB improvement in signal-to-quantization-noise ratio per extra bit.

Q6. Suppose the ADC is modified to operate with 12-bit resolution instead of 10-bit resolution. Discuss how this change affects the ADC's operation.

Answer: Increasing resolution from 10 to 12 bits increases the number of quantization levels from $2^{10} = 1024$ to $2^{12} = 4096$ (a 4x increase), which *decreases* the quantization step size Δ by a factor of 4 for the same voltage range, and correspondingly decreases the maximum quantization error and mean squared quantization error (each by a factor of 4, i.e., roughly 12 dB better signal-to-quantization-noise ratio). The trade-off is that each sample now requires more bits to store/transmit, increasing the bit rate and file size proportionally (by $12/10 = 1.2\times$), and requiring more capable hardware.

Q7. Discuss the time-domain representation of a speech signal. What information can be gathered from it?

Answer: The time-domain representation plots the signal's amplitude directly against time. From it we can gather: overall loudness/energy variation over time, the location of silence vs. active speech regions, rough duration and timing of phonemes/words, and whether a segment is likely voiced (regular, high-amplitude, quasi-periodic waveform) or unvoiced (noise-like, lower and irregular amplitude). However, it hides detailed frequency-domain information such as pitch, formant locations, and harmonic structure, which is why frequency-domain analysis is also essential.

Q8. Define amplitude, frequency, and phase for a sinusoidal signal.

Answer: For $x(t) = A \sin(2\pi ft + \phi)$: **Amplitude** A is the maximum magnitude (peak value) the signal attains. **Frequency** f is the number of complete oscillation cycles per second (Hz). **Phase** ϕ is the angular offset (in radians) that shifts the waveform earlier or later in time relative to a reference sine wave starting at $t = 0$.

Q9. "A speech signal is considered non-stationary and quasi-periodic in nature." Write the significance of this statement.

Answer: This statement means that a speech signal's statistical/spectral characteristics change continuously over time (non-stationary — hence it cannot be analyzed using a single, global Fourier transform), yet within a very short time window (20-30 ms) voiced segments repeat in a nearly (but not perfectly) periodic manner (quasi-periodic). Its significance is that it justifies and necessitates **short-time (frame-based) analysis**: speech must be broken into short overlapping frames, each

treated as approximately stationary, before frequency-domain or feature-extraction techniques (like the DFT, LPC, or MFCC) can be meaningfully applied.

Chapter 5

Time-Domain and Frequency-Domain Analysis; LTI Systems

5.1 Time-Domain vs. Frequency-Domain Representation

Real-life scenario: Think of a graphic equalizer on a music app — those bars that bounce up and down for “bass,” “mid,” and “treble.” That display is a frequency-domain view of the song: it’s telling you how much energy is present at low, medium, and high frequencies right now. Compare that to the plain waveform you see when editing an audio clip (just a wiggly line going up and down) — that’s the time-domain view. Both are the *same* sound, just viewed through two different lenses.

The **time-domain representation** of a signal plots amplitude vs. time — it directly shows *when* things happen (loudness changes, silence, timing), but it hides *what frequencies* make up the signal (pitch, formants, harmonics).

The **frequency-domain representation** plots amplitude/energy vs. frequency — it directly reveals *what* frequencies are present and their relative strengths, which is essential because speech carries almost all of its phonetic identity in its frequency (spectral) content, not simply in its raw time-domain shape.

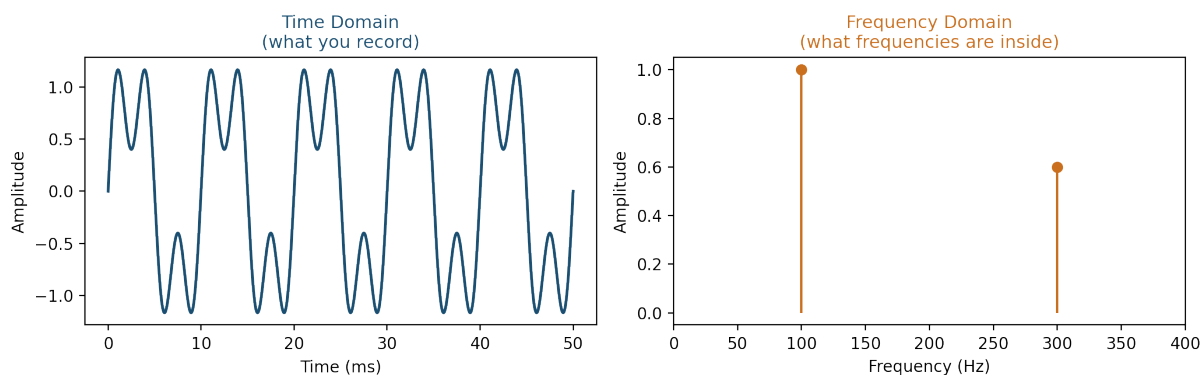


Figure 5.1: Time domain (when things happen) vs. frequency domain (what pitches are present)

While the time-domain shows *when* a signal changes, the frequency-domain shows *what frequencies* make up the signal. Since speech is non-stationary, we typically compute frequency content over short overlapping windows, giving rise to the **spectrogram** — a 2-D time-frequency representation (time on x-axis, frequency on y-axis, and color/intensity representing energy).

Significance of Spectrograms: A spectrogram shows how the frequency content of a signal evolves over time — like a video of the spectrum, instead of just one snapshot. It reveals formant tracks (visible as bright/dark bands that move over time as vowels change), voiced/unvoiced boundaries, pitch harmonics (visible as fine horizontal striations for voiced speech), and is used extensively in phonetics research, speech recognition front-ends, speaker identification, and diagnosing pronunciation errors in language-learning applications.

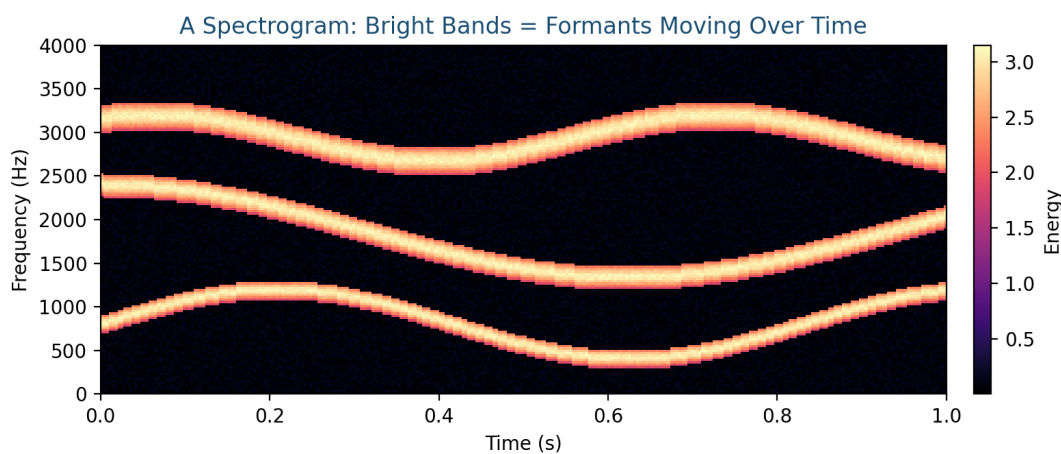


Figure 5.2: A spectrogram: the bright, moving bands are formants tracking a changing vowel

Voiced vs. Unvoiced, in the time domain: even without any frequency analysis, voiced and unvoiced sounds already look very different when you simply plot amplitude vs. time — voiced sounds show a regular, repeating pattern (because the vocal cords vibrate periodically), while unvoiced sounds look like random noise (because they're generated by turbulent airflow, not vibration).

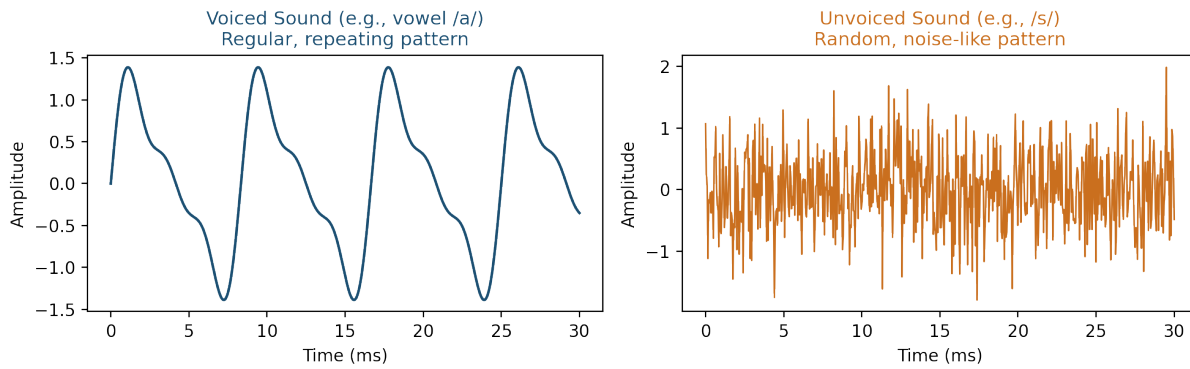


Figure 5.3: Voiced sounds repeat regularly; unvoiced sounds look like random noise

5.2 Discrete-Time Systems and LTI Systems

A **discrete-time system** is anything that takes an input sequence $x[n]$ and transforms it into an output sequence $y[n]$: $y[n] = T(x[n])$. Example: a microphone converts acoustic speech into an electrical signal; in speech processing, discrete-time systems are used to model the vocal-tract filter.

A **Linear Time-Invariant (LTI) system** is a discrete-time system that satisfies **two** properties — think of these as a “fairness contract” the system must obey:

5.2.1 (a) Linearity — the Principle of Superposition

A system is linear if it satisfies both:

- **Scalability (Homogeneity):** if $y[n] = T(x[n])$, then $T(ax[n]) = aT(x[n])$ — scaling the input scales the output by the same factor (double the input, double the output, no surprises).
- **Additivity:** if $y_1[n] = T(x_1[n])$ and $y_2[n] = T(x_2[n])$, then $T(x_1[n] + x_2[n]) = y_1[n] + y_2[n]$ (feed in two signals together, and you get exactly the sum of what each would have produced alone).

Combined (superposition), for arbitrary constants a, b :

$$T(ax_1[n] + bx_2[n]) = aT(x_1[n]) + bT(x_2[n]) = ay_1[n] + by_2[n]$$

5.2.2 (b) Time Invariance

A system is time-invariant if its behaviour does not change over time: if $y[n] = T(x[n])$, then for a delayed input $x[n - n_0]$ (where $n_0 > 0$), the output should be the correspondingly delayed $y[n - n_0]$. In other words, the system produces the same output shape whether the input is applied “today” or “tomorrow” — only the time location of the output shifts, not its shape.

Why LTI systems matter in speech processing: The human vocal tract, microphones, and communication channels can all be reasonably approximated as LTI systems over

short time intervals, which lets us use the powerful mathematical machinery of convolution, Fourier/Z-transforms, and transfer functions to analyze and model them.

5.3 Worked Examples — LTI Properties

Example 1: Testing Time-Invariance. Consider the transformation $y[n] = x[n] + x[n - 1]$. Check whether this system is time-invariant, using input $x[n] = \{1, 2, 3\}$ and a shift of $n_0 = 1$.

Solution:

Step 1 — Original output: $y[n] = x[n] + x[n - 1] = \{1 + 0, 2 + 1, 3 + 2\} = \{1, 3, 5\}$.

Step 2 — Shift the input by $n_0 = 1$: the shifted input is $x[n - 1] = \{0, 1, 2\}$.

Step 3 — Compute R.H.S. — apply the system to the shifted input:

$$T(x[n - 1]) = T(\{0, 1, 2\}) = \{0 + 0, 1 + 0, 2 + 1\} = \{0, 1, 3\}$$

Step 4 — Compare with L.H.S. — the correspondingly shifted original output $y[n - 1]$, which is $\{1, 3, 5\}$ shifted right by one position = $\{0, 1, 3, 5\}$ truncated to the same length as the R.H.S., i.e. $\{0, 1, 3\}$.

Since $T(x[n - 1]) = y[n - 1] = \{0, 1, 3\}$ for the overlapping samples, the system **is time-invariant**.

Example 2: Testing Linearity (Superposition). Consider $y[n] = T(x[n]) = 2 \cdot x[n]$. Let $x_1[n] = \{1, 2\}$, $x_2[n] = \{3, 4\}$, and take $a = 3$, $b = 2$. Verify the superposition property.

Solution:

Step 1: $y_1[n] = 2x_1[n] = \{2, 4\}$; $y_2[n] = 2x_2[n] = \{6, 8\}$.

Step 2 — L.H.S.:

$$T(3x_1[n] + 2x_2[n]) = T(\{3 \cdot 1 + 2 \cdot 3, 3 \cdot 2 + 2 \cdot 4\}) = T(\{9, 14\}) = \{18, 28\}$$

Step 3 — R.H.S.:

$$3y_1[n] + 2y_2[n] = 3\{2, 4\} + 2\{6, 8\} = \{6, 12\} + \{12, 16\} = \{18, 28\}$$

Since L.H.S. = R.H.S. = $\{18, 28\}$, the system **is linear**.

Example 3 (Question Bank). Consider the system $y[n] = x[n] + 2x[n - 2]$. For input $x[n] = \{2, 4, 6\}$, check whether the system is time-invariant.

Solution:

Original output: $y[n] = x[n] + 2x[n - 2] = \{2 + 0, 4 + 0, 6 + 2(2)\} = \{2, 4, 10\}$.

Shift input by 1: $x[n - 1] = \{0, 2, 4, 6\}$. Apply system: $T(x[n - 1])[n] = x[n - 1] + 2x[n - 3]$. Evaluating at $n = 0, 1, 2, 3$: $\{0 + 0, 2 + 0, 4 + 0, 6 + 2(2)\} = \{0, 2, 4, 10\}$.

Compare to $y[n - 1]$, i.e., $\{2, 4, 10\}$ shifted right by 1 = $\{0, 2, 4, 10\}$. These match exactly, so the system **is time-invariant**.

Chapter 6

Convolution in the Time Domain

Real-life scenario: Ever noticed how your voice sounds different (echoey, boomy, or muffled) depending on whether you're in a bathroom, an empty hall, or a small carpeted room? That's because each room has its own "impulse response" — a unique acoustic fingerprint. Your voice (input) gets combined with the room's characteristics (impulse response) through exactly the mathematical operation this section explains: convolution. This is also exactly how digital "reverb" effects in music software work — they convolve your dry vocal with the impulse response of a real concert hall to make it sound like you're singing there.

6.1 Impulse Decomposition

Here's a clever trick: any discrete-time signal can be broken down into a sum of tiny, individual "spikes" (impulses), each scaled and placed at the right time. Mathematically, any signal $x[n]$ can be written as a weighted sum of delayed unit impulses:

$$x[n] = \sum_{k=-\infty}^{\infty} x[k] \delta[n - k]$$

where the **unit impulse** is defined as $\delta[n] = 1$ if $n = 0$, and 0 otherwise (a single spike of height 1 at exactly one instant, and zero everywhere else).

Example: Impulse Decomposition. Suppose $x[n] = \{3, 4, 5\}$, i.e., $x[0] = 3, x[1] = 4, x[2] = 5$. Write $x[n]$ as a sum of impulses and verify.

Solution:

$$x[n] = 3\delta[n] + 4\delta[n - 1] + 5\delta[n - 2]$$

For $n = 0$: $3\delta[0] + 4\delta[-1] + 5\delta[-2] = 3(1) + 4(0) + 5(0) = 3$. For $n = 1$: $3\delta[1] + 4\delta[0] + 5\delta[-1] = 0 + 4(1) + 0 = 4$. For $n = 2$: $3\delta[2] + 4\delta[1] + 5\delta[0] = 0 + 0 + 5(1) = 5$.

The signal is successfully reconstructed as $\{3, 4, 5\}$.

6.2 Impulse Response and Convolution

If we know how a system responds to a single, tiny "spike" input, do we know everything about that system? Remarkably, **yes** — for an LTI system. The **impulse response** $h[n]$

of a system is defined as the system's output when the input is a unit impulse $\delta[n]$. This is the single most important property of LTI systems: they can be **completely characterized** by their impulse response.

Given an LTI system, the output $y[n]$ for any input $x[n]$ is given by the **sum of weighted and delayed impulse responses**:

$$y[n] = \sum_{k=-\infty}^{\infty} x[k] h[n-k] = x[n] * h[n]$$

This operation is called the **convolution** of $x[n]$ with $h[n]$. Think of it like this: break the input into tiny spikes (as we just did above), let the system respond to each spike individually (each response is a scaled, shifted copy of $h[n]$), and then simply add up all those individual responses. That sum is the convolution. This process is also called **filtering** the input with the system's impulse response, and it is fundamental to modeling how speech production and processing systems work.

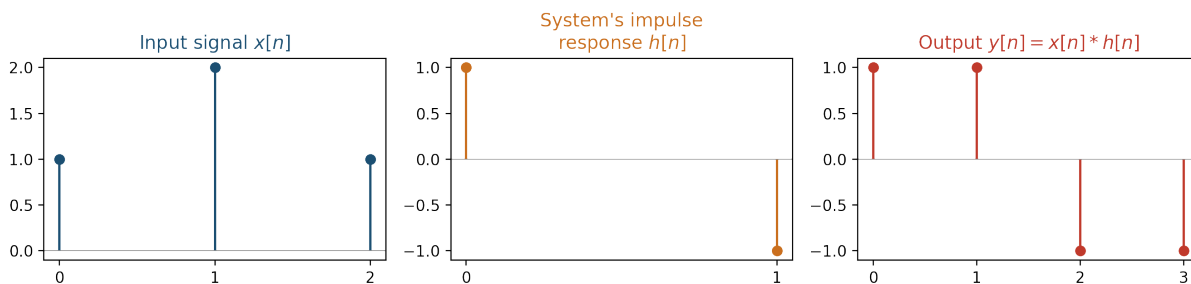


Figure 6.1: Convolution: combining an input signal with a system's impulse response

6.2.1 Properties of Convolution

- **Commutative:** $x[n] * h[n] = h[n] * x[n]$ — whether we treat $x[n]$ as the input or $h[n]$ as the input, the output is the same.
- **Associative:** $x[n] * (h[n] * g[n]) = (x[n] * h[n]) * g[n]$.
- **Distributive:** $x[n] * (h_1[n] + h_2[n]) = x[n] * h_1[n] + x[n] * h_2[n]$.
- **Shift Property:** if $x[n - n_0]$ is a shifted version of $x[n]$, then $x[n - n_0] * h[n] = y[n - n_0]$ (time-invariance is preserved).
- **Impulse Property:** $x[n] * \delta[n] = x[n]$ — convolving with a unit impulse leaves the signal unchanged.
- **Scaling Property:** $(a \cdot x[n]) * h[n] = a \cdot (x[n] * h[n])$.
- **Finite Duration Property:** if $x[n]$ has length N_1 and $h[n]$ has length N_2 , then the output $y[n]$ has length $N_1 + N_2 - 1$.

6.3 Limitations of Time-Domain Convolution

- **Computationally heavy:** requires summing many shifted and scaled versions of signals — grows expensive for long signals.

- **Limited insight into filtering:** it is difficult to directly visualize how a filter will affect frequency content; frequency-domain magnitude/phase plots are far more intuitive.
- **No direct spectral interpretation:** bandwidth, resonance, and spectral energy cannot easily be measured from a time-domain impulse response alone — Fourier or Z-transform analysis is needed.
- **Harder system characterization:** poles, zeros, and stability are not visible from the time-domain impulse response directly; frequency-domain (or Z-domain) analysis simplifies system design considerably.
- **Poor frequency visibility:** complex speech/music signals look irregular and cluttered in the time domain, making it hard to identify harmonics or formants directly.

6.4 Worked Numerical Examples — Convolution

Example 1. Using convolution, find $y[n]$ for $x[n] = \{1, 2, 1\}$ (at $n = 0, 1, 2$) and $h[n] = \{1, -1\}$ (at $n = 0, 1$).

Solution:

Step 1 — Output length: $\text{len}(y) = \text{len}(x) + \text{len}(h) - 1 = 3 + 2 - 1 = 4$, i.e., we need $y[0], y[1], y[2], y[3]$.

Step 2 — Compute each output sample using $y[n] = \sum_k x[k]h[n-k]$ (equivalently, “flip-and-slide” h across x , multiply overlapping terms, and sum):

$$y[0] = x[0]h[0] = 1 \times 1 = 1$$

$$y[1] = x[0]h[1] + x[1]h[0] = 1(-1) + 2(1) = -1 + 2 = 1$$

$$y[2] = x[1]h[1] + x[2]h[0] = 2(-1) + 1(1) = -2 + 1 = -1$$

$$y[3] = x[2]h[1] = 1(-1) = -1$$

Result: $y[n] = \{1, 1, -1, -1\}$.

Example 2 (Question Bank). $x[n] = \{2, 1, 2, 4, 3\}$ ($n = 0..4$), $h[n] = \{1, -1, 2\}$ ($n = 0..2$). Find $y[n]$.

Solution: Output length = $5 + 3 - 1 = 7$.

$$y[0] = x[0]h[0] = 2(1) = 2$$

$$y[1] = x[0]h[1] + x[1]h[0] = 2(-1) + 1(1) = -2 + 1 = -1$$

$$y[2] = x[0]h[2] + x[1]h[1] + x[2]h[0] = 2(2) + 1(-1) + 2(1) = 4 - 1 + 2 = 5$$

$$y[3] = x[1]h[2] + x[2]h[1] + x[3]h[0] = 1(2) + 2(-1) + 4(1) = 2 - 2 + 4 = 4$$

$$y[4] = x[2]h[2] + x[3]h[1] + x[4]h[0] = 2(2) + 4(-1) + 3(1) = 4 - 4 + 3 = 3$$

$$y[5] = x[3]h[2] + x[4]h[1] = 4(2) + 3(-1) = 8 - 3 = 5$$

$$y[6] = x[4]h[2] = 3(2) = 6$$

Result: $y[n] = \{2, -1, 5, 4, 3, 5, 6\}$.

Example 3 (Question Bank). $x[n] = \{1, 3, 2, 1\}$, $h[n] = \{2, -1\}$. Find $y[n]$.

Solution: Output length = $4 + 2 - 1 = 5$.

$$y[0] = 1(2) = 2$$

$$y[1] = 1(-1) + 3(2) = -1 + 6 = 5$$

$$y[2] = 3(-1) + 2(2) = -3 + 4 = 1$$

$$y[3] = 2(-1) + 1(2) = -2 + 2 = 0$$

$$y[4] = 1(-1) = -1$$

Result: $y[n] = \{2, 5, 1, 0, -1\}$.

Example 4 (Question Bank). Express $x[n] = \{3, 4, 5\}$ as a sum of impulses and use convolution with $h[n] = \{1, 2\}$ to find $y[n]$.

Solution: $x[n] = 3\delta[n] + 4\delta[n-1] + 5\delta[n-2]$ (shown earlier). Convoluting with $h[n] = \{1, 2\}$, output length = $3 + 2 - 1 = 4$.

$$y[0] = 3(1) = 3; \quad y[1] = 3(2) + 4(1) = 6 + 4 = 10; \quad y[2] = 4(2) + 5(1) = 8 + 5 = 13;$$

$$y[3] = 5(2) = 10.$$

Result: $y[n] = \{3, 10, 13, 10\}$.

Example 5 (Question Bank — Commutative Property Check). $x[n] = \{2, -1, 3\}$, $h[n] = \{1, -2, 1\}$. Compute $y[n]$ and verify $x * h = h * x$.

Solution: Output length $= 3 + 3 - 1 = 5$.

$$y[0] = 2(1) = 2$$

$$y[1] = 2(-2) + (-1)(1) = -4 - 1 = -5$$

$$y[2] = 2(1) + (-1)(-2) + 3(1) = 2 + 2 + 3 = 7$$

$$y[3] = (-1)(1) + 3(-2) = -1 - 6 = -7$$

$$y[4] = 3(1) = 3$$

Result: $y[n] = \{2, -5, 7, -7, 3\}$. Repeating the computation with the roles of $x[n]$ and $h[n]$ swapped gives the identical sequence $\{2, -5, 7, -7, 3\}$, confirming the **commutative property**: $x[n] * h[n] = h[n] * x[n]$.

Example 6 (Question Bank). $x[n] = \{1, 2, 1, 3\}$, $h[n] = \{1, 0, -1\}$. Determine output length and compute all $y[n]$.

Solution: Output length $= 4 + 3 - 1 = 6$.

$$y[0] = 1(1) = 1$$

$$y[1] = 1(0) + 2(1) = 2$$

$$y[2] = 1(-1) + 2(0) + 1(1) = -1 + 0 + 1 = 0$$

$$y[3] = 2(-1) + 1(0) + 3(1) = -2 + 0 + 3 = 1$$

$$y[4] = 1(-1) + 3(0) = -1$$

$$y[5] = 3(-1) = -3$$

Result: $y[n] = \{1, 2, 0, 1, -1, -3\}$.

Chapter 7

Pole-Zero Modeling of Speech

7.1 Why Pole-Zero Modeling?

Real-life scenario: Think of an old-school guitar equalizer pedal with “boost” and “cut” knobs. Turning a “boost” knob up amplifies a certain frequency band (like a pole does); turning a “cut” knob down suppresses a certain frequency band (like a zero does). Your vocal tract behaves exactly like a built-in, ever-changing equalizer: it naturally boosts certain frequencies (formants) and suppresses others, and pole-zero modeling is simply the mathematical language for describing that built-in equalizer.

In digital speech processing, the **vocal tract is considered a linear system**. It shapes the sound produced by the vocal cords via physical movements of the tongue, lips, and jaw. Rather than explicitly describing all of these complicated physical movements, DSP represents the vocal tract using a transfer function known as the **Pole-Zero Model**:

$$\text{Excitation Signal } (x[n]) \xrightarrow{\text{Vocal Tract System } H(z)} \text{Speech } (y[n])$$

The vocal tract’s transfer function is written as:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2} + \dots + b_M z^{-M}}{a_0 + a_1 z^{-1} + a_2 z^{-2} + \dots + a_M z^{-M}} = \frac{(z - z_1)(z - z_2) \dots}{(z - p_1)(z - p_2) \dots}$$

where $Y(z)$ is the Z-transform of the output (numerator, containing **zeros**) and $X(z)$ is the Z-transform of the input (denominator, containing **poles**).

- A **pole** is a value of z at which $H(z) \rightarrow \infty$ (the denominator becomes zero) — it **amplifies/boosts** certain frequencies, like turning up the “boost” knob. In speech, poles represent the **resonances (formants)** of the vocal tract.
- A **zero** is a value of z at which $H(z) = 0$ (the numerator becomes zero) — it **reduces/cancels** certain frequencies, like turning down the “cut” knob. In speech, zeros model phenomena such as nasal sounds and fricatives, which introduce spectral dips.

7.2 The Z-Transform (Brief)

The Fourier Transform alone is not always sufficient because it cannot capture whether a system is **stable or unstable**, and does not directly apply to systems with feedback (IIR filters), low-latency, or resource-constrained scenarios. The **Z-transform** generalizes the DTFT by introducing a complex variable $z = re^{j\omega}$ (magnitude r and angle ω), allowing us to analyze exponentially growing/decaying signal components, not just pure oscillations.

- The Z-domain is the discrete-time counterpart of the continuous-time **s-domain** (Laplace transform), related by $z = e^{sT}$, where T is the sampling interval and $s = \sigma + j\omega$.
- The **s-plane's imaginary axis** ($\sigma = 0$, i.e., pure oscillation, stability boundary) maps onto the **unit circle** ($|z| = 1$) in the z-plane.
- **Stability in the z-domain:** for a causal discrete-time system, all poles must lie **strictly inside the unit circle** for the system to be stable (bounded response). A pole exactly on the unit circle gives a sustained (undamped) oscillation; a pole outside the unit circle gives an unbounded, unstable (exponentially growing) response — think of a microphone shrieking with feedback when placed too close to a speaker, a real-world example of an unstable, “runaway” system.

7.3 Significance of Pole and Zero Locations

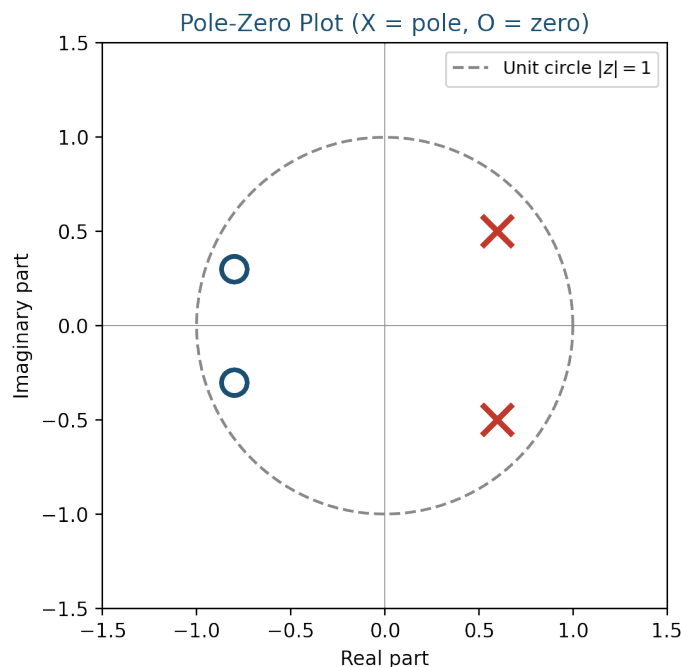


Figure 7.1: A pole-zero plot: poles (X) boost nearby frequencies, zeros (O) cancel them

- **Poles near the unit circle** → sharp peaks (strong resonance) in the frequency response.

- **Poles farther inside the circle** → weaker resonance, smoother frequency response.
- **Poles outside the unit circle** → unstable system (response grows unbounded).
- **Zeros near the unit circle** → deep notches (strong attenuation) at those frequencies.
- **Zeros farther from the unit circle** → weaker attenuation.
- **Zeros exactly on the unit circle** → complete cancellation (zero response) at that exact frequency.

By convention, in a pole-zero plot, poles are marked with an **X** and zeros with an **O** (or a circle). Poles and zeros must either be purely real or occur in complex-conjugate pairs (since the system's coefficients are real).

7.4 Pole-Zero Modeling for Speech

Speech is modeled as $s[n] = e[n] * h[n]$, where $e[n]$ is the excitation (glottal pulses for voiced sounds, or noise for unvoiced sounds) and $h[n]$ is the vocal-tract filter.

- **Poles correspond to formants** — the resonant frequencies of the vocal tract. For example, the vowel /a/ has formants around 700 Hz, 1200 Hz, and 2500 Hz; each of these appears as a complex-conjugate pole pair near the unit circle in the z-plane.
- **Zeros model anti-resonances**, such as those introduced by nasal sounds or fricatives. For example, nasal consonants (/m/, /n/, /ŋ/) involve a side (nasal) cavity that creates sharp anti-resonances, modeled as zeros.

7.5 Worked Numerical Examples — Pole-Zero Modeling

Example 1. Determine the poles and zeros for $H(z) = \frac{z+2}{z^2 + \frac{1}{4}}$.

Solution:

Zeros — set numerator to zero: $z+2=0 \Rightarrow z=-2$. So the system has one zero at $z=-2$ (the system completely suppresses response at $z=-2$).

Poles — set denominator to zero: $z^2 + \frac{1}{4} = 0 \Rightarrow z^2 = -\frac{1}{4} \Rightarrow z = \pm\sqrt{-\frac{1}{4}}$. Since $\sqrt{-1} = j$, we get $z = \pm\frac{j}{2}$.

Result: Poles at $z = +j/2$ and $z = -j/2$; zero at $z = -2$. The system's response becomes theoretically infinite (very large) at $z = \pm j/2$.

Example 2. Pole-zero plot of a second-order transfer function

$$H(z) = \frac{(z + 0.5 + j0.82)(z + 0.5 - j0.82)}{(z - 0.35 + j0.6)(z - 0.35 - j0.6)}$$

Solution: The numerator gives a complex-conjugate zero pair: $z = -0.5 \mp j0.82$ (marked with O on the plot, in the left half of the unit circle). The denominator gives a complex-conjugate pole pair: $z = 0.35 \mp j0.6$ (marked with X, in the right half). Since $|0.35 + j0.6| = \sqrt{0.35^2 + 0.6^2} \approx 0.695 < 1$, both poles lie **inside** the unit circle $\Rightarrow$ the system is **stable**. The pole pair being closer to the origin than the unit circle means moderate (not razor-sharp) resonance.

Exercise (try yourself). Draw the pole-zero plot for the third-order system

$$H(z) = \frac{3(z+2)}{(z+1)(z+1-2j)(z-1+2j)}$$

Hint: Zero at $z = -2$. Poles at $z = -1$, $z = -1 + 2j$, and $z = 1 - 2j$. Check: are the complex poles conjugates of each other? (No — note $-1 + 2j$ and $1 - 2j$ are *not* conjugate pairs, since a true conjugate of $-1 + 2j$ would be $-1 - 2j$. This is a useful trap to watch for when verifying such problems.)

7.6 Chapter 5-7 : Practice / Exam Questions

Q1. Discuss the limitations of representing signals in the time domain and explain the advantages of analysing signals in the frequency domain.

Answer: See Section 6.3 above: time-domain convolution/analysis is computationally heavy, gives poor insight into filtering behaviour, provides no direct way to measure bandwidth/resonance/spectral energy, makes stability/pole-zero characterization difficult, and makes it hard to visually identify frequency components (harmonics, formants) in cluttered real-world signals like speech or music. Frequency-domain analysis directly reveals the frequency content, is much more intuitive for filter/system design, and directly exposes resonances and bandwidth.

Q2. Define a Linear Time-Invariant (LTI) system along with its mathematical representation. Explain the major properties necessary for a discrete-time system to be an LTI system.

Answer: An LTI system is one satisfying both **linearity** (superposition: $T(ax_1[n] + bx_2[n]) = aT(x_1[n]) + bT(x_2[n])$) and **time-invariance** (a delayed input produces a correspondingly delayed output: if $y[n] = T(x[n])$ then $T(x[n - n_0]) = y[n - n_0]$). Mathematically represented generally as $y[n] = T(x[n])$, and for LTI systems specifically as $y[n] = x[n] * h[n]$, where $h[n]$ is the system's impulse response.

Q3. Write the principle of superposition with reference to an LTI system.

Answer: The principle of superposition states that the response of a system to a linear combination of inputs equals the same linear combination of the individual responses to each input: if $x_1[n] \rightarrow y_1[n]$ and $x_2[n] \rightarrow y_2[n]$, then for any constants a, b : $ax_1[n] + bx_2[n] \rightarrow ay_1[n] + by_2[n]$.

Q4. Define convolution. What role does impulse response play in convolution? List at least three properties of convolution.

Answer: Convolution is the mathematical operation $y[n] = \sum_k x[k]h[n - k] = x[n] * h[n]$ that computes the output of an LTI system by summing scaled and shifted copies of its impulse response, one for each input sample. The impulse response $h[n]$ completely characterizes the LTI system — knowing $h[n]$ lets us compute the output for *any* input via convolution, without needing to know the system's internal physical structure. Three properties: commutative ($x * h = h * x$), associative ($x * (h * g) = (x * h) * g$), and distributive ($x * (h_1 + h_2) = x * h_1 + x * h_2$).

Q5. Briefly describe the finite duration property of convolution (and prepare similarly for all properties of convolution).

Answer: The finite duration property states that if $x[n]$ has finite length N_1 samples and $h[n]$ has finite length N_2 samples, then their convolution $y[n] = x[n] * h[n]$ has finite length $N_1 + N_2 - 1$ samples. This follows because the "flip and slide" process of convolution can only produce non-zero overlap over that many shift positions. (Prepare similarly: e.g., Impulse property — convolving any signal with $\delta[n]$ returns the signal unchanged, since $\delta[n]$ contributes a single unit weight at zero shift.)

Q6. Write the shortcomings of time-domain convolution.

Answer: See Section 6.3: computational cost grows with signal length, poor visual/intuitive insight into filtering, no direct measurement of bandwidth/resonance/energy distribution, difficulty in assessing system stability or characterizing poles/zeros, and poor visibility of frequency components in visually cluttered signals like speech.

Q7. Define pole-zero modeling. Why is it considered one of the important processes in speech processing? Describe the vocal tract in terms of a transfer function in pole-zero modeling. What do poles and zeros represent?

Answer: Pole-zero modeling represents a linear system's transfer function $H(z)$ as a ratio of polynomials in z , whose roots are called zeros (numerator roots) and poles (denominator roots). It is important in speech processing because the vocal tract itself behaves like a resonant acoustic tube, and its resonant/anti-resonant behavior is very efficiently captured by just a handful of pole and zero locations rather than needing to model detailed airflow physics. The vocal tract is represented as

$$H(z) = \frac{(z - z_1)(z - z_2) \cdots}{(z - p_1)(z - p_2) \cdots}.$$
 Poles represent resonances (formants, frequencies that are amplified); zeros represent anti-resonances (frequencies that are attenuated/cancelled, as seen in nasals and fricatives).

Q8. What will be the impact on a linear system if the poles are set to zero? What if the zeros are set to zero?

Answer: If all poles are placed at the origin ($z = 0$), the system becomes a pure FIR (all-zero) system with no resonant/feedback behavior — the impulse response becomes finite in duration and there is no possibility of instability. If all zeros are set to zero (i.e., placed at $z = 0$, which contributes only a constant gain/delay and no frequency shaping), the system becomes a pure all-pole (IIR, recursive) system — which is exactly the model used by Linear Predictive Coding (LPC) for the vocal tract, since it very efficiently represents resonances (formants) without needing explicit zeros.

Q9. What are poles and zeros of a transfer function? How do you obtain them? How do poles and zeros affect system stability and the frequency response? What does the pole-zero plot represent?

Answer: Poles are the values of z that make the denominator of $H(z)$ zero (making $H(z) \rightarrow \infty$); zeros are the values of z that make the numerator zero (making $H(z) = 0$). They are obtained by factoring the numerator and denominator polynomials of $H(z)$ and solving for their roots. Poles determine stability — for a causal system, all poles must lie strictly inside the unit circle for stability; poles near the unit circle create sharp resonant peaks, while zeros near the unit circle create deep notches (attenuation). The pole-zero plot is a graph in the complex z -plane (with the unit circle drawn for reference) showing the locations of all poles (marked X) and zeros (marked O), providing an immediate visual summary of the system's stability and frequency-shaping behavior.

Q10. What is the significance of poles on the left/right half of the $j\omega$ -axis (s-plane)? What happens when a pole lies on the $j\omega$ -axis? What is pole-zero cancellation?

Answer: In the continuous-time s-plane, poles in the **left half-plane** correspond to a stable, decaying response (system is BIBO stable); poles in the **right half-plane** correspond to an unstable, exponentially growing response. A pole exactly **on the $j\omega$ -axis** produces a sustained, undamped oscillation (marginal stability — the system neither decays nor blows up). **Pole-zero cancellation** occurs when a pole and a zero are located at exactly the same point in the s-plane (or z-plane); their effects cancel out algebraically, effectively reducing the system's order and removing that particular resonance/anti-resonance from the observable response.

Q11. Discuss the criteria for pole placement within the Z-domain to ensure stability of a digital speech synthesis filter.

Answer: For a causal, stable digital filter, every pole of $H(z)$ must lie strictly **inside** the unit circle ($|p_i| < 1$). If any pole lies on or outside the unit circle, the impulse response will not decay (or will grow without bound), making the synthesis filter unstable and producing an output that diverges or oscillates indefinitely rather than replicating a natural, decaying speech resonance.

Q12. Explain the phenomenon of aliasing in the context of speech signals. What is the time-domain representation of a speech signal?

Answer: Aliasing in speech signals occurs when sampling below the Nyquist rate causes high-frequency components (e.g., high formants, fricative energy) to fold back and appear as spurious low-frequency components, corrupting the true spectral content and degrading intelligibility/recognition accuracy. The time-domain representation of a speech signal is simply its amplitude plotted against time — showing the waveform's envelope, zero crossings, and relative energy over the course of an utterance.

Q13. What is the impulse response of a system? Discuss different categories of filters based on impulse-response duration.

Answer: The impulse response $h[n]$ is a system's output when the input is a unit impulse $\delta[n]$; it fully characterizes an LTI system. Based on duration: **FIR (Finite Impulse Response)** filters have an impulse response that becomes exactly zero after a finite number of samples (no feedback/recursion; always stable; can have exactly linear phase). **IIR (Infinite Impulse Response)** filters have an impulse response that theoretically continues forever (due to feedback/recursion, i.e., poles); they can achieve sharper frequency responses with fewer coefficients but must be carefully designed (pole placement) to remain stable.

Q14. What is Zero-Crossing Rate (ZCR)? What are voiced and unvoiced segments in a waveform? How can ZCR distinguish them? What is short-time energy and how is it calculated? What is an amplitude/spectral envelope? How can silence be detected? Why is short-time analysis necessary?

Answer: ZCR is the rate at which a signal's amplitude changes sign (crosses zero)

per unit time/frame; it is a simple time-domain feature. Voiced segments have a mostly smooth, quasi-periodic waveform dominated by low-frequency energy $\Rightarrow$ **low ZCR, high energy**. Unvoiced segments are noise-like with rapid, random sign changes $\Rightarrow$ **high ZCR, low energy**. **Short-time energy** is the sum (or average) of squared sample amplitudes within a short analysis frame, $E = \sum_n x[n]^2$ over the frame — it measures the local loudness of that frame. An **amplitude envelope** is a smooth curve tracing the peaks of the time-domain waveform over time; a **spectral envelope** is a smooth curve tracing the overall shape of the frequency spectrum (ignoring fine harmonic detail), carrying information about phonemes and speaker identity. **Silence regions** can be detected as frames with both very low short-time energy and (often) ZCR patterns distinct from noise. **Short-time analysis is necessary** because speech is non-stationary; only within short (quasi-stationary) windows do these time-domain and frequency-domain measures remain meaningful.

Chapter 8

Frequency Analysis of Speech Signals: The Fourier Transform

Real-life scenario: Imagine a live band playing — guitar, drums, and vocals, all mixed into a single sound wave reaching your ear. Somehow, your brain can still pick out “that’s a guitar” and “that’s a voice” separately. The Fourier Transform is the mathematical tool that does exactly this for computers: given one messy, mixed-up waveform, it tells you exactly which frequencies (instruments/pitches) are present and how loud each one is. This is precisely how music apps generate those animated frequency-bar visualizers.

8.1 Exponential Representation of Signals — Euler’s Formula

Euler’s formula states:

$$e^{j\theta} = \cos \theta + j \sin \theta$$

which represents a **rotating vector (phasor)** — picture a clock hand of length 1, spinning counter-clockwise at a constant speed; its horizontal and vertical shadow positions trace out cosine and sine waves respectively. This is the key mathematical trick that lets us convert sinusoids into a form that Fourier analysis can handle directly:

$$\cos \theta = \frac{e^{j\theta} + e^{-j\theta}}{2}, \quad \sin \theta = \frac{e^{j\theta} - e^{-j\theta}}{2j}$$

Why do we need this? The Fourier Transform (and DFT) is naturally defined in terms of complex exponentials $e^{\pm j\theta}$, not directly in terms of sin or cos. So before applying FT/DFT, we first rewrite every sinusoidal component of a signal using Euler’s formula — a bit like converting a recipe from cups to grams before you can use a digital kitchen scale.

Example. Consider the signal $3 \cos(3\pi t) + 4 \cos(2\pi t) + 5 \cos(4\pi t)$; here $A = \{3, 4, 5\}$ and angular frequencies are $(3\pi, 2\pi, 4\pi)$ rad/s. Since $\omega = 2\pi f$, the corresponding frequencies are $f = \{1.5, 1, 2\}$ Hz, so each sinusoid contributes complex exponen-

tials at $\pm f$: i.e., $\pm 1.5, \pm 1, \pm 2$ Hz. Using Euler's formula, the signal becomes:

$$x(t) = \frac{3}{2}e^{j3\pi t} + \frac{3}{2}e^{-j3\pi t} + 2e^{j2\pi t} + 2e^{-j2\pi t} + \frac{5}{2}e^{j4\pi t} + \frac{5}{2}e^{-j4\pi t}$$

8.2 The Fourier Transform

The **Fourier Transform** converts a waveform from the time domain into the frequency domain, revealing the frequencies present in a signal and their corresponding amplitudes — exactly like turning that messy band recording into a clean list: “Guitar at 200 Hz, loud; Vocals at 800 Hz, medium; Cymbals at 6000 Hz, soft.” For a continuous-time signal $x(t)$:

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi ft} dt$$

where $x(t)$ is the time-domain signal, $X(f)$ is its frequency-domain representation, and f is frequency in Hz.

Example: FT of a cosine. Let $x(t) = \cos(2\pi f_0 t) = \frac{1}{2}e^{j2\pi f_0 t} + \frac{1}{2}e^{-j2\pi f_0 t}$. Using the known transform pair $\mathcal{F}\{e^{j2\pi f_0 t}\} = \delta(f - f_0)$ and $\mathcal{F}\{e^{-j2\pi f_0 t}\} = \delta(f + f_0)$:

$$X(f) = \frac{1}{2}\delta(f - f_0) + \frac{1}{2}\delta(f + f_0)$$

Here $\delta(f - f_0)$, the **Dirac delta function**, indicates that all the signal's energy is concentrated exactly at frequency f_0 , with zero energy at any other frequency.

Why “negative” frequency? A real cosine always contains *two* frequency components: one at $+f_0$ and one at $-f_0$. This is not a signal that “oscillates backward in time” — it arises purely from the complex-exponential representation: $e^{+j2\pi f_0 t}$ represents a counter-clockwise rotating phasor, $e^{-j2\pi f_0 t}$ a clockwise rotating one, and a real cosine requires the sum of both rotations (so their imaginary parts cancel, leaving a real-valued signal). Hence a real cosine's spectrum is always symmetric about $f = 0$.

Example: Two-cosine signal. $x(t) = 2\cos(100\pi t) + 3\cos(\frac{300}{4}\pi t)$. Find the frequencies and Fourier Transform.

Step 1 — Find frequencies using $\omega = 2\pi f$: $100\pi = 2\pi f \Rightarrow f = 50$ Hz. Similarly $\frac{300}{4}\pi = 2\pi f \Rightarrow f = 37.5$ Hz. So the signal contains only two frequencies: 50 Hz and 37.5 Hz.

Step 2 — Convert each cosine to complex exponentials: $2\cos(100\pi t) = e^{j2\pi(50)t} + e^{-j2\pi(50)t}$ (using $\cos \theta = (e^{j\theta} + e^{-j\theta})/2$, the leading 2 cancels the denominator 2). Similarly, $3\cos(2\pi(37.5)t) = \frac{3}{2}e^{j2\pi(37.5)t} + \frac{3}{2}e^{-j2\pi(37.5)t}$.

Step 3 — Apply FT: the first cosine gives $\delta(f - 50) + \delta(f + 50)$; the second gives

$$\frac{3}{2}\delta(f - 37.5) + \frac{3}{2}\delta(f + 37.5).$$

8.2.1 Properties of the Fourier Transform

Property	Formula
Linearity	$\mathcal{F}\{ax_1(t) + bx_2(t)\} = aX_1(f) + bX_2(f)$
Time Shift	$\mathcal{F}\{x(t - t_0)\} = e^{-j2\pi ft_0}X(f)$ (magnitude unchanged, only phase changes)
Frequency Shift	$\mathcal{F}\{x(t)e^{j2\pi f_0 t}\} = X(f - f_0)$ (shifts the spectrum along the frequency axis)
Scaling	$\mathcal{F}\{x(at)\} = \frac{1}{ a }X\left(\frac{f}{a}\right)$ (speeding up the signal spreads out its spectrum)
Convolution	$\mathcal{F}\{x(t) * h(t)\} = X(f)H(f)$ (convolution in time = multiplication in frequency — one of the most powerful, widely used properties in all of signal processing)

Inverse Fourier Transform: used to reconstruct the original time-domain signal from its frequency-domain representation:

$$x(t) = \int_{-\infty}^{\infty} X(f) e^{j2\pi ft} df$$

or, in terms of $\omega = 2\pi f$:

$$x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(\omega) e^{j\omega t} d\omega$$

8.3 Discrete-Time Fourier Transform (DTFT)

For a **discrete-time** signal $x[n]$, the Fourier transform decomposes the signal into a sum of sinusoids with different digital frequencies:

$$X(\omega) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n}$$

Here n (samples) is discrete, but ω (digital angular frequency) is **continuous**, so $X(\omega)$ is a continuous curve — the DTFT shows the spectrum at *every* possible frequency. In terms of a continuous normalized frequency f_{norm} : $X(f) = \sum_n x[n] e^{-j2\pi f_{norm} n}$.

Inverse DTFT: any sequence can be represented as a linear combination of complex exponentials whose frequencies lie in the range $[-\pi, \pi]$:

$$x[n] = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(\omega) e^{j\omega n} d\omega$$

Why the range $-\pi$ to π ? Discrete-time sinusoids are periodic in ω with period 2π (i.e., $e^{j(\omega+2\pi)n} = e^{j\omega n}$), so any interval of length 2π contains all the unique spectral information; $[-\pi, \pi]$ is simply the conventional choice.

8.4 Discrete Fourier Transform (DFT)

Computers cannot handle a continuous curve like the DTFT (that would need infinite numbers) — they can only work with a finite list of numbers. So in practice, we compute the spectrum only at a finite set of evenly-spaced frequency “checkpoints,” called **bins**. This practical, computer-friendly version is the **DFT**.

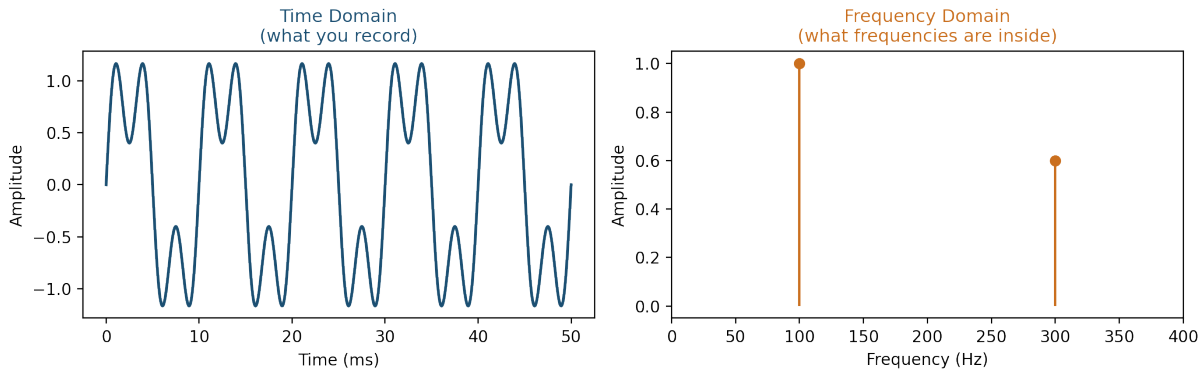


Figure 8.1: Time domain and frequency domain are two views of the same signal

The **DFT** samples the (continuous) DTFT at a finite number of discrete frequencies, for finite-length signals. For a signal of length N :

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}$$

where n = time index, k = frequency-bin index, and N = number of samples. The DFT is essentially a “sampled DTFT”: $X[k] = X(\omega_k = \frac{2\pi k}{N})$; k represents the **DFT frequency bins** — think of bins as buckets, each one responsible for detecting “how much energy is present near this particular frequency.” For each bin k , the DFT effectively multiplies the signal by a complex exponential “basis function” at that bin’s frequency and sums the result.

Orthogonality property of DFT basis functions:

$$\sum_{n=0}^{N-1} e^{j\frac{2\pi}{N}(k-m)n} = \begin{cases} N, & m = k \\ 0, & m \neq k \end{cases}$$

If a basis function is compared with itself ($k = m$), the terms perfectly align, so the sum is N . If two *different* basis functions are compared ($k \neq m$), their positive and

negative contributions cancel exactly over one complete period, so the sum is 0. This orthogonality guarantees the DFT cleanly separates different frequency components without any “leakage” between exactly-aligned bins — every bucket only catches its own frequency, with zero spillover into neighboring buckets.

Frequency resolution of the DFT: $\Delta f = \frac{F_s}{N}$, i.e., the spacing between adjacent DFT bins in Hz, where F_s is the sampling frequency and N is the number of points in the DFT.

8.5 Worked Numerical Examples — DFT

Example 1. Let $x[n] = 2 \cos\left(\frac{2\pi}{5}n\right) = e^{j2\pi n/5} + e^{-j2\pi n/5}$. Take $N = 5$. Find the 5-point DFT.

Solution:

Step 1 — Compute the 5 samples:

$$\begin{aligned} x[n] &= \left[2 \cos 0, 2 \cos \frac{2\pi}{5}, 2 \cos \frac{4\pi}{5}, 2 \cos \frac{6\pi}{5}, 2 \cos \frac{8\pi}{5} \right] \\ &= [2, 0.618, -1.618, -1.618, 0.618] \end{aligned}$$

Step 2 — DFT formula for $N = 5$: $X[k] = \sum_{n=0}^4 x[n] e^{-j\frac{2\pi}{5}kn}$.

Step 3 — Bin 0: $X[0] = 2 + 0.618 - 1.618 - 1.618 + 0.618 = \mathbf{0}$.

Step 4 — Bin 1: Since $x[n] = e^{j2\pi n/5} + e^{-j2\pi n/5}$, we get

$$X[1] = \sum_{n=0}^4 (e^{j2\pi n/5} + e^{-j2\pi n/5}) e^{-j2\pi n/5} = \sum_{n=0}^4 1 + \sum_{n=0}^4 e^{-j4\pi n/5} = 5 + 0 = \mathbf{5}$$

(the second sum vanishes by the orthogonality property.)

Step 5 — Bin 2: Similarly, $X[2] = \sum e^{-j2\pi n/5} + \sum e^{-j6\pi n/5} = 0 + 0 = \mathbf{0}$.

Bins 3 and 4: $X[3] = 0$ (by symmetry with bin 2); $X[4] = 5$ (same as bin 1, since the signal's energy is symmetric about bin 0 for a real signal).

Result: $X[k] = [0, 5, 0, 0, 5]$ — energy concentrated exactly at bins 1 and 4 (i.e., $\pm \frac{2\pi}{5}$), confirming the input frequency.

Example 2 (Full worked 8-point DFT). A sine wave $x(t) = \sin(2\pi ft)$ with $f = 1$ Hz, amplitude $A = 1$, is sampled at $F_s = 8$ Hz. Apply an 8-point DFT.

Solution:

Step 1 — Sampling: $T_s = 1/F_s = 1/8$ s, so sampling instants are $t = n/8$. Then:

$$x[n] = A \sin(2\pi f_0 t) = \sin\left(2\pi \cdot 1 \cdot \frac{n}{8}\right) = \sin\left(\frac{\pi}{4}n\right)$$

Step 2 — Sample values at $n = 0, \dots, 7$ (angles $0, \pi/4, \pi/2, 3\pi/4, \pi, 5\pi/4, 3\pi/2, 7\pi/4$):

$$x[n] = [0, 0.707, 1, 0.707, 0, -0.707, -1, -0.707]$$

Step 3 — Frequency spacing: $N = 8 \Rightarrow \Delta f = F_s/N = 8/8 = 1$ Hz. Since our signal is at 1 Hz, we expect a peak exactly at bin $k = 1$ (and, by symmetry, at bin $k = 7 = N - 1$, corresponding to -1 Hz).

Step 4 — Compute $X[1] = \sum_{n=0}^7 x[n] e^{-j2\pi n/8}$ term by term (using $e^{-j\theta} = \cos \theta - j \sin \theta$):

$$n=0: 0 \cdot e^{-j0} = 0$$

$$n=1: 0.707 e^{-j\pi/4} = 0.707(\cos 45^\circ - j \sin 45^\circ) = 0.707(0.707 - j0.707) = 0.5 - j0.5$$

$$n=2: 1 \cdot e^{-j\pi/2} = 1(0 - j1) = -j$$

$$n=3: 0.707 e^{-j3\pi/4} = 0.707(-0.707 - j0.707) = -0.5 - j0.5$$

$$n=4: 0 \cdot e^{-j\pi} = 0$$

$$n=5: -0.707 e^{-j5\pi/4} = -0.707(-0.707 + j0.707) = 0.5 - j0.5$$

$$n=6: -1 \cdot e^{-j3\pi/2} = -1(0 - j(-1)) = -j \text{ (careful sign-work gives } -j)$$

$$n=7: -0.707 e^{-j7\pi/4} = -0.707(0.707 + j0.707) = -0.5 - j0.5$$

Step 5 — Sum all terms:

$$\begin{aligned} X[1] &= 0 + (0.5 - j0.5) + (-j) + (-0.5 - j0.5) + 0 + (0.5 - j0.5) + (-j) + (-0.5 - j0.5) \\ &= -4j \end{aligned}$$

Step 6 — Magnitude and phase:

$$|X[1]| = \sqrt{0^2 + (-4)^2} = 4, \quad \text{phase} = -90^\circ$$

Step 7 — Why magnitude 4, not amplitude 1? For a sinusoid whose frequency exactly matches a DFT bin, the general relationship is:

$$|X[k]| = \frac{A \times N}{2}$$

Here $A = 1$, $N = 8 \Rightarrow |X[1]| = \frac{1 \times 8}{2} = 4$, matching our computed result exactly.

Result: $X[1] = -4j$ (magnitude 4, phase -90°); by symmetry $X[7] = +4j$ (magnitude 4, phase $+90^\circ$); all other bins are zero. The sine wave's energy is concentrated exactly at bin 1 and (symmetrically) bin 7, confirming the 1 Hz frequency component. This is a general pattern: the DFT acts like a bank of frequency detectors — since the signal's frequency exactly matches a DFT bin, all other frequency components perfectly cancel out due to orthogonality of the DFT basis functions, leaving all the energy concentrated in that one matching bin (and its mirror).

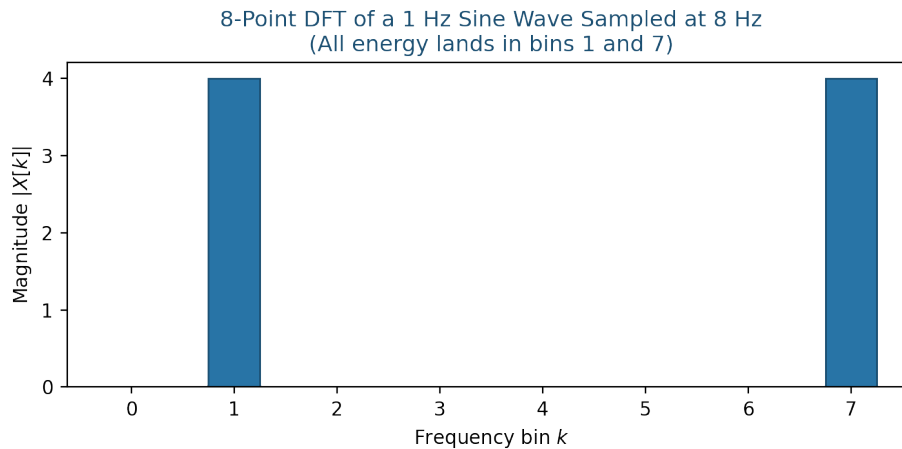


Figure 8.2: The result: a single sharp spike, exactly where the 1 Hz tone lives

Example 3 (Question Bank). A discrete-time signal is $x[n] = \{1, 0, -1, 0\}$ for $n = 0, 1, 2, 3$. Compute the 4-point DFT.

Solution: $N = 4$, so $X[k] = \sum_{n=0}^3 x[n]e^{-j\frac{2\pi}{4}kn} = \sum_{n=0}^3 x[n]e^{-j\frac{\pi}{2}kn}$.

$$X[0] = x[0] + x[1] + x[2] + x[3] = 1 + 0 - 1 + 0 = 0$$

$$X[1] = x[0]e^0 + x[1]e^{-j\pi/2} + x[2]e^{-j\pi} + x[3]e^{-j3\pi/2} = 1(1) + 0(-j) + (-1)(-1) + 0(j) = 1 + 0 + 1 + 0 = 2$$

$$X[2] = x[0] - x[1] + x[2] - x[3] = 1 - 0 - 1 - 0 = 0$$

$$X[3] = x[0]e^0 + x[1]e^{-j3\pi/2} + x[2]e^{-j3\pi} + x[3]e^{-j9\pi/2} = 1 + 0 + 1 + 0 = 2$$

Result: $X[k] = [0, 2, 0, 2]$.

8.6 Spectral Leakage

Real-life scenario: Ever noticed a faint “clicking” sound between two songs on a badly-made playlist, right at the join? That’s a real-world discontinuity artifact — very similar in spirit to spectral leakage, where an abrupt “edge” in a signal creates unwanted extra frequency content that shouldn’t really be there.

If the signal frequency does not align exactly with a DFT bin, the energy “spreads” across multiple neighbouring bins (giving nonzero, complex values in several bins instead of a single clean spike). This is called **spectral leakage**. It occurs because the DFT implicitly assumes the analyzed frame is one period of a periodic signal; if the actual signal frequency doesn’t fit an integer number of cycles within the frame length, an artificial discontinuity is created at the frame edges, which manifests as smeared energy in the frequency domain. **Spectral leakage can be reduced** by applying a tapering window function (e.g., Hamming or Hanning window) to the frame before taking the DFT, which smoothly forces the edges of the frame toward zero, reducing the artificial discontinuity.

8.7 Complexity of the DFT, and the Fast Fourier Transform (FFT)

Real-life scenario: This is the reason your phone can show you a live, smoothly updating spectrum visualizer or perform real-time voice recognition without lagging. If phones had to use the slow, direct DFT formula, this kind of real-time audio processing simply wouldn’t be possible on limited battery/processing power — the FFT is what makes it fast enough to run live.

The direct DFT computation requires, for each of the N output bins, a sum over N input terms (each involving a complex multiplication) — giving N^2 multiplications and additions overall. So the **complexity of the DFT is $O(N^2)$** . For $N = 1024$, this is about **1 million operations**.

The **Fast Fourier Transform (FFT)**, popularized by **James W. Cooley and John W. Tukey in 1965**, is not a new transform — it is simply a much faster *algorithm* for computing the exact same DFT values, using a **divide-and-conquer** approach (the same

general strategy as merge sort in computer science: split the big problem into smaller identical sub-problems, solve those, then combine).

- It uses **even-odd decomposition** to break the problem into smaller sub-problems.
- By exploiting the symmetry and periodicity of the complex exponential terms (called **twiddle factors**), the FFT avoids redundant computations and reuses intermediate results.
- This reduces the computational complexity from $O(N^2)$ (DFT) to $O(N \log_2 N)$ (FFT).

Efficiency gain for $N = 1024$: Direct DFT $\approx N^2 = 1024^2 \approx 1,048,576$ operations. FFT $\approx N \log_2 N = 1024 \times 10 = 10,240$ operations. This is roughly a **100-fold reduction** in computation — a massive efficiency gain that makes real-time spectral analysis of speech practically feasible.

8.7.1 Even-Odd Decomposition (Radix-2 FFT)

The N -point DFT is split by separating the input into **even-indexed** samples ($n = 0, 2, 4, \dots$) and **odd-indexed** samples ($n = 1, 3, 5, \dots$) — similar to how you might split a big deck of cards into two smaller decks to sort faster, then merge them back:

$$F_k = \sum_{m=0}^{N/2-1} x_{2m} e^{-j \frac{2\pi k(2m)}{N}} + \sum_{m=0}^{N/2-1} x_{2m+1} e^{-j \frac{2\pi k(2m+1)}{N}}$$

This says: **DFT = (DFT of even samples) + (DFT of odd samples)**, where each of these two sub-DFTs is itself only an $N/2$ -point DFT:

$$\text{Even part: } \sum_{m=0}^{N/2-1} x_{2m} e^{-j \frac{2\pi k m}{N/2}} \quad \text{Odd part: } \sum_{m=0}^{N/2-1} x_{2m+1} e^{-j \frac{2\pi k(2m+1)}{N}}$$

Simplifying the odd-part exponential: $e^{-j \frac{2\pi k(2m+1)}{N}} = e^{-j \frac{4\pi k m}{N}} e^{-j \frac{2\pi k}{N}} = e^{-j \frac{2\pi k m}{N/2}} \cdot e^{-j \frac{2\pi k}{N}}$. The second factor, $e^{-j \frac{2\pi k}{N}}$, does **not** depend on m , so it can be pulled outside the sum:

$$e^{-j \frac{2\pi k}{N}} \sum_{m=0}^{N/2-1} x_{2m+1} e^{-j \frac{2\pi k m}{N/2}}$$

This constant factor is called the **Twiddle Factor**:

$$W_N^k = e^{-j \frac{2\pi k}{N}}$$

If we define $E[k]$ as the $(N/2)$ -point DFT of the even samples and $O[k]$ as the $(N/2)$ -point DFT of the odd samples, the original N -point DFT becomes:

$$F_k = E[k] + W_N^k O[k]$$

8.7.2 Butterfly Network

A **butterfly** is the basic computational unit of the FFT. It takes two input values A and B , combines them using a twiddle factor W , and produces two outputs:

$$X_1 = A + WB, \quad X_2 = A - WB$$

The FFT Butterfly: One Basic Building Block

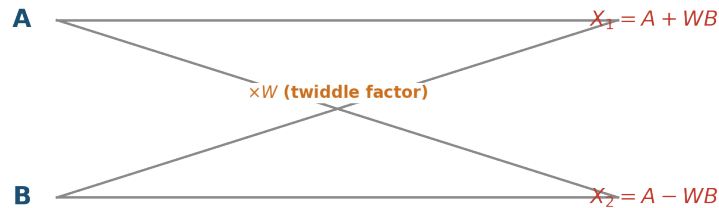


Figure 8.3: The FFT butterfly: the basic Lego brick the entire FFT is built from

Each butterfly performs just one complex multiplication (by W), one addition, and one subtraction — and produces both outputs needed for the next stage. The full FFT is built from many butterfly operations arranged in $\log_2 N$ stages, which is exactly why the total complexity becomes $O(N \log_2 N)$ rather than $O(N^2)$.

8.8 Chapter 8 : Practice / Exam Questions

Q1. What role does the Fourier transform play in speech processing? How can a sinusoidal signal be represented in complex exponential form? Write the importance of the Dirac delta function in the Fourier transform.

Answer: The Fourier transform converts a speech signal from the time domain (where frequency content is hidden) into the frequency domain, revealing pitch, formants, and harmonic structure — information essential for nearly all speech analysis, recognition, and coding tasks. A sinusoid can be represented in complex-exponential form via Euler's identity: $\cos \theta = (e^{j\theta} + e^{-j\theta})/2$ and $\sin \theta = (e^{j\theta} - e^{-j\theta})/2j$, converting real sinusoids into a sum of two counter-rotating complex phasors. The Dirac delta function $\delta(f - f_0)$ represents an idealized signal with energy concentrated at exactly one frequency f_0 and zero everywhere else; it is important because it is precisely the Fourier Transform of a pure complex exponential, letting us express the spectrum of any sinusoid as a pair of perfectly localized spikes.

Q2. List the properties of the Fourier Transform. Define the inverse Fourier Transform.

Answer: Linearity, Time Shift, Frequency Shift, Scaling, and Convolution (see the property table above — convolution in time domain becomes multiplication in frequency domain, and vice versa). The Inverse Fourier Transform reconstructs the time-domain signal from its frequency-domain representation: $x(t) = \int_{-\infty}^{\infty} X(f)e^{j2\pi ft} df$.

Q3. Define the Discrete Fourier Transform along with its mathematical representation. Define the inverse DTFT. What is the basis function in DFT? Write the orthogonality principle of DFT basis functions.

Answer: The DFT converts a finite-length discrete-time sequence into a finite set of discrete frequency-domain samples: $X[k] = \sum_{n=0}^{N-1} x[n]e^{-j\frac{2\pi}{N}kn}$. The Inverse DTFT reconstructs $x[n]$ from the continuous DTFT spectrum: $x[n] = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(\omega)e^{j\omega n} d\omega$. The DFT's **basis functions** are the complex exponentials $e^{j\frac{2\pi}{N}kn}$ for $k = 0, \dots, N-1$ — each represents a pure sinusoid at a specific discrete frequency, and the DFT expresses any finite sequence as a weighted sum of these basis functions. The **orthogonality principle** states $\sum_{n=0}^{N-1} e^{j\frac{2\pi}{N}(k-m)n} = N$ if $k = m$, and 0 otherwise — meaning different basis functions do not interfere with each other, allowing the DFT to cleanly separate frequency components.

Q4. What will be the Fourier Transform of a discrete-time unit impulse signal? Show steps.

Answer: Let $x[n] = \delta[n]$. By definition of the DTFT: $X(\omega) = \sum_{n=-\infty}^{\infty} \delta[n]e^{-j\omega n}$. Since $\delta[n]$ is nonzero (equal to 1) only at $n = 0$, every term in the sum vanishes except the $n = 0$ term: $X(\omega) = 1 \cdot e^{-j\omega(0)} = 1$. **Result:** the Fourier Transform of a unit impulse is a constant, $X(\omega) = 1$ for all ω — i.e., an impulse contains *equal*

energy at every frequency (a “flat” or “white” spectrum), which makes intuitive sense since an impulse is the most abrupt, “broadband” signal possible.

Q5. Explain spectral leakage and its impact on FFT-based spectral analysis. How can it be avoided?

Answer: Spectral leakage happens when a signal’s true frequency does not align exactly with one of the DFT’s discrete frequency bins, causing its energy to “spread” across several neighbouring bins instead of appearing as one sharp spike — this happens because the finite-length analysis window implicitly creates artificial discontinuities at its edges (the DFT effectively assumes periodic extension of the frame). Impact: reduced frequency resolution, blurred spectral peaks, and masking of nearby weaker frequency components. It can be reduced (not fully eliminated) by applying a tapering window (Hamming, Hanning) before computing the DFT, which smoothly reduces the signal amplitude to near zero at the frame edges, minimizing the artificial discontinuity.

Q6. Write a short note on the role of the twiddle factor in FFT. Estimate the computational complexity of DFT vs FFT for $N = 1024$ and state the efficiency gain. Discuss the significance of even-odd decomposition in FFT. Define twiddle factor and explain the butterfly network.

Answer: The twiddle factor $W_N^k = e^{-j2\pi k/N}$ is the constant complex-exponential multiplier that appears when combining the even-part and odd-part sub-DFTs in the FFT’s divide-and-conquer recursion; it “rotates” the odd-indexed sub-DFT results by the correct phase amount before combining them with the even-indexed results. **Even-odd decomposition** is significant because it recursively splits an N -point DFT into two $(N/2)$ -point DFTs (then four $(N/4)$ -point DFTs, and so on), and because the sub-DFTs’ twiddle factors repeat with predictable symmetry, allowing results to be reused instead of recomputed — this recursive halving is exactly what reduces complexity from $O(N^2)$ to $O(N \log_2 N)$. For $N = 1024$: DFT requires $\approx N^2 \approx 1,048,576$ operations; FFT requires $\approx N \log_2 N = 1024 \times 10 = 10,240$ operations — roughly a **100x** efficiency gain. The **butterfly network** is the basic FFT computational unit: given two inputs A, B and twiddle factor W , it computes $X_1 = A + WB$ and $X_2 = A - WB$ using just one complex multiplication; the full FFT chains together $\frac{N}{2} \log_2 N$ such butterflies across $\log_2 N$ stages.

Q7. Show steps of how the Fast Fourier Transform reduces the complexity of the Discrete Fourier Transform. Why is stationary Fourier analysis insufficient for speech signals?

Answer: (See derivation in Section 8.6: even-odd split $\rightarrow$ twiddle factor extraction $\rightarrow$ butterfly combination $\rightarrow$ recursive halving $\rightarrow O(N \log_2 N)$.) Stationary (single, whole-signal) Fourier analysis is insufficient for speech because speech is **non-stationary** — its frequency content changes continuously as the speaker moves through different phonemes. A single FT over an entire utterance would average away all this crucial time-varying information (e.g., it could not tell you when a vowel became a consonant). Instead, speech must be analyzed using **short-time** (frame-

based) Fourier analysis (STFT), computing a separate spectrum for each short, quasi-stationary frame, giving rise to the spectrogram.

Q8. Explain how the time-domain convolution of the glottal excitation source and the vocal-tract impulse response translates into the frequency domain.

Answer: Speech is produced as $s[n] = e[n] * h[n]$ (excitation convolved with vocal-tract impulse response). By the Convolution Property of the Fourier Transform, convolution in the time domain becomes **multiplication** in the frequency domain: $S(f) = E(f) \cdot H(f)$. This means the speech spectrum is simply the product of the excitation spectrum (harmonics of pitch, or noise) and the vocal-tract's frequency response (formant resonances) — a much simpler relationship to analyze than the original time-domain convolution, and it is the direct motivation for later using the logarithm (turning this multiplication into an addition) in cepstral analysis.

Chapter 9

Spectral Analysis Models: Filter Banks and Linear Prediction

Real-life scenario: A graphic equalizer on a stereo system has multiple sliders — “60 Hz,” “250 Hz,” “1 kHz,” “4 kHz,” “16 kHz” — each slider boosting or cutting a specific frequency band. That’s literally a bank of bandpass filters in everyday life! This chapter explains the engineering behind building such a “bank of filters” for analyzing speech, plus an alternative smart trick (Linear Prediction) for the same purpose.

9.1 The Spectral Envelope

The **spectral envelope** is the smooth curve outlining the peaks and valleys of the speech spectrum — imagine tracing a smooth line just over the tops of all the little spikes in a spectrum plot, ignoring the fine jagged detail. It carries crucial information about both **phonemes** (which sound is being produced — via formant locations) and **speaker identity** (via the overall vocal-tract shape). Extracting this spectral envelope is the goal of the signal-processing “front end” of a speech recognition system.

Two dominant methods exist for estimating the short-time spectral envelope:

1. **The Bank-of-Filters (BOF) model**
2. **The Linear Predictive Coding (LPC) model**

9.2 Bank-of-Filters (BOF) Model

The bank-of-filters approach performs spectral analysis by passing the speech signal through a **set of bandpass filters** that collectively span the frequency range of interest (just like the equalizer sliders above), then measuring the energy in each band over time.

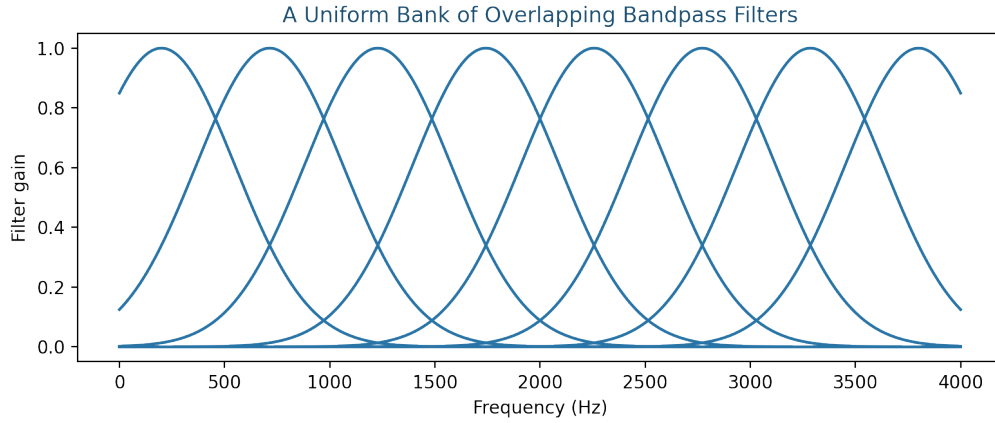


Figure 9.1: A bank of overlapping bandpass filters covering the frequency range

9.2.1 Types of Digital Filters

- **Uniform Filter Banks:** filters spaced uniformly across the frequency range, all with equal bandwidth. The number of filters Q should not be too small (fewer than 8, which under-resolves the spectrum) nor too large (more than 32, which can leave some bands with very low energy for certain speakers). Uniform banks do **not** match human auditory perception (which is non-linear).
- **Non-Uniform Filter Banks:** filters with varying bandwidths — narrower at low frequencies, wider at high frequencies — designed according to criteria related to human auditory perception (e.g., the Mel scale). More complex and computationally heavier than uniform banks.
- **FIR vs. IIR filter banks:** FIR filters are always stable, depend only on current and past inputs, can achieve exactly linear phase (preserving waveform shape), and their computational cost is significantly reduced by implementing them via the FFT (Short-Time Fourier Transform, STFT) — this is a highly efficient way to realize a uniform FIR filter bank (the “polyphase filter-bank structure”).
- **Tree structures:** non-uniform filter banks (e.g., octave-spaced designs) can be efficiently implemented using a tree of Quadrature Mirror Filters (QMFs) with successive stages of filtering and sample-rate reduction.
- **By passband shape:** Low-pass (passes low frequencies), High-pass (passes high frequencies), Band-pass (passes a specific band), Notch (rejects a narrow band), All-pass (passes all frequencies but alters phase).

9.2.2 Complete BOF Model Structure

A complete bank-of-filters front-end analyzer follows this canonical structure for each of the Q channels, applied in parallel:

$$\begin{array}{ccccccc}
 s(n) & \xrightarrow{\text{Bandpass Filter}_i} & s_i(n) & \xrightarrow{\text{Nonlinearity}} & v_i(n) & \xrightarrow{\text{Lowpass Filter}} & t_i(n) \\
 & & \xrightarrow{\text{Downsample}} & u_i(m) & \xrightarrow{\text{Amplitude Compression}} & x_i(m) &
 \end{array}$$

Step 1 — Bandpass Filtering: The sampled speech signal $s(n)$ is passed through a bank of Q bandpass filters, each with its own impulse response $h_i(n)$, covering a specific frequency band. Bands may (and generally do) overlap. The output of the i -th filter is $s_i(n)$.

Step 2 — Nonlinearity (Rectification): To measure the *energy* in each band, the bandpass signals $s_i(n)$ are passed through a nonlinearity, typically a **full-wave** or **half-wave rectifier** — the same idea as an old AM radio, which rectifies a radio wave to recover the audio riding on top of it.

- A **full-wave rectifier** takes the absolute value of the signal (flips negative parts positive).
- A **half-wave rectifier** sets negative parts to zero, keeping only the positive half.

This shifts the signal's spectrum toward the low-frequency band while creating high-frequency "images." Rectification produces: high-frequency harmonics/images ($2f, 4f, 6f, \dots$) which are artifacts; a DC component (constant offset at 0 Hz); and low-frequency envelope terms (slow variations tracking the energy).

Step 3 — Lowpass Filtering: A lowpass filter is applied to the rectified output to smooth the signal and eliminate the newly created high-frequency images, keeping the DC component that estimates the speech signal's energy in that frequency band. For quasi-periodic (voiced) speech, this concentrates energy in a low-frequency band around DC.

Step 4 — Sampling Rate Reduction (Decimation): The bandwidth of the resulting lowpass signal $u_i(n)$ relates to the rate of change of speech harmonics, typically around 20–30 Hz. For efficiency, the signal is therefore **resampled at a much lower rate**, on the order of 40–60 Hz.

Step 5 — Amplitude Compression: The dynamic range of the signal is compressed using a logarithmic or μ -law encoding scheme, greatly reducing the speech data rate while retaining the important characteristics of the signal.

Example of the overall data reduction achieved: a speech signal with a raw bit rate of 240 kbps can be reduced to about 5.6 kbps using a 16-channel filter bank plus compression — nearly a **40:1** data reduction, while retaining the important characteristics of the speech signal. Instead of sending the complete speech waveform, we extract and send only its compact spectral-envelope representation.

Worked Example. A narrowband speech signal is sampled at 8 kHz with 8-bit precision. Suppose this signal is processed through a 10-channel filter bank, where each channel output is sampled at 100 Hz and quantized with 6-bit precision. (i) Compute the raw bit rate. (ii) Compute the compressed bit rate. (iii) Determine the compression ratio.

Solution:

(i) *Raw bit rate:* $R_{org} = F_s \times \text{precision} = 8000 \times 8 = 64,000 \text{ bps} = \mathbf{64 \text{ kbps}}$.

(ii) *Each channel produces* $100 \text{ Hz} \times 6 \text{ bits} = 600 \text{ bits/sec}$. For 10 channels: $R_{compressed} = 10 \times 600 = 6000 \text{ bps} = \mathbf{6 \text{ kbps}}$.

$$(iii) \text{ Compression Ratio} = \frac{R_{org}}{R_{compressed}} = \frac{64}{6} = \mathbf{10.67 : 1}.$$

9.3 The Mel Filter — a Non-Uniform, Perceptually Motivated Filter Bank

Real-life scenario: You can easily tell apart a 100 Hz hum and a 200 Hz hum (they sound clearly different in pitch), but you probably can't tell apart a 10,000 Hz tone from a 10,100 Hz tone (they'd sound basically identical) — even though both pairs are "100 Hz apart." Your ears simply don't care as much about small frequency differences at high frequencies. The Mel scale is built to match exactly this everyday listening experience.

Core idea: “Humans perceive frequency logarithmically.” Rather than spacing filters uniformly in Hz, filters can instead be spaced according to a **critical band scale** based on perceptual studies of how different frequency bands contribute to speech articulation — such as the **Mel scale** or the **Bark scale**.

Mel Scale: converts an ordinary frequency in Hz into a scale that approximately represents human perception of pitch:

$$m = M(f) = 2595 \log_{10} \left(1 + \frac{f}{700} \right)$$

$$f = 700 (10^{m/2595} - 1)$$

where f = frequency in Hz, m = corresponding Mel value. The Mel scale gives **nearly linear spacing at low frequencies** and **logarithmic (compressed) spacing at high frequencies**, matching the way the human cochlea actually resolves frequencies (finer resolution at low frequencies, coarser at high frequencies).

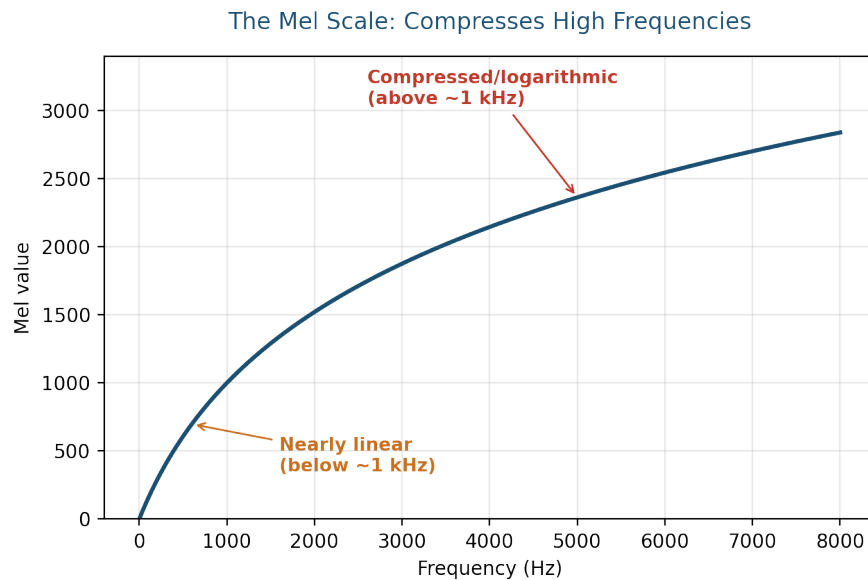


Figure 9.2: The Mel scale: matches how finely your ear can actually tell frequencies apart

9.3.1 How to Implement a Mel Filter Bank

1. Convert the **lowest** and **highest** frequency of interest to Mel using the Mel formula.
2. Create **equally spaced points on the Mel scale** between these two Mel values. (These equally-spaced Mel points correspond to *unequally* spaced frequencies in Hz.) If N triangular filters are needed, the spacing is $(\text{Mel}_{high} - \text{Mel}_{low}) / (N + 2 - 1)$.
3. **Convert these Mel points back to Hz** using $f = 700(10^{m/2595} - 1)$.
4. **Approximate to the nearest frequency bins** (of the DFT) and construct **triangular filters** centered at each of these Hz points, each overlapping its neighbours by 50%.

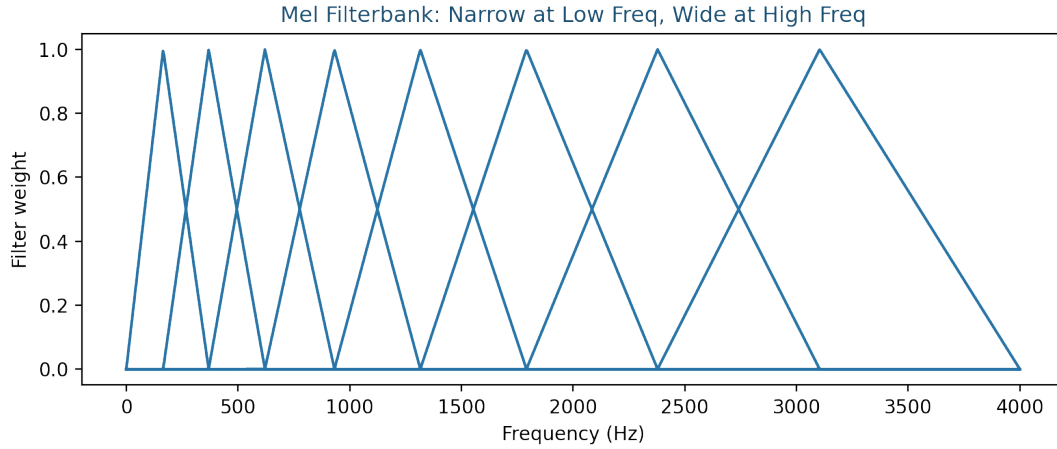


Figure 9.3: Mel filterbank: narrow triangles at low frequencies, wide triangles at high frequencies

Mel Filterbank output (Mel binning): After computing the DFT power spectrum $|X_t[k]|^2$, each triangular Mel filter m computes a weighted sum of power-spectrum bins that it covers:

$$Y_t[m] = \sum_{k=1}^N W_m[k] |X_t[k]|^2$$

where k is the DFT bin number ($1, \dots, N$) and m is the Mel-filter-bank index ($1, \dots, M$). $W_m[k]$ is the triangular weighting function of filter m at bin k . The output for each Mel-scale slot represents the (weighted) energy from the range of frequency bands covered by that filter — this collapsing/binning of many DFT bins into far fewer Mel-filter outputs is what dramatically reduces feature dimensionality for later steps such as MFCC computation.

Worked Example. A speech processing system uses a Mel-scale filter bank with 20 filters over the frequency range 300 Hz to 8000 Hz. Calculate the mel-scale spacing between adjacent filters, and the corresponding Mel-scale frequency for filter centers.

Solution:

Step 1 — Convert range endpoints to Mel:

$$\text{Mel}_{low} = 2595 \log_{10} \left(1 + \frac{300}{700} \right) = 2595 \log_{10}(1.4286) \approx 2595 \times 0.1549 \approx 401.9$$

$$\text{Mel}_{high} = 2595 \log_{10} \left(1 + \frac{8000}{700} \right) = 2595 \log_{10}(12.4286) \approx 2595 \times 1.0944 \approx 2840.0$$

Step 2 — Mel-scale spacing between the $N = 20$ filters (requires $N + 2$ boundary points, giving $N + 1$ intervals):

$$\Delta_{mel} = \frac{2840.0 - 401.9}{20 + 1} = \frac{2438.1}{21} \approx 116.1 \text{ mels}$$

Step 3 — Mel-scale center frequencies: the i -th boundary point is at $\text{Mel}_{low} + i \times \Delta_{mel}$ for $i = 0, 1, \dots, 21$; e.g., the first internal filter center is at $401.9 + 116.1 = 518.0$ mels, the second at 634.1 mels, and so on up to 2840.0 mels. Each of these Mel values can then be converted back to Hz using $f = 700(10^{m/2595} - 1)$ to find the actual (unevenly spaced, closer together at low frequencies) filter center frequencies in Hz.

9.4 Applications of the Filter Bank Model

- **Speech Recognition:** Mel filters are used to extract MFCC features.
- **Speaker Identification:** filter banks preserve fine spectral-envelope detail that distinguishes individual voices.
- **Source Separation (the “Cocktail Party Problem”):** separating overlapping voices/instruments.
- **Speech Enhancement:** spatial filter banks assist multichannel speech enhancement in noisy environments.
- **Speech Synthesis:** filter banks approximate vocal-tract resonances; modern systems (Tacotron, WaveNet) use Mel-spectrograms derived from filter banks as their input representation.
- **Musical Analysis:** extracting timbre/harmonic features for instrument recognition, genre classification, music transcription.

Chapter 10

Linear Predictive Coding (LPC)

Real-life scenario: Ever noticed how a text message keyboard predicts your next word based on what you already typed? "I am going t..." → it suggests "to." Linear Prediction does something conceptually similar but for audio samples: it predicts the next tiny sound sample based on the pattern of the samples that came just before it. This turns out to be a remarkably effective way to describe the shape of someone's vocal tract using just a handful of numbers.

10.1 The Core Idea of Linear Prediction

Linear prediction refers to the process of estimating the present speech sample as a **weighted sum of its previous samples**. This works because speech samples are **not independent** but **highly correlated** — if one sample has a particular value, the samples immediately before and after it are likely to have similar or related values (just like consecutive frames of a smoothly moving video look similar to each other).

$$x'[n] = a_1x[n-1] + a_2x[n-2] + \cdots + a_px[n-p]$$

where $x'[n]$ is the *predicted* current sample, $x[n-k]$ for $k = 1, \dots, p$ are the previous speech samples, a_i are the **prediction coefficients**, and p is the **prediction order**.

Important nuance: the term "linear" does *not* mean the speech waveform itself is a straight line — it means that the previous samples are combined using a linear operation (multiplied by coefficients, then added) to form the prediction.

10.2 Predicted Sample and Prediction Error

The full predicted sample, including the excitation term, is:

$$x'[n] = \sum_{k=1}^p a_k x[n-k] + Gu(n)$$

where G is the **excitation gain** and $u(n)$ is the **excitation signal**. The **prediction error** is the difference between the actual and predicted sample — i.e., how "wrong" our prediction was:

$$e[n] = x[n] - x'[n]$$

Worked Example. If $x[n-1] = 4$ and $x[n-2] = 2$, and the coefficients $a_1 = 0.6$ and $a_2 = 0.2$, find the predicted sample $x'[n]$.

Solution:

$$x'[n] = a_1x[n-1] + a_2x[n-2] = (0.6)(4) + (0.2)(2) = 2.4 + 0.4 = \mathbf{2.8}$$

10.3 Prediction Order

The **prediction order** p is the number of previous speech samples used to predict the current sample — essentially, “how far back do we look?”

- A **higher prediction order** generally allows a more detailed model of the speech signal (captures more resonances/formants), but also increases the number of coefficients and the computational complexity.
- Too **low** a prediction order under-models the vocal tract, missing important formants and giving a poor (over-smoothed) spectral envelope estimate.
- Too **high** a prediction order starts to model pitch-period fine structure and noise rather than just the smooth vocal-tract envelope, leading to overfitting.
- Practical LPC commonly uses an order related to the sampling rate and speech bandwidth (a common rule of thumb: $p \approx F_s(\text{kHz}) + 4$ to $F_s(\text{kHz}) + 5$, allowing roughly one complex pole pair per kHz of bandwidth for formants, plus a few extra for spectral tilt/glottal shaping).

LPC coefficients are chosen so that the **total prediction error energy is minimized**: $E = \sum_n e^2[n]$.

10.4 All-Pole Interpretation of LPC

$$H(z) = \frac{G}{1 - a_1z^{-1} - a_2z^{-2} - \dots - a_pz^{-p}}$$

This describes speech production as a chain — the same source-filter idea from Chapter 7, now written using LPC’s specific notation:

$$u(n) \xrightarrow{\times G} (\text{scaled excitation}) \xrightarrow{\text{All-pole filter } H(z)} s(n) \text{ (speech)}$$

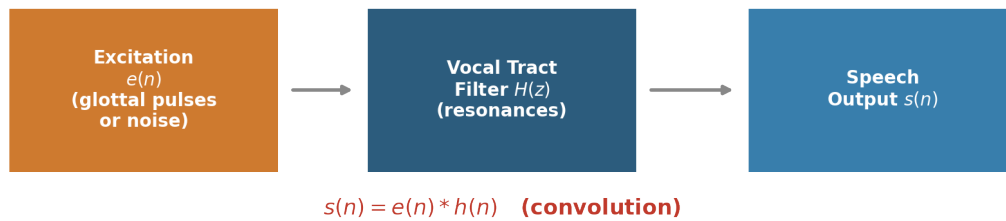


Figure 10.1: Excitation, scaled by a gain, passed through the vocal-tract filter, becomes speech

1. **Excitation** $u(n)$: the source that produces speech. For **voiced** speech, it is a periodic excitation due to vibration of the vocal cords; for **unvoiced** speech, it is a noise-like excitation due to turbulent airflow. So $u(n)$ represents *what* is exciting the vocal tract.
2. **Gain** G : controls the strength/energy of the excitation — $Gu(n)$ means the excitation is scaled to the required strength (e.g., louder speech needs a larger excitation gain).
3. **All-pole filter** $H(z)$: the vocal tract acts approximately like a filter that shapes/modifies the excitation, with $a_1, \dots, a_p$ (the LPC coefficients) describing the vocal-tract characteristics.
4. **Speech** $s(n)$: the final output after the excitation passes through the vocal-tract filter.

“**All-pole**” means the transfer function has **poles but no zeros** in its model (denominator only, no numerator polynomial beyond the constant gain G). This is important because poles are exactly what create resonances (formants) — since LPC’s primary goal is to capture the resonant characteristics of the vocal tract, an all-pole model is a very efficient (few parameters) and generally very effective choice for voiced, non-nasal speech sounds. (Note: nasal sounds, which have anti-resonances/zeros, are only approximately modeled by a pure all-pole system — a higher prediction order is typically needed to compensate.)

10.5 Estimating LPC Coefficients: Autocorrelation Method

The **autocorrelation method** determines the prediction coefficients by measuring the similarity between a speech signal and delayed versions of itself:

$$R[k] = \sum_n x[n] x[n - k]$$

where $R[k]$ is the autocorrelation at lag k , $x[n]$ is the speech sample, $x[n - k]$ is the delayed sample, and k is the lag. The autocorrelation method is computationally attractive because the resulting matrix structure (a Toeplitz matrix) can be exploited very efficiently (via the **Levinson-Durbin recursion**). The analysis minimizes the mean-squared prediction error over the selected (windowed) frame.

Covariance method (brief contrast): the covariance method computes a similar correlation but over a *fixed* analysis interval without windowing the signal to zero at the edges — it typically gives a lower prediction error for a *given* frame (since it does not force the signal to taper at the boundaries), but its resulting normal-equation matrix is not Toeplitz, so it cannot use the efficient Levinson-Durbin recursion and does not guarantee a stable (all-poles-inside-unit-circle) filter, unlike the autocorrelation method.

10.5.1 Durbin’s Recursion (Levinson-Durbin Algorithm)

Given the autocorrelation values $r(0), r(1), \dots, r(p)$, Durbin’s method recursively computes the LPC coefficients:

$$\begin{aligned} E^{(0)} &= r(0) \\ k_i &= \left\{ r(i) - \sum_{j=1}^{i-1} \alpha_j^{(i-1)} r(|i-j|) \right\} / E^{(i-1)}, \quad 1 \leq i \leq p \\ \alpha_i^{(i)} &= k_i \quad \alpha_j^{(i)} = \alpha_j^{(i-1)} - k_i \alpha_{i-j}^{(i-1)} \quad E^{(i)} = (1 - k_i^2) E^{(i-1)} \end{aligned}$$

The quantities k_i are called **reflection coefficients** (also related to acoustic-tube models of the vocal tract). This recursive procedure is far more efficient than direct matrix inversion, and it is guaranteed to produce a stable all-pole filter (all poles inside the unit circle) whenever $|k_i| < 1$ for all i — this is called the **whitening property/behavior** of the LPC model: as more correctly-estimated poles are added, the residual prediction error signal $e[n]$ becomes progressively “whiter” (flatter spectrum, closer to an uncorrelated noise-like signal), because each additional pole removes more of the correlated (predictable) structure from the signal.

10.6 Typical LPC Model Parameters

The typical set of parameters describing an LPC model of a speech frame includes: the **prediction order** p , the **prediction (LPC) coefficients** $a_1, \dots, a_p$, the **excitation gain** G , and the **voiced/unvoiced decision** with (if voiced) the **pitch period**.

10.7 Chapter 9-10 : Practice / Exam Questions

Q1. State how human pitch perception is incorporated in filter-bank modelling for speech signal spectral analysis. Write the significance of Mel-scale in speech processing.

Answer: Human pitch perception is non-linear with respect to frequency in Hz — we perceive equal ratios of frequency as roughly equal steps in pitch, especially at higher frequencies. This is incorporated into filter-bank modelling using **non-uniform** filters spaced according to the **Mel scale** (or Bark scale): narrower filter bandwidths at low frequencies (where human hearing is most sensitive to small frequency differences) and wider bandwidths at high frequencies (where human resolution is coarser). This matches the filter bank's frequency resolution to actual perceptual importance, improving the efficiency and recognition-relevance of extracted features (like MFCC) compared to a uniformly-spaced filter bank.

Q2. List out the different types of filters in signal processing. Distinguish between uniform and non-uniform digital filters. How is a full-wave rectifier different from a half-wave rectifier? What are the shortcomings of the filter-bank model?

Answer: Filter types: low-pass, high-pass, band-pass, notch (band-reject), and all-pass. **Uniform** filter banks have equally spaced center frequencies and equal bandwidths across the whole range; **non-uniform** filter banks have varying bandwidths (e.g., Mel-spaced), typically narrower at low and wider at high frequencies, better matching human perception at the cost of higher design/computation complexity. A **full-wave rectifier** takes the absolute value of the entire signal (both positive and negative excursions contribute to the output); a **half-wave rectifier** keeps only the positive part of the signal and sets the negative part to zero (discarding half the signal's energy information). Shortcomings of the filter-bank model: computationally more expensive than some alternatives (especially non-uniform banks), the choice of the number of filters Q is a compromise (too few under-resolves the spectrum, too many gives low-energy/noisy bands), and it does not explicitly model the speech production process (unlike LPC, which has a clear all-pole vocal-tract interpretation).

Q3. Define linear prediction along with its mathematical representation. What is prediction error in linear prediction? How does prediction order impact an LPC model? What role does an all-pole filter play in the LPC model? Define excitation gain. List the typical LPC model parameters.

Answer: See Sections 10.1-10.6 above for full explanations and formulas: $x'[n] = \sum_{k=1}^p a_k x[n-k]$; prediction error $e[n] = x[n] - x'[n]$; higher order = more detail but more complexity, lower order = underfitting; the all-pole filter $H(z) = G/(1 - \sum a_k z^{-k})$ efficiently models vocal-tract resonances (formants) via its poles; excitation gain G scales the strength of the excitation signal; typical parameters: prediction order p , coefficients $a_1, \dots, a_p$, gain G , voiced/unvoiced decision, and pitch period.

Q4. Explain autocorrelation along with its mathematical representation. How does the covariance method differ from the autocorrelation method? Write the whitening behaviour of the LPC model.

Answer: Autocorrelation $R[k] = \sum_n x[n]x[n-k]$ measures the similarity between a signal and a delayed version of itself at lag k ; in LPC, it is used (via Durbin's recursion) to solve for the prediction coefficients that minimize mean-squared prediction error over a windowed frame. The **covariance method** computes prediction-error minimization over a fixed analysis interval without windowing the signal to zero at the edges, giving (typically) a lower error for that specific frame, but its normal-equation matrix is not Toeplitz (so it can't use the efficient Levinson-Durbin recursion) and it does not guarantee filter stability. **Whitening behavior:** as the LPC model correctly captures more of the correlated (predictable) structure of speech (adding more poles/coefficients), the residual prediction-error signal $e[n]$ becomes progressively closer to a flat-spectrum, uncorrelated ("white") signal — ideally approximating the true underlying excitation source.

Q5. Explain the principle of Linear Predictive Coding in speech processing. Describe how the current speech sample is predicted from previous samples, and explain the significance of LPC coefficients, prediction error, and prediction order in representing the speech signal.

Answer: LPC models speech production as an excitation signal passing through an all-pole vocal-tract filter. The current sample is predicted as a linear (weighted) combination of p previous samples: $x'[n] = \sum_{k=1}^p a_k x[n-k]$. The **LPC coefficients** a_k capture the resonant (formant) structure of the vocal tract for that frame — they effectively encode the vocal-tract's shape. **Prediction error** $e[n] = x[n] - x'[n]$ represents the part of the signal that cannot be explained by the vocal-tract resonances alone — ideally, this approximates the original excitation source (glottal pulse train for voiced sounds, noise for unvoiced). **Prediction order** p determines how many resonances (poles) the model can represent — it must be chosen carefully (too low misses formants; too high starts to model noise/pitch structure rather than the smooth vocal-tract envelope).

Q6. Explain the major steps involved in LPC analysis of a speech signal, describing the role of pre-emphasis, framing and windowing, autocorrelation, LPC coefficient estimation, and parameter conversion.

Answer: **(1) Pre-emphasis:** a first-order FIR filter $H(z) = 1 - \alpha z^{-1}$ boosts high-frequency energy to compensate for the natural spectral tilt (roll-off) of the glottal source, improving the reliability of LPC estimation at higher frequencies. **(2) Framing and Windowing:** the (pre-emphasized) signal is sliced into short overlapping frames (typically 20-30 ms) to satisfy quasi-stationarity, and each frame is multiplied by a tapering window (e.g., Hamming) to reduce edge discontinuities/spectral leakage. **(3) Autocorrelation:** the autocorrelation sequence $R[0], \dots, R[p]$ is computed for each windowed frame. **(4) LPC coefficient estimation (Durbin's method):** the autocorrelation values are converted into LPC coefficients $a_1, \dots, a_p$ using the recursive Levinson-Durbin algorithm. **(5) Parameter conversion:** the raw LPC coefficients are optionally converted into other equivalent representations (reflection

coefficients, log-area ratios, or LPC cepstral coefficients) that may be more robust or more suitable for the downstream recognition task.

Q7. Explain the autocorrelation method for estimating LPC coefficients. How is the autocorrelation sequence used to formulate the LPC normal equations? Mention one advantage of the autocorrelation method.

Answer: The autocorrelation method windows the speech frame (tapering it to zero at the edges) and computes the autocorrelation $R[k] = \sum_n x[n]x[n-k]$ for lags $k = 0, \dots, p$. Minimizing the total squared prediction error with respect to the coefficients a_k leads to a system of p linear equations (the “normal equations” or Yule-Walker equations), expressed compactly using the autocorrelation values arranged in a symmetric Toeplitz matrix: $\mathbf{R}\mathbf{a} = \mathbf{r}$, where $\mathbf{R}$ is built from $R[0], \dots, R[p-1]$ and $\mathbf{r}$ from $R[1], \dots, R[p]$. One key advantage: because $\mathbf{R}$ is Toeplitz, the equations can be solved very efficiently using the recursive Levinson-Durbin algorithm (rather than a general, slower matrix inversion), and the resulting filter is guaranteed to be stable.

Q8. Differentiate between the autocorrelation method and covariance method of LPC analysis with respect to analysis interval, treatment of boundary samples, and prediction-error minimization.

Answer: **Analysis interval:** the autocorrelation method applies a fixed window and effectively assumes the signal is zero outside the frame; the covariance method uses a fixed interval but allows the predictor to reference samples slightly outside the current frame’s boundary (using previously available samples). **Boundary samples:** the autocorrelation method tapers samples toward zero at the frame edges (via windowing), introducing some edge-effect error; the covariance method does not window the signal, avoiding this taper-related distortion but sacrificing the Toeplitz structure. **Prediction-error minimization:** the autocorrelation method minimizes error over an assumed-periodic-extension of the (windowed) frame, guaranteeing a stable filter; the covariance method minimizes the error strictly over the exact analysis interval (often achieving a numerically lower error for that frame), but does not guarantee stability.

Q9. The prediction order p is an important parameter in LPC analysis. Explain what may happen if the prediction order is chosen too low or too high. How would you select a suitable prediction order for speech signals?

Answer: If p is **too low**, the model cannot represent all the formant resonances present in the signal, giving an over-smoothed spectral envelope that misses important spectral peaks (poor recognition/synthesis quality). If p is **too high**, the model starts fitting fine pitch-harmonic structure and noise rather than just the smooth vocal-tract envelope (overfitting), wasting computation and potentially introducing spurious spectral peaks. A suitable order is typically chosen based on the sampling rate: since each complex-conjugate pole pair can model roughly one formant per 1 kHz of bandwidth, a common rule of thumb is $p \approx F_s(\text{in kHz}) + 4$ to $+5$ (e.g., for 8 kHz speech, $p \approx 10$ –13).

Q10. What is prediction error in Linear Predictive Coding? Explain how minimizing the prediction error helps in estimating the LPC coefficients, and how the resulting error signal relates to the excitation of the speech production system.

Answer: Prediction error $e[n] = x[n] - x'[n]$ is the portion of each speech sample that the linear predictor (based on previous samples) fails to explain. LPC coefficients are found precisely by choosing the a_k values that **minimize the total (mean-squared) prediction error** over the analysis frame — this is a standard least-squares optimization problem solved via the normal equations (Yule-Walker equations), typically via Durbin's recursion. Once the coefficients are optimally chosen, the remaining error signal $e[n]$ represents everything about the speech sample *not* explained by the vocal-tract resonance structure — ideally, this residual closely approximates the original **excitation signal** (glottal pulses for voiced speech, or noise for unvoiced speech) that was originally filtered by the vocal tract to produce the speech.

Part II

MODULE II — FEATURE EXTRACTION AND SPEECH RECOGNITION

Chapter 11

Feature Extraction: An Overview

Real-life scenario: When Shazam identifies a song in seconds, it isn't comparing your recorded clip sample-by-sample against millions of songs in its database — that would be far too slow. Instead, it extracts a small set of distinctive "fingerprint" numbers from your audio and matches *those* instead. Speech recognition works on the same principle: turn a huge, messy waveform into a small set of meaningful numbers (features), then work with those numbers instead of the raw sound.

11.1 What is a Feature?

A **feature** is a numeric or categorical representation of raw data that isolates a specific, meaningful attribute for analysis. In speech, features may be considered the **smallest and fundamental building blocks of speech sounds** — they describe the psychological and physiological components that make up a single sound (phoneme). Formally, a **speech feature is a measurable data point extracted from a raw audio signal**.

Feature Extraction is the process of converting a speech signal into a small set of meaningful numerical values that represent important characteristics of the speech. A speech waveform contains a very large number of raw samples; instead of feeding all these samples directly into a speech-processing system, we extract useful, compact information from them — such as frequency content, energy, pitch, spectral shape, and vocal-tract characteristics. Common speech features include **MFCC, LPCC, pitch, energy, and zero-crossing rate**.

11.2 Why Do We Need Feature Extraction?

- **High computational cost:** Raw speech contains thousands of samples per second; processing all of them directly for every downstream task would be computationally very expensive. Feature extraction converts this large amount of raw data into a compact representation.
- **Removing irrelevant information:** The raw waveform contains a lot of information not useful for distinguishing sounds (e.g., microphone/channel characteristics, absolute phase). Feature extraction retains only the information useful for the task while discarding the rest.
- **Easier speech comparison:** Different speech sounds vary in many subtle raw-sample-level ways; features provide a more compact, standardized representation that is far easier to compare mathematically (e.g., via distance measures).

- **Enabling downstream applications:** Feature extraction is a critical stage in speech recognition, speaker identification, emotion recognition, speech classification, and voice-controlled systems.

A good speech feature should: contain relevant speech information, be compact, be robust to noise, be relatively insensitive to speaker-to-speaker variation (unless the task specifically requires speaker discrimination), and help distinguish between different speech sounds.

11.3 Types of Features

11.3.1 Static Features

Static features are characteristics extracted from a segment of data that describe its **overall properties over a fixed window of time**, ignoring how the signal changes moment to moment. Subtypes:

- **Acoustic Features:** Pitch, Energy, and Duration — the fundamental physical properties of speech. This category is itself often subdivided into temporal, spectral, cepstral, and prosodic features.
- **Prosodic / Long-Term Features:** span across multiple phone segments, syllables, or entire sentences, carrying emotional and stylistic context:
 - **Fundamental Frequency (F_0 /Pitch):** the rate of vocal-cord vibration; reflects intonation and stress.
 - **Duration/Rhythm:** speaking rate, pause durations, vowel elongation.
 - **Intensity:** long-term volume variation.
 - Used in Speech Emotion Recognition (SER), emphasis detection, and Text-to-Speech (TTS) synthesis.
- **Temporal (Time-Domain) Features:** extracted directly from the raw waveform over time:
 - **Root Mean Square (RMS) Energy:** measures amplitude/loudness variations across frames.
 - **Zero-Crossing Rate (ZCR):** tracks how often the signal changes sign; distinguishes voiced speech (low ZCR, high energy) from unvoiced speech/silence (high ZCR, low energy).
- **Spectral (Frequency-Domain) Features:** derived by transforming the signal into the frequency domain via the Short-Time Fourier Transform (STFT):
 - **Spectrogram:** a visual map plotting frequency over time, with intensity/color showing signal energy.
 - **Spectral Centroid:** pinpoints the “center of mass” of the spectrum; correlates with perceived sound “brightness.”
 - **Spectral Bandwidth:** measures the frequency spread around the centroid.
- **Cepstral Features:** features mathematically warped to mimic how the human ear actually filters and perceives sound scales:
 - **MFCC (Mel-Frequency Cepstral Coefficients):** mimics human auditory perception; the most widely used feature for speech recognition.
 - **LPCC (Linear Prediction Cepstral Coefficients):** models the vocal tract as a digital filter, representing formants (vocal-tract filter estimation).

- **Glottal Features:** measure the state, tension, and structural health of the vocal folds (glottis):
 - **Jitter:** short-term variation in the pitch (F_0) period (frequency instability).
 - **Shimmer:** short-term variation in peak-to-peak amplitude (loudness instability).
 - **Harmonics-to-Noise Ratio (HNR):** ratio of periodic energy to acoustic noise.
 - Used in clinical speech pathology (detecting voice disorders), age/gender classification, and speaker identification.
- **Deep Learning & Learned Embeddings:** e.g., spectrogram representations fed as 2-D images into a Convolutional Neural Network (CNN), or dense vector embeddings extracted from massive pre-trained models (e.g., wav2vec 2.0, HuBERT).

11.3.2 Dynamic Features

Dynamic features capture **how the acoustic properties of a voice change over time** — they track the movement, speed, and acceleration of the speech articulators (tongue, lips, jaw) across successive frames, using first and second derivatives of static features. Types: **Delta** (first-order temporal derivative, measuring the rate of change of the spectrum from one frame to the next) and **Delta-Delta** (second-order temporal derivative, i.e., the derivative of the Delta features — capturing acceleration). These are covered in full detail with worked numericals in Chapter 13.

11.4 Chapter 11 : Practice / Exam Questions

Q1. Define a feature in the context of speech signal processing. What is meant by feature extraction?

Answer: A feature is a numeric or categorical representation of raw speech data that isolates a specific, meaningful attribute for analysis — essentially the smallest fundamental building block describing a speech sound. Feature extraction is the process of converting a raw speech waveform (containing thousands of raw samples) into a compact set of meaningful numerical values representing important characteristics (frequency content, energy, pitch, spectral shape, vocal-tract characteristics) for use in downstream tasks.

Q2. Explain the need for feature extraction in speech processing.

Answer: Feature extraction is needed because: (1) raw speech has very high computational cost to process directly (thousands of samples per second), (2) it removes irrelevant information not useful for distinguishing sounds, (3) it makes comparing different speech sounds far easier (compact numerical vectors vs. raw waveforms), and (4) it is essential for enabling downstream applications such as speech recognition, speaker identification, and emotion recognition.

Q3. List out different types of features used in speech processing, with examples of each.

Answer: Static features: Acoustic (pitch, energy, duration), Prosodic (F0, rhythm, intensity), Temporal (RMS energy, ZCR), Spectral (spectrogram, spectral centroid/bandwidth), Cepstral (MFCC, LPCC), Glottal (jitter, shimmer, HNR), and Deep-Learning-based (CNN spectrogram embeddings, wav2vec/HuBERT embeddings). **Dynamic features:** Delta (first derivative) and Delta-Delta (second derivative) of any of the above static features.

Q4. What properties make a speech feature "good"? Why is robustness to noise an important criterion?

Answer: A good speech feature: contains relevant speech information, is compact (low-dimensional), is robust to noise, is relatively insensitive to irrelevant speaker-to-speaker variation, and helps distinguish between different speech sounds. Robustness to noise is critical because real-world speech recognition systems must operate in noisy environments (traffic, background chatter, device noise); a feature that changes drastically under small amounts of noise would make the recognizer unreliable outside of pristine studio-recording conditions.

Q5. Differentiate between static and dynamic features with examples.

Answer: Static features describe the overall properties of a signal within one fixed analysis frame, ignoring how the signal changes from frame to frame (e.g., a single frame's MFCC vector, energy, or ZCR value). Dynamic features instead capture the *rate of change* of these static features across consecutive frames — i.e., Delta (first derivative, velocity of change) and Delta-Delta (second derivative, acceleration of

change) coefficients — which encode information about the trajectory/movement of speech articulators over time, not just a single-instant snapshot.

Q6. What are glottal features? Name three examples and their significance.

Answer: Glottal features measure the physical state, tension, and health of the vocal folds. Examples: **Jitter** (short-term instability in the pitch period, indicating irregular vocal-fold vibration), **Shimmer** (short-term instability in peak-to-peak amplitude, indicating irregular loudness), and **Harmonics-to-Noise Ratio (HNR)** (ratio of periodic to noisy energy, indicating voice "clarity" or breathiness). They are significant in clinical speech pathology (detecting voice disorders like vocal-fold nodules), age/gender classification, and speaker identification.

Chapter 12

Cepstral Analysis

Real-life scenario: Imagine trying to figure out a singer’s vocal quality from a song that also has a strong, repetitive drum beat mixed in. The drum beat creates a very regular, repeating ripple in the spectrum (because it’s periodic), while the singer’s vocal-tract shape creates a smoother, slower pattern. Cepstral analysis is a clever trick that lets us mathematically “unmix” these two — separating the fast, repetitive ripple (like the drumbeat, or in speech, the pitch) from the slow, smooth shape (like the singer’s tone, or in speech, the vocal-tract/formant shape).

12.1 What is the Cepstrum?

Cepstrum Analysis is a **nonlinear** signal processing technique performed on a time-domain signal to study its **periodic content** — such as harmonics, sidebands, and orders. The word “**cepstrum**” was coined by **reversing the first syllable** in the word “**spectrum**” (spec → ceps). Correspondingly, the cepstrum exists in a domain called “**quefrency**” (a reversal of the first syllable in “**frequency**”), which — importantly — has **units of time**, not frequency.

The cepstrum is computed by:

1. Taking the **Fourier Transform** of a time signal to create a frequency spectrum,
2. **Converting it to a log scale**,
3. Performing an **inverse Fourier Transform** on this log spectrum.

Because it involves computing the spectrum of the signal, and then performing *another* spectral-style operation on top of that spectrum, the cepstrum is often described as the “**spectrum of a spectrum.**”

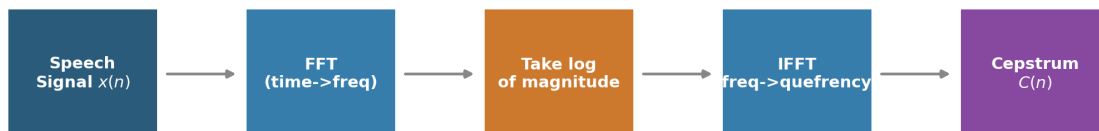


Figure 12.1: The cepstrum pipeline: FFT, then log, then inverse FFT

12.1.1 New Terminology in the Cepstral Domain

Since we are now working in a transformed domain, some familiar frequency-domain terms are given new “reversed” names (a bit of playful jargon, but it does show up in exam questions):

Cepstral-domain term	Corresponds to (frequency-domain analogue)
Quefrency	Frequency (this is the horizontal axis of the cepstrum, with units of time)
Saphe	Phase
Liftering	Filtering (performed on cepstral coefficients)
Rahmonics	Harmonics (harmonic-like patterns observed in the cepstrum)

12.1.2 Why Does This Work? — The Core Insight

Speech harmonics are usually **regularly spaced** in frequency (think of the evenly-spaced rungs of a ladder). When we take the log-magnitude spectrum and then apply the inverse Fourier transform, these regularly-spaced harmonics — which look like a rapidly oscillating ripple in the log-spectrum — collapse into a **single sharp peak** in the cepstral (quefrency) domain, at a position corresponding to their spacing. This is exactly analogous to how the Fourier Transform turns a periodic time-domain signal into a sparse set of frequency-domain peaks — cepstral analysis does the same “compression” trick, one level higher, treating the log-spectrum itself as the “signal” to be analyzed.

A prominent cepstral peak indicates the pitch period of voiced speech. For example, a 100 Hz fundamental frequency corresponds to a $1/100 = 10$ ms pitch period, which will appear as a peak around **10 ms quefrency** in the cepstrum.

12.2 Steps to Compute the Cepstrum

1. **Step 1:** Take the time-domain signal and apply an **FFT** to convert it into the frequency domain.
2. **Step 2:** Take the magnitude of the FFT output (discarding phase for now) and compute its $\log_{10}$. This step reduces the dynamic range of the data (compressing large variations into a smaller range), which makes it much easier to detect the small harmonic ripples that would otherwise be hidden by dominant large peaks.
3. **Step 3:** Convert the resulting log-frequency signal into the **quefrency domain** using an **Inverse Fourier Transform**.

12.2.1 Master Formula

$$C[n] = \sum_{n=0}^{N-1} \log \left(\left| \sum_{n=0}^{N-1} X[n] e^{-j(\frac{2\pi}{N})kn} \right| \right) e^{j(\frac{2\pi}{N})kn}$$

This single formula visually captures the whole “FFT $\rightarrow$ Log $\rightarrow$ Inverse FFT” pipeline: the innermost sum is the forward DFT (green $\rightarrow$ magenta in the color-coded slide), the $\log(\cdot)$ of its magnitude is the orange stage, and the outer sum with $e^{+j(\cdots)}$ is the inverse DFT (green), converting the log-spectrum into the cepstral signal $C[n]$.

$$\text{Time Signal } x(n) \xrightarrow{\text{Fourier Transform}} \xrightarrow{\text{Log}} \xrightarrow{\text{Inverse Fourier Transform}} \text{Cepstral Signal } C(n)$$

12.3 Principle of Cepstral Analysis — Why the Logarithm?

When two signals $G(x)$ and $F(x)$ are **convolved** in the time domain, their spectra are **multiplied** in the frequency domain (Convolution Property of the FT). In the **cepstral domain**, taking the **logarithm** converts this multiplication back into an **addition**:

$$\log |S(\omega)| = \log |G(\omega)| + \log |F(\omega)|$$

This is the entire reason cepstral analysis is powerful: **it can help us separate different components of a signal that were originally combined by convolution/multiplication, more easily** — because after the log step, they simply *add*, and additive components are far easier to separate (e.g., by simple linear filtering / lifting) than multiplicative or convolved ones. For example, in speech processing, cepstral analysis can help identify/separate a human voice from background music or noise (a simple analogy is karaoke, where signal-processing techniques attempt to separate/reduce vocal and musical components).

12.4 Real Cepstrum

The **real cepstrum** of a signal $x(n)$ (sometimes just called “the cepstrum”) is calculated by determining the **natural logarithm of the magnitude** of the Fourier transform of $x(n)$, then obtaining the **inverse Fourier transform** of the resulting sequence:

$$c(n) = \frac{1}{2\pi} \int_{-\pi}^{\pi} \ln |X(\omega)| e^{j\omega n} d\omega$$

where n is the quefrency index, $X(\omega)$ is the frequency spectrum, ω is the digital angular frequency, and $e^{j\omega n}$ is the complex exponential used for the inverse transform. The real cepstrum uses **only the magnitude** of the signal’s spectrum, entirely discarding phase information.

12.4.1 Steps to Calculate the Real Cepstrum

$$c_r(n) = \mathcal{F}^{-1} \{ \ln |\mathcal{F} \{ x(n) \} | \}$$

1. Compute the DFT ($\mathcal{F}$) of the signal $x(n)$.
2. Take the absolute magnitude of the result.

3. Compute the natural logarithm of that magnitude.
4. Take the Inverse DFT (IDFT).

Because the real cepstrum discards phase data, it is used when we only need to analyze the **periodicities** or the **overall shape (envelope)** of a signal — exactly what is needed for identifying pitch and the smooth spectral envelope (vowel/consonant identity), which is why the real cepstrum forms the foundation for MFCC.

Applications of Real Cepstrum:

- **Speech Analysis:** extracting the pitch (fundamental frequency) of a speaker; isolating the spectral envelope, used in speech recognition to identify vowels and consonants.
- **Audio Processing:** it is the direct foundation for computing Mel-Frequency Cepstral Coefficients (MFCCs), the core feature used by voice assistants (e.g., Alexa, Siri) to understand voice commands.

12.5 Complex Cepstrum

The **complex cepstrum** of a sequence $x(n)$ is calculated by finding the **complex natural logarithm** of the Fourier transform of $x(n)$, then taking the inverse Fourier transform of the resulting sequence:

$$\hat{x}(n) = \frac{1}{2\pi} \int_{-\pi}^{\pi} \ln X(\omega) e^{j\omega n} d\omega$$

Unlike the real cepstrum, the complex cepstrum uses **both the magnitude and the phase**, making it a **fully reversible transformation** (the original signal can be exactly recovered from its complex cepstrum).

12.5.1 Steps to Calculate the Complex Cepstrum

$$c_c(n) = \mathcal{F}^{-1}\{\ln(\mathcal{F}\{x(n)\})\}$$

1. Compute the DFT of the signal $x(n)$.
2. Express the result in **polar form**: $X(\omega) = |X(\omega)|e^{j\angle X(\omega)}$.
3. Compute the **complex logarithm**: $\ln(X(\omega)) = \ln|X(\omega)| + j\angle X(\omega)$ (log of magnitude *plus* the phase, appearing as the imaginary part).
4. Take the **Inverse DFT (IDFT)** of this complex log spectrum.

Because the complex cepstrum preserves phase data, it is used when we need to **modify the signal** and convert it back to the time domain accurately (e.g., signal separation/reconstruction tasks that require the original phase information to be retained).

Applications of Complex Cepstrum:

- **Echo Removal & Dereverberation:** identifies the exact delay time of an echo; by removing the echo's peak in the quefrency domain and converting back, echoes can be completely removed from audio/telecom recordings.

- **Speech Synthesis (Text-to-Speech):** allows engineers to separate the vocal-tract filter from the glottal source, so pitch or duration can be modified independently to create natural-sounding synthetic (or “robotic”) speech.
- **Deconvolution of Blurring:** restores signals or 1-D imaging data that have been distorted/blurred by a transmission channel.

Real vs. Complex Cepstrum — Exam Comparison

- **Real Cepstrum:** uses only magnitude $|X(\omega)|$; **not** fully reversible (phase is lost); used for pitch detection and spectral envelope analysis (MFCC foundation).
- **Complex Cepstrum:** uses both magnitude and phase of $X(\omega)$; **is** fully reversible; used for echo removal, TTS source-filter separation, and deconvolution.

12.6 Cepstral Analysis for the Speech Signal — Source-Filter Separation

Real-life scenario: This is exactly how pitch-detection apps (used by singers to check if they’re hitting the right note) and auto-tune software work under the hood. They need to isolate “what pitch am I singing” (the excitation/pitch) completely separately from “what vowel am I singing” (the vocal-tract shape) — and cepstral analysis is the classic technique that makes this separation possible.

Cepstral analysis is primarily used in speech processing to **separate the excitation source (pitch)** from the **vocal-tract filter (formants)**. This mathematical process **deconvolves** a speech signal by transforming a *multiplicative* relationship in the frequency domain into an *additive* one in the cepstral domain.

Speech $s(n)$ is generated by **convolving** an excitation source $e(n)$ (glottal pulses or air noise) with a vocal-tract filter $h(n)$:

$$s(n) = e(n) * h(n)$$

In the standard frequency domain, this convolution becomes a **multiplication**:

$$S(\omega) = E(\omega) \cdot H(\omega)$$

Linear filters cannot easily separate multiplied signals — you can’t just “subtract out” one multiplied component from another. However, the **logarithm transforms this multiplicative relationship into an additive one**, and additive things ARE easy to separate:

$$\log |S(\omega)| = \log |E(\omega)| + \log |H(\omega)|$$

where $\log |H(\omega)|$ represents the **spectral envelope** (vocal-tract shape) and $\log |E(\omega)|$ represents the **rapidly varying ripples** (harmonics/pitch).

Why can we now separate them? Because the vocal tract changes *slowly* over time, its log-spectrum $\log |H(\omega)|$ is **smooth** and varies **slowly** with frequency — this corresponds to **low quefrency** (near $n = 0$) components in the cepstrum. Because the glottal excitation consists of **periodic pulses** (for voiced speech), its log-spectrum $\log |E(\omega)|$ has **sharp, rapid ripples** (harmonics) — this corresponds to **high quefrency** components (a peak around the pitch period) in the cepstrum. Once transformed to the cepstral domain, these two contributions occupy *different, largely non-overlapping* quefrency regions, so they can be separated simply by **lifting** (keeping only low-quefrency components to recover the spectral envelope, or keeping only the high-quefrency peak to recover the pitch period).

Chapter 13

Mel-Frequency Cepstral Coefficients (MFCC)

Real-life scenario: MFCC is arguably the single most important feature in the entire history of speech recognition. Every time Alexa, Siri, Google Assistant, or any pre-2020-era speech recognizer processed your voice, MFCC (or something very similar) was almost certainly the first set of numbers computed from your audio. Even many modern deep-learning systems still use Mel-spectrograms (a close cousin of MFCC) as their starting representation.

13.1 Motivation

Humans do not perceive pitch linearly. Humans are much better at detecting small changes in **low** frequencies than in **high** frequencies (recall the Mel scale from Chapter 9). **Mel-Frequency Cepstral Coefficients (MFCCs)** are designed to exploit exactly this fact — they are the most widely used feature-extraction technique in traditional speech processing, representing the power spectrum of an audio signal based on how the human ear actually perceives sound. MFCC provides a very compact representation — typically only **12-13 coefficients per time frame** — that captures the “fingerprint” of the sound.

13.2 The Seven Steps to Extract MFCC



Figure 13.1: The seven-step MFCC recipe

A/D Conversion → Pre-emphasis → Framing & Windowing → DFT
→ Mel Filterbank → Log → IDFT

13.2.1 Step 1 — A/D Conversion

Samples the audio clip and digitizes the content — i.e., converting the analog signal into discrete space (exactly the sampling/quantization process from Chapter 4). A sampling frequency of **8 kHz or 16 kHz** is often used for speech.

13.2.2 Step 2 — Pre-emphasis

Boosts the amount of energy present in the **high frequencies**. For voiced segments (like vowels), there is naturally *more* energy at lower frequencies than higher frequencies — a phenomenon called **spectral tilt**, related to the physical behavior of the glottal source (how the vocal folds produce sound). Think of it like turning up the “treble” knob on a stereo to compensate for a recording that sounds too bassy — it makes information in the higher formants more available to the acoustic model, and makes the whole system less susceptible to noise introduced later in the pipeline. (Recall the pre-emphasis filter: $H(z) = 1 - \alpha z^{-1}$, with $s'(n) = s(n) - \alpha s(n-1)$, $\alpha \approx 0.95$ to 0.98 .)

13.2.3 Step 3 — Framing and Windowing

Speech is **not stationary** for a long duration, but it can be assumed **approximately stationary over a short interval** — the same idea as taking still photos from a video: each individual photo is “frozen” and simple to analyze, even though the full video shows continuous motion. The speech signal is therefore divided into small, overlapping frames/windows:

- **Frame length** $\approx$ **20-25 ms** — the duration used for each analysis window.
- **Frame shift** $\approx$ **10 ms** — the amount the window moves forward each step.

Because the frame length is *larger* than the frame shift, **consecutive frames overlap**. A window function (typically **Hamming** or **Hanning**) is then multiplied by each frame; this maintains the original frequency information better, with less distortion/noise, compared to using a simple rectangular window (which would introduce sharp discontinuities at the frame edges).

Effect of Window Choice on Spectral Leakage:

Window	Main lobe width	Side-lobe level	Comment
Rectangular	Narrow	Large	High spectral leakage
Hanning (Hann)	Slightly wider	Reduced	Less leakage than rectangular
Hamming	Wider	Good reduction	Good balance of leakage vs. resolution
Blackman	Widest	Strongest suppression	Best leakage suppression, worst resolution

13.2.4 Step 4 — Discrete Fourier Transform (DFT)

Apply the DFT to each windowed frame to extract information in the frequency domain, giving the frame's spectrum.

13.2.5 Step 5 — Mel Filterbank

Equipment measurements of frequency (in Hz) are **not the same** as our hearing perception. For humans, perceived loudness changes non-uniformly with frequency, and perceived frequency resolution **decreases** as frequency increases (i.e., humans are less sensitive to distinguishing high frequencies from each other). The **Mel scale** maps measured Hz-frequency to the frequency resolution actually perceived by humans; this mapping is **non-linear**.

Procedure:

1. **Square the output of the DFT** — this reflects the *power* of the speech at each frequency, $|X[k]|^2$, giving the **DFT power spectrum**.
2. Apply **triangular Mel-scale filter banks** to transform this into the **Mel-scale power spectrum**.
3. The output for each Mel-scale slot represents the energy collected from the range of frequency bands it covers — this is called **Mel Binning**:

$$Y_t[m] = \sum_{k=1}^N W_m[k] |X_t[k]|^2$$

The triangular bandpass filters are made **wider at higher frequencies** to directly reflect the fact that human hearing is less sensitive to fine frequency distinctions there. All of this effort is designed to mimic how the **basilar membrane** in the inner ear senses sound vibrations — the basilar membrane has about **15,000 hair cells** at birth, and the frequency response of these hair cells is approximated by the triangular Mel filters.

13.2.6 Step 6 — Log

The Mel filterbank output is a power spectrum. Just as with frequency perception, humans are **less sensitive to small energy changes at high energy levels** than to the same-sized small changes at low energy levels — this relationship is **logarithmic**. So we take the **log** of the Mel filterbank output. This reduces acoustic variations that are not significant for speech recognition (e.g., compresses the effect of loudness variation).

13.2.7 Step 7 — IDFT (Inverse Discrete Fourier Transform)

The Inverse DFT is applied to **separate the pitch information from the formants** — i.e., this is exactly the cepstral analysis step described in Chapter 12, applied specifically to the log Mel-spectrum. **MFCC keeps only the first 12 (or 12-13) cepstral coefficients and discards the rest**, since these lowest-frequency coefficients capture the smooth spectral envelope (vocal-tract/formant shape) — exactly the information most useful for distinguishing phonemes — while the higher-frequency coefficients mostly reflect fine pitch-harmonic detail, which is less useful (and even undesirable, being speaker/pitch-dependent) for phoneme-based recognition.

MFCC pipeline, one line: Raw Speech $\rightarrow$ A/D $\rightarrow$ Pre-emphasis $\rightarrow$ Framing+Windowing $\rightarrow$ DFT $\rightarrow$ Mel Filterbank (square, then triangular binning) $\rightarrow$ Log $\rightarrow$ IDFT $\rightarrow$ keep first 12-13 coefficients = **MFCC vector**.

13.3 Applications of MFCC

- **Automated Speech Recognition (ASR):** converts spoken language into text (used in virtual assistants like Siri, Alexa).
- **Phoneme Classification:** identifies individual structural sounds (like “sh” or “ah”) to map out spoken words.
- **Keyword Spotting:** powers low-computational-footprint voice-activation triggers like “Hey Siri” or “OK Google.”
- **Speaker Identification:** recognizes who is speaking from a known database, by capturing the unique physical structure of their vocal tract.
- **Speaker Verification:** acts as a security layer to verify a user’s “voiceprint” for banking or secure device access.

Chapter 14

Linear Prediction Cepstral Coefficients (LPCC)

Real-life scenario: Voice-based banking authentication (“my voice is my password”) typically relies on features closer to LPCC than MFCC, because LPCC is better at capturing the physical, biometric shape of *your* particular vocal tract — the very thing that makes your voice uniquely yours, rather than just the words you’re saying.

14.1 What is LPCC?

LPCC (Linear Prediction Cepstral Coefficients) is a digital feature-extraction technique that calculates the smooth spectral envelope of a vocal signal by **merging Linear Predictive Coding (LPC) with Cepstral Analysis**.

MFCC vs. LPCC — the fundamental philosophical difference: MFCC models *human hearing* (the perceptual side of the speech chain); LPCC mathematically models the *biological mechanism of the human vocal tract* (the production side of the speech chain — throat, tongue, mouth). LPCC isolates the source (vocal cords) from the vocal-tract filter (throat, tongue, mouth), just like the source-filter model.

14.2 Steps to Extract LPCC



Figure 14.1: The LPCC recipe: from raw speech to cepstral coefficients via LPC

Pre-emphasis → Frame Blocking → Windowing
→ Autocorrelation Analysis → LPC Analysis → LPC Parameter Conversion
→ Parameter Weighting → Temporal Derivative

14.2.1 Step 1 — Pre-emphasis

The digital speech signal $s(n)$ passes through a **first-order FIR filter** to balance the spectrum:

$$H(z) = 1 - \bar{a}z^{-1}, \quad 0.9 \leq a \leq 1.0$$

This amplifies the higher frequencies that naturally drop off during human speech production. The output of pre-emphasis is:

$$s(n)' = s(n) - \alpha s(n-1), \quad \alpha \approx 0.95 \text{ to } 0.98$$

14.2.2 Step 2 — Frame Blocking

Frame blocking segments the (pre-emphasized) speech waveform into short, overlapping frames so the signal can be treated as approximately stationary. The signal $s(n)'$ is blocked into groups of N samples, with adjacent frames separated by M samples. If there are L total frames, the l -th frame is denoted $x_l(n)$.

14.2.3 Step 3 — Windowing

The continuous audio is sliced into short, overlapping frames (typically **20-30 ms**). A window function — the **Hamming Window** — is multiplied by each frame:

$$w(n) = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right), \quad 0 \leq n \leq N-1$$

This eliminates edge discontinuities and minimizes spectral leakage. The resulting windowed signal is:

$$\bar{x}_\ell(n) = x_\ell(n) w(n), \quad 0 \leq n \leq N-1$$

14.2.4 Step 4 — Autocorrelation Analysis

Each windowed frame undergoes autocorrelation analysis to find periodic repetitions and minimize the error between the actual and predicted signal frames. For each frame, $p+1$ autocorrelation values are computed (where p , the prediction order, is typically **8 to 16**). The **zeroth autocorrelation value**, $R_\ell(0)$, gives the **energy of the ℓ -th frame**; higher lags describe short-term similarity between samples. These autocorrelation values become the input to LPC coefficient computation.

14.2.5 Step 5 — LPC Analysis

LPC analysis converts the $p+1$ autocorrelation values into an **LPC parameter set** — which may be LPC coefficients, reflection coefficients, log-area-ratio coefficients, or cepstral coefficients. The method used to convert autocorrelation coefficients into LPC parameters is called the **Durbin method**, implemented via the **Levinson-Durbin recursion algorithm**:

$$E^{(0)} = r(0), \quad k_i = \left\{ r(i) - \sum_{j=1}^{L-1} \alpha_j^{(i-1)} r(|i-j|) \right\} / E^{(i-1)}, \quad 1 \leq i \leq p$$

$$\alpha_i^{(i)} = k_i, \quad \alpha_j^{(i)} = \alpha_j^{(i-1)} - k_i \alpha_{i-j}^{(i-1)}, \quad E^{(i)} = (1 - k_i^2) E^{(i-1)}$$

At each recursion step, a **reflection coefficient** k_i is obtained and the predictor coefficients are updated; the recursion also tracks the residual **prediction error energy** $E^{(i)}$.

14.2.6 Step 6 — LPC Parameter Conversion

The LPC parameter set that can be derived **directly** from LPC coefficients is the set of **cepstral coefficients** c_m , via a direct recursive formula:

$$c_0 = \ln \sigma^2$$

$$c_m = a_m + \sum_{k=1}^{m-1} \left(\frac{k}{m} \right) c_k a_{m-k}, \quad 1 \leq m \leq p$$

$$c_m = \sum_{k=1}^{m-1} \left(\frac{k}{m} \right) c_k a_{m-k}, \quad m > p$$

This final conversion yields **highly uncorrelated, robust parameters** — c_m is derived recursively from the LPC coefficients $a_1, \dots, a_p$ and the gain term (via c_0). Typically Q **cepstral coefficients** are generated, where Q may be *greater* than the LPC order p (a common rule of thumb: $Q \approx \frac{3}{2}p$).

14.2.7 Step 7 — Parameter Weighting and Temporal Derivatives

One feature vector is produced per analysis frame, containing weighted LPC-derived cepstral coefficients. **Temporal cepstral derivatives** (Delta coefficients) may be appended to capture spectral dynamics. The resulting feature vector is:

$$V(t) = [c_1(t), c_2(t), \dots, c_Q(t), \Delta c_1(t), \Delta c_2(t), \dots, \Delta c_Q(t)]$$

14.3 Applications of LPCC

Because LPCC accurately maps the individual, physical attributes of a person's vocal apparatus, it excels in tasks that focus on **identity and vocal nuance** rather than raw linguistic content:

1. **Speaker Identification & Verification:** biometric security systems use LPCC to recognize an individual's distinct "voice print."
2. **Speech Emotion Recognition (SER):** subtle changes in throat tension due to anger, sadness, or excitement drastically affect the LPCC envelope.
3. **Speech Compression & Coding:** telecommunication networks use LPCC (and LPC) to transmit high-quality speech at incredibly low bit rates.

14.4 MFCC vs. LPCC — Comparison

Metric	LPCC (Linear Prediction Cepstral Coefficients)	MFCC (Mel-Frequency Cepstral Coefficients)
Underlying model	Mimics the biological vocal tract (production side)	Mimics the psychological human ear (perception side)
Frequency scale	Linear frequency scale	Non-linear Mel frequency scale
Noise tolerance	Vulnerable — performance drops in noisy environments	Highly robust against background noise
Best used for	Speaker ID, biometric security, emotion detection	Automatic Speech Recognition (ASR), translation

14.5 Chapter 12-13 : Practice / Exam Questions

Q1. Define cepstrum. Explain the origin of the terms "cepstrum" and "quefrency."

Answer: The cepstrum is the result of taking the Fourier transform of a time-domain signal, computing the logarithm of its magnitude spectrum, and then taking the inverse Fourier transform of that log-spectrum — a "spectrum of a spectrum," used to study periodic content (harmonics) hidden within the original spectrum. The term "cepstrum" was coined by reversing the first syllable of "spectrum"; correspondingly, "quefrency" reverses the first syllable of "frequency" and represents the cepstrum's horizontal axis, measured in units of time.

Q2. What are the steps used to compute the cepstrum of a signal? Write the master mathematical formula.

Answer: Steps: (1) apply FFT to the time signal to get the frequency spectrum; (2) take the log of the magnitude of the FFT output; (3) apply an Inverse Fourier Transform to convert this log-magnitude spectrum into the quefrency (cepstral) domain. Formula: $C[n] = \sum_n \log \left(\left| \sum_n X[n] e^{-j \frac{2\pi}{N} kn} \right| \right) e^{j \frac{2\pi}{N} kn}$.

Q3. Differentiate between the Real Cepstrum and the Complex Cepstrum. Which one is used in MFCC computation, and why?

Answer: The Real Cepstrum uses only the magnitude of the Fourier transform ($c_r(n) = \mathcal{F}^{-1}\{\ln |\mathcal{F}\{x(n)\}|\}$) and is not fully reversible (phase is discarded); it is used for analyzing periodicities/spectral envelope (pitch detection, formant envelope). The Complex Cepstrum uses both magnitude and phase ($c_c(n) = \mathcal{F}^{-1}\{\ln(\mathcal{F}\{x(n)\})\}$) and is fully reversible; it is used for echo removal and source-filter separation for TTS. **MFCC uses the (real) cepstrum** because MFCC only needs the smooth spectral-envelope information (to identify phonemes), not the exact phase, and discarding phase makes the representation simpler, more compact, and more robust.

Q4. Why is the logarithm step essential in cepstral analysis? Explain with reference to the speech production model.

Answer: Speech is produced as $s(n) = e(n) * h(n)$ (excitation convolved with vocal-tract filter), which in the frequency domain becomes a *multiplication*: $S(\omega) = E(\omega)H(\omega)$. Linear filters cannot separate multiplicatively combined signals. Taking the logarithm converts this multiplication into an *addition*: $\log |S(\omega)| = \log |E(\omega)| + \log |H(\omega)|$, and additive components occupying different regions (low-quefrency for the smoothly-varying vocal tract H , high-quefrency for the periodic excitation E) can then be separated by simple liftering. Without the log step, this separation would not be possible using simple linear operations.

Q5. Explain the basic steps involved in extracting MFCC features from a raw speech signal.

Answer: (1) A/D Conversion — digitize the analog signal (8/16 kHz sampling); (2)

Pre-emphasis — boost high-frequency energy to compensate for spectral tilt; (3) Framing & Windowing — slice into 20-25 ms overlapping frames (10 ms shift), apply a Hamming/Hanning window; (4) DFT — convert each frame to the frequency domain; (5) Mel Filterbank — square the DFT output (power spectrum), apply triangular Mel-spaced filters (Mel binning) to mimic human frequency perception; (6) Log — take the logarithm of the Mel filterbank energies (mimicking human loudness perception); (7) IDFT — apply an inverse DFT to separate pitch from formants (cepstral step), keeping only the first 12-13 coefficients as the final MFCC vector.

Q6. Why is pre-emphasis applied before framing in the MFCC pipeline? What is spectral tilt?

Answer: Pre-emphasis is applied early because voiced speech naturally has much more energy at low frequencies than high frequencies (a phenomenon called spectral tilt, caused by the physical behavior of the glottal source). Without correcting for this tilt, the higher formants (which carry important phonetic information) would be under-represented relative to the dominant low-frequency energy, making them harder for the acoustic model to use, and also making the system more susceptible to later-introduced noise. Boosting high-frequency energy via a simple first-order FIR filter ($s'(n) = s(n) - \alpha s(n-1)$) corrects this imbalance before further processing.

Q7. Why is a Hamming or Hanning window preferred over a rectangular window in MFCC/LPCC framing? Discuss spectral leakage with respect to window choice.

Answer: A rectangular window has a narrow main lobe but very large side lobes, causing significant spectral leakage — energy from the true frequency spreads widely across other frequency bins due to the sharp discontinuities it creates at the frame edges. Hamming and Hanning windows taper the signal smoothly to (near) zero at the frame edges, greatly reducing these edge discontinuities and side-lobe leakage (at the cost of a slightly wider main lobe / slightly reduced frequency resolution). Hamming gives a good balance of leakage reduction with reasonable resolution, which is why it is the standard choice in both MFCC and LPCC pipelines.

Q8. Explain why the Mel filterbank uses triangular filters that are wider at higher frequencies.

Answer: Human auditory frequency resolution decreases as frequency increases — i.e., we are much better at distinguishing between two close frequencies at, say, 200 Hz vs. 300 Hz, than between two frequencies that are equally far apart at, say, 7000 Hz vs. 7100 Hz. To mimic this perceptual behavior, the Mel filterbank spaces its triangular filters according to the (compressive, logarithmic-like) Mel scale, which naturally makes the filters narrower (finer resolution) at low frequencies and progressively wider (coarser resolution) at high frequencies — directly matching the frequency-resolving behavior of the human basilar membrane.

Q9. Why does MFCC keep only the first 12-13 cepstral coefficients and discard the rest?

Answer: After the IDFT step, the low-frequency coefficients (roughly the first 12-13) capture the smooth spectral envelope of the vocal tract — i.e., the formant structure that determines phoneme identity — while higher-frequency coefficients mostly capture fine pitch-harmonic detail (the excitation source), which varies with the speaker's pitch and is generally not useful (and can even be counter-productive) for recognizing *what* was said, as opposed to *who* said it or at what pitch. Discarding them keeps the feature vector compact and focused on phonetically relevant information.

Q10. Explain the concept of LPCC. How does it relate to LPC?

Answer: LPCC (Linear Prediction Cepstral Coefficients) is a feature-extraction technique that computes the smooth spectral envelope of speech by combining Linear Predictive Coding (LPC) with cepstral analysis. It relates to LPC directly: the LPC coefficients $a_1, \dots, a_p$ (computed via the autocorrelation method and Durbin's recursion) are converted into cepstral coefficients c_m using a direct recursive formula ($c_0 = \ln \sigma^2$; $c_m = a_m + \sum_{k=1}^{m-1} (k/m) c_k a_{m-k}$ for $m \leq p$; similar formula for $m > p$ without the a_m term). This conversion yields more robust, less-correlated parameters than the raw LPC coefficients themselves.

Q11. List and briefly explain the steps to extract LPCC from a speech signal.

Answer: (1) Pre-emphasis — balance spectrum via a first-order FIR filter; (2) Frame Blocking — segment into overlapping N-sample frames, separated by M samples; (3) Windowing — apply a Hamming window to each frame; (4) Autocorrelation Analysis — compute $p + 1$ autocorrelation values per frame ($R(0)$ = frame energy); (5) LPC Analysis — convert autocorrelation values to LPC coefficients via Durbin's (Levinson-Durbin) recursion; (6) LPC Parameter Conversion — recursively convert LPC coefficients into cepstral coefficients c_m ; (7) Parameter Weighting & Temporal Derivatives — form the final feature vector, optionally appending Delta cepstral coefficients.

Q12. Differentiate between MFCC and LPCC in terms of underlying model, frequency scale, noise tolerance, and best use-cases.

Answer: See the comparison table in Section 13.6: MFCC mimics the human ear (perceptual model), uses a non-linear Mel frequency scale, is highly robust to background noise, and is best for ASR/translation tasks. LPCC mathematically mimics the vocal tract (production model), uses a linear frequency scale, is more vulnerable to noise, and is best for speaker identification, biometric security, and emotion detection tasks.

Q13. Why is LPCC considered more vulnerable to background noise compared to MFCC?

Answer: LPCC is derived from an all-pole linear-prediction model that assumes the signal is well-approximated by resonances of a clean vocal-tract filter driven by a simple excitation; background noise directly corrupts the autocorrelation estimates used to derive the LPC coefficients (since noise adds uncorrelated energy across all lags), distorting the estimated formant/pole locations. MFCC, on the other hand, computes energy within broad Mel-frequency bands (via filterbank summation) be-

fore taking logs, which has an inherent smoothing/averaging effect that makes it comparatively more robust to random additive noise; it does not rely on precisely fitting an all-pole resonance model to the exact (noise-corrupted) spectral fine structure.

Q14. What are the applications of the real cepstrum and complex cepstrum, respectively?

Answer: Real cepstrum: pitch (fundamental frequency) extraction, spectral envelope isolation for vowel/consonant identification in speech recognition, and forms the mathematical foundation of MFCC. Complex cepstrum: echo removal/dereverberation (identifying and removing echo delay peaks in the quefrency domain), speech synthesis/TTS (separating vocal-tract filter from glottal source to independently modify pitch/duration), and deconvolution of signals/images blurred by a transmission channel.

Chapter 15

Dynamic Features and Normalization

Real-life scenario: Think about how you'd recognize someone is about to say "b" vs "p" in fast speech, purely from watching their lips. It's not just the final lip shape that matters — it's *how fast* the lips are moving and in *what direction*, right before and after the sound. Dynamic features capture exactly this "motion" information mathematically, on top of the plain static snapshot (MFCC/LPCC) of each individual frame.

15.1 Delta (Δ) Coefficients

A **Delta coefficient** represents the **rate of change** of a speech feature (e.g., an MFCC coefficient) over time — essentially the *velocity* of the feature's trajectory, similar to how a car's speedometer tells you the rate of change of position, not just the position itself.

$$\Delta c_t = \frac{c_{t+1} - c_{t-1}}{2}$$

where c_t is the feature value at the current frame, c_{t+1} is the feature value at the next frame, c_{t-1} is the feature value at the previous frame, and Δc_t is the rate of change of the feature.

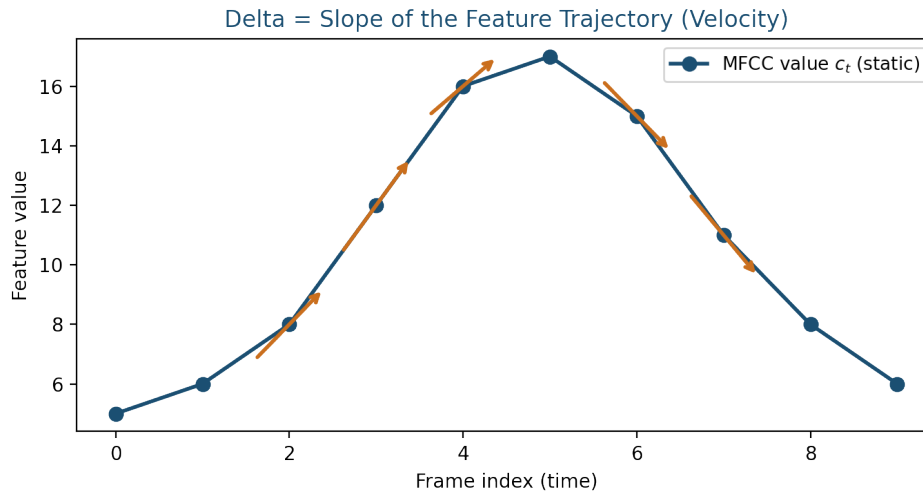


Figure 15.1: Delta = how steeply the feature curve is rising or falling at each point

Worked Example. Suppose three consecutive MFCC values are 10, 12, and 16. Determine the delta coefficient.

Solution: Given $c_{t-1} = 10$, $c_t = 12$, $c_{t+1} = 16$.

$$\Delta c_t = \frac{c_{t+1} - c_{t-1}}{2} = \frac{16 - 10}{2} = 3$$

The positive value of delta (Δ) indicates that the feature is **increasing with time**.

15.2 Delta-Delta ($\Delta\Delta$) Coefficients

A **Delta-Delta coefficient** represents the rate of change of the **Delta coefficient itself** — i.e., it is the *acceleration* of the speech feature (continuing the car analogy: if Delta is speed, Delta-Delta is how hard you're pressing the accelerator/brake).

$$\Delta\Delta c_t = \frac{\Delta c_{t+1} - \Delta c_{t-1}}{2}$$

where Δc_{t-1} is the delta coefficient of the previous frame, Δc_{t+1} is the delta coefficient of the next frame, and $\Delta\Delta c_t$ is the delta-delta coefficient of the current frame.

Worked Example. Given delta values for three consecutive frames: $\Delta c_{t-1} = 2$, $\Delta c_t = 3$, and $\Delta c_{t+1} = 5$. Determine the delta-delta coefficient.

Solution:

$$\Delta\Delta c_t = \frac{\Delta c_{t+1} - \Delta c_{t-1}}{2} = \frac{5 - 2}{2} = 1.5$$

The positive delta-delta value (+1.5) indicates that the **rate of change is itself increasing with time** (the feature trajectory is accelerating).

Worked Example (Full 5-frame calculation). Consider five consecutive frames of one MFCC coefficient: $c_{t-2} = 8$; $c_{t-1} = 10$; $c_t = 12$; $c_{t+1} = 16$; $c_{t+2} = 18$. Determine the delta and delta-delta coefficients at frame t .

Solution:

Step 1 — Delta at frame t :

$$\Delta c_t = \frac{c_{t+1} - c_{t-1}}{2} = \frac{16 - 10}{2} = 3$$

Step 2 — Delta at frame $t - 1$ (using the frames immediately before and after $t - 1$, i.e., $t - 2$ and t):

$$\Delta c_{t-1} = \frac{c_t - c_{t-2}}{2} = \frac{12 - 8}{2} = 2$$

Step 3 — Delta at frame $t + 1$ (using the frames immediately before and after $t + 1$, i.e., t and $t + 2$):

$$\Delta c_{t+1} = \frac{c_{t+2} - c_t}{2} = \frac{18 - 12}{2} = 3$$

Step 4 — Delta-Delta at frame t :

$$\Delta \Delta c_t = \frac{\Delta c_{t+1} - \Delta c_{t-1}}{2} = \frac{3 - 2}{2} = 0.5$$

The positive Delta-Delta value (+0.5) indicates that the rate of change of the MFCC coefficient is itself gently increasing (mild positive acceleration) around frame t .

Why do we need Delta and Delta-Delta features at all? Static features (a single frame's MFCC vector) only capture a "snapshot" of the vocal-tract shape at one instant. But phonemes are fundamentally distinguished not just by their static shape but by how quickly and in what direction the vocal tract is *moving* through that shape (co-articulation effects, transitions between phonemes). Appending Delta and Delta-Delta coefficients to the static MFCC/LPCC vector lets the recognizer directly see this trajectory information (velocity and acceleration), which has been empirically shown to substantially improve speech recognition accuracy.

15.3 Normalization

15.3.1 Why Normalize?

Real-life scenario: If you record the same sentence on your laptop's built-in mic versus a professional studio microphone, the raw numbers will look quite different — even though you said the exact same words. That's an annoying, irrelevant difference we'd like to cancel out before comparing recordings. Normalization is speech processing's way of saying, "ignore the recording equipment, focus on what was actually said."

Raw feature values (e.g., MFCCs) can vary considerably due to factors unrelated to pho-

netic content — different microphones, recording channels, speaker volume, or channel distortions can shift or scale the feature values. **Normalization** removes such irrelevant, systematic variation, making features more comparable across different recording conditions and speakers.

15.3.2 Mean and Variance Normalization

Mean Normalization:

1. Calculate the **mean** of the feature values:

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i$$

where μ is the mean, N is the number of frames, x_i is the feature value at the i -th frame.

2. **Subtract** the mean from each feature value:

$$x'_i = x_i - \mu$$

Variance Normalization: to make the spread of the values consistent, divide x'_i by the standard deviation:

$$x''_i = \frac{x_i - \mu}{\sigma}, \quad \text{where } \sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$

Worked Example. Suppose the values of one MFCC coefficient from four speech frames are: 8, 10, 12, 14. Perform mean and variance normalization.

Solution:

Step 1 — Calculate the mean:

$$\mu = \frac{8 + 10 + 12 + 14}{4} = \frac{44}{4} = 11$$

Step 2 — Subtract the mean ($x_i - \mu$):

$$x_1 - \mu = 8 - 11 = -3, \quad x_2 - \mu = 10 - 11 = -1$$

$$x_3 - \mu = 12 - 11 = 1, \quad x_4 - \mu = 14 - 11 = 3$$

Mean normalization is complete: the mean-normalized sequence is $\{-3, -1, 1, 3\}$.

Step 3 — Calculate standard deviation:

$$\sigma^2 = \frac{(-3)^2 + (-1)^2 + (1)^2 + (3)^2}{4} = \frac{9 + 1 + 1 + 9}{4} = \frac{20}{4} = 5$$

$$\Rightarrow \sigma = \sqrt{5} = 2.236$$

(A small standard deviation means the feature values are close to their mean; a large standard deviation means they are more widely spread.)

Step 4 — Divide by standard deviation:

$$x_1'' = \frac{8 - 11}{2.236} = \frac{-3}{2.236} = -1.34$$

Similarly, $x_2'' = \frac{-1}{2.236} = -0.447$, $x_3'' = \frac{1}{2.236} = 0.447$, $x_4'' = \frac{3}{2.236} = 1.34$.

Final normalized values: $\{-1.34, -0.447, 0.447, 1.34\}$.

15.3.3 Cepstral Mean Normalization (CMN)

Cepstral Mean Normalization (CMN) is a specific application of mean normalization to cepstral features, such as MFCCs. It calculates the mean of each cepstral coefficient over a speech segment, and subtracts that mean from each coefficient:

$$c'_{t,k} = c_{t,k} - \mu_k$$

where $c_{t,k}$ is the k -th cepstral coefficient at frame t , μ_k is the mean of the k -th coefficient over the speech segment, and $c'_{t,k}$ is the normalized coefficient.

Why is CMN especially useful? A large part of channel distortion (e.g., a particular microphone's frequency response, or a constant-gain telephone line effect) appears as a roughly **constant additive bias** in the log/cepstral domain (recall: convolution in time = multiplication in frequency = addition in log/cepstral domain). Subtracting the segment mean from each cepstral coefficient removes exactly this kind of constant channel bias, making the extracted MFCC/LPCC features far more robust to changes in recording equipment or channel — a critical technique for building speaker- and channel-independent speech recognition systems.

Chapter 16

Vector Quantization (VQ)

Real-life scenario: Think of a paint shop that offers only 24 “standard” paint colors, even though in principle there are millions of possible RGB colors. When you bring in a custom color to match, the shop finds the *closest* standard color from their limited palette. Vector Quantization does exactly this for speech features: instead of dealing with infinitely many possible feature vectors, it builds a small “palette” (codebook) of representative vectors and matches every incoming frame to its closest palette entry.

16.1 Motivation for Vector Quantization

Vector Quantization (VQ) is used to represent a **large number** of speech feature vectors using a **small set of representative vectors**, called **codewords**. When speech is processed, each frame can produce a feature vector, such as an MFCC vector: $X_t = [c_1, c_2, c_3, \dots, c_n]$. Working with a very large number of these vectors requires substantial computation, storage, and time. VQ addresses this by:

- Reducing the number of vectors that need to be stored/processed, saving computational storage and time.
- Reducing overall computational complexity.
- Grouping similar MFCC (or other feature) vectors and representing them with a single, shared codeword.

16.2 Vector Quantization Terminology

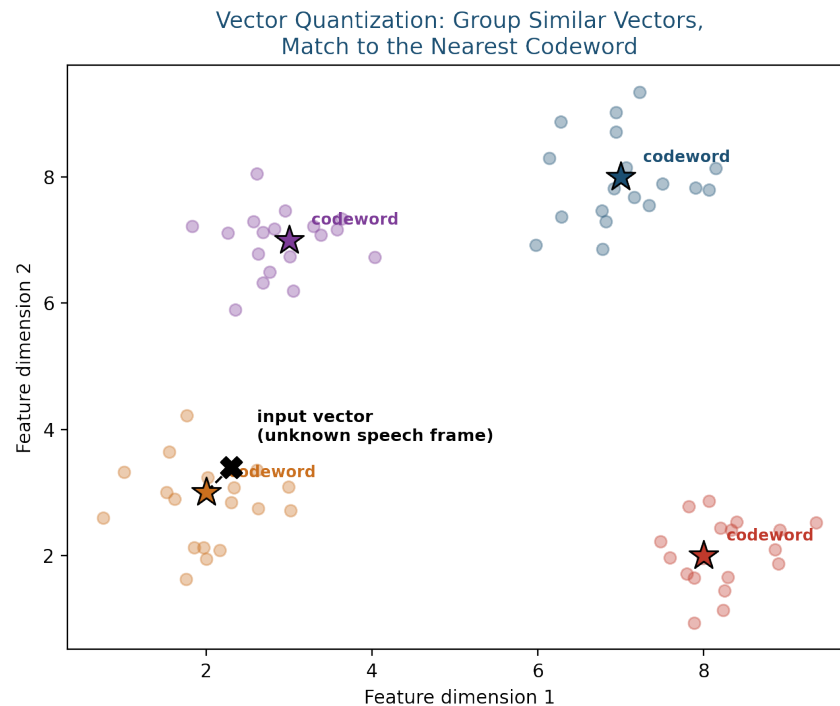


Figure 16.1: VQ groups similar feature vectors and represents each group by one codeword

- **Vector:** a collection of feature values representing one speech frame. Example: if we extract 3 MFCC coefficients from a speech frame, $X = [4.2, 7.5, 3.6]$ — this is a 3-dimensional feature vector.
- **Codeword:** a representative vector that stands in for a *group* of similar feature vectors. Example: if several MFCC vectors are $[2.0, 3.4, 1.9]$, $[2.2, 3.6, 1.7]$, $[2.1, 3.5, 1.8]$ — these are quite similar, so VQ can represent them all using one representative vector, $C_1 = [2.1, 3.5, 1.8]$.
- **Codebook:** the collection of *all* the codewords: $C = \{C_1, C_2, C_3, C_4\}$.

16.3 Distance Measure — Euclidean Distance

In Vector Quantization, we need to determine **which codeword is most similar** to a given speech feature vector. To do this, we calculate the **distance** between the input vector and each codeword; the codeword with the **smallest distance** is considered the closest match.

Euclidean Distance measures the straight-line distance between two vectors. For two n -dimensional vectors $X = [x_1, x_2, \dots, x_n]$ and $C = [c_1, c_2, \dots, c_n]$:

$$d(x, c) = \sqrt{\sum_{i=1}^n (x_i - c_i)^2}$$

where $d(x, c)$ is the Euclidean distance, x_i is the i -th component of the input vector, and c_i is the i -th component of the codeword.

Worked Example. An input MFCC vector is $x = [2, 3]$, and we have two codewords: $C1 = [1, 2]$ and $C2 = [5, 4]$. Determine which codeword the vector x belongs to.

Solution:

Step 1 — Euclidean distance to C1:

$$d(x, C1) = \sqrt{(2 - 1)^2 + (3 - 2)^2} = \sqrt{1 + 1} = \sqrt{2} \approx \mathbf{1.41}$$

Step 2 — Euclidean distance to C2:

$$d(x, C2) = \sqrt{(2 - 5)^2 + (3 - 4)^2} = \sqrt{9 + 1} = \sqrt{10} \approx \mathbf{3.16}$$

Step 3 — Decision: the input vector always belongs to the closest codeword, i.e., the one with the **lower** Euclidean distance. Since $d(x, C1) = 1.41 < d(x, C2) = 3.16$, **the vector x belongs to codeword C1.**

16.4 Codebook Generation (Brief Overview — LBG Algorithm)

A codebook is typically generated using an iterative clustering algorithm such as the **LBG (Linde-Buzo-Gray) algorithm**, a generalization of k-means clustering for vector quantization:

1. Start with an initial codebook (e.g., a single codeword equal to the mean of all training vectors).
2. **Split** each codeword into two (perturbing it slightly), doubling the codebook size.
3. **Assign** each training vector to its nearest codeword (using Euclidean distance).
4. **Update** each codeword to be the centroid (mean) of all vectors assigned to it.
5. Repeat steps 3-4 until convergence (codewords stop changing significantly).
6. Repeat steps 2-5 (splitting) until the desired codebook size is reached.

16.5 Applications of Vector Quantization

- **Speech Compression/Coding:** representing speech using a small codebook index instead of full feature vectors dramatically reduces bit rate for storage/transmission.
- **Speaker Recognition:** a speaker-specific codebook can be built from that speaker's training data; during recognition, the input speech's average quantization distortion against each speaker's codebook is compared, and the speaker with the lowest distortion is identified.

- **Discrete HMM-based Speech Recognition:** VQ is used to convert continuous-valued feature vectors into discrete symbols (codebook indices), which is a prerequisite for using a **discrete HMM** (see Chapter 15).

16.6 Chapter 15-16 : Practice / Exam Questions

Q1. Define delta and delta-delta coefficients. Write their mathematical formulas and explain their physical significance.

Answer: Delta coefficients ($\Delta c_t = (c_{t+1} - c_{t-1})/2$) represent the rate of change (velocity) of a static feature over time — how quickly the spectral shape is transitioning between phonemes. Delta-Delta coefficients ($\Delta\Delta c_t = (\Delta c_{t+1} - \Delta c_{t-1})/2$) represent the rate of change of the Delta coefficients (acceleration) — how quickly the *rate* of spectral change is itself changing. Physically, these capture articulator movement dynamics (tongue, lips, jaw velocity and acceleration) that static features alone cannot represent, and they significantly improve speech/phoneme recognition accuracy when appended to static MFCC/LPCC vectors.

Q2. Why are dynamic features (Delta, Delta-Delta) important in speech recognition, in addition to static features?

Answer: Static features only describe an isolated snapshot of the vocal-tract shape at one instant, ignoring how it got there or where it is headed. But phoneme identity, especially in continuous speech, is heavily influenced by coarticulation — how a sound transitions from and into neighbouring sounds. Dynamic features directly encode this transition information (velocity and acceleration of the feature trajectory), providing crucial context that dramatically improves the discriminative power and robustness of the overall feature representation, especially for consonants and transitions, which are inherently more about *change* than static shape.

Q3. Define normalization in the context of speech features. Explain mean normalization and variance normalization with formulas.

Answer: Normalization adjusts feature values to remove irrelevant, systematic variation (e.g., due to microphone, channel, or recording-level differences) so that features become more directly comparable across different recordings/speakers. **Mean normalization:** compute the mean $\mu = \frac{1}{N} \sum x_i$ over a segment, then subtract it from every value: $x'_i = x_i - \mu$. **Variance normalization:** additionally divide by the standard deviation σ : $x''_i = (x_i - \mu)/\sigma$, which further standardizes the spread/scale of the feature values.

Q4. What is Cepstral Mean Normalization (CMN)? Why is it particularly effective at removing channel distortion?

Answer: CMN is the application of mean normalization specifically to cepstral coefficients (e.g., MFCC): $c'_{t,k} = c_{t,k} - \mu_k$, where μ_k is the mean of the k -th cepstral coefficient computed over an utterance/segment. It is particularly effective against channel distortion because a channel's frequency response acts as a roughly time-invariant multiplicative filter in the frequency domain, which becomes an *additive, constant bias* once we take logs (cepstral domain) — subtracting the segment mean from each coefficient removes exactly this constant bias, leaving the channel-independent, phonetically relevant variation intact.

Q5. Define Vector Quantization. Why is it used in speech processing? Define vector, codeword, and codebook.

Answer: Vector Quantization (VQ) represents a large number of speech feature vectors using a small set of representative vectors called codewords, reducing storage, computation, and processing time while grouping similar vectors together. **Vector:** a collection of feature values representing one speech frame (e.g., a 3-D MFCC vector). **Codeword:** a single representative vector standing in for a cluster of similar feature vectors. **Codebook:** the complete collection of all codewords used by the VQ system.

Q6. Define Euclidean Distance and explain its role in Vector Quantization. Given an input vector and a set of codewords, how do you decide which codeword the input belongs to?

Answer: Euclidean distance measures the straight-line distance between two n -dimensional vectors: $d(x, c) = \sqrt{\sum_{i=1}^n (x_i - c_i)^2}$. In VQ, it is used to measure how similar an input feature vector is to each codeword in the codebook. The input vector is assigned to (i.e., "belongs to") the codeword with the **smallest** Euclidean distance — this codeword is considered the closest match/best representative for that input vector.

Q7. Briefly describe how a VQ codebook is generated (LBG algorithm).

Answer: The LBG algorithm starts with a single codeword (the mean of all training vectors), then iteratively: (1) splits each existing codeword into two slightly perturbed codewords (doubling codebook size); (2) assigns every training vector to its nearest codeword (Euclidean distance); (3) updates each codeword to be the centroid of the vectors assigned to it; (4) repeats assignment/update until convergence; and (5) repeats the whole split-converge cycle until the desired codebook size is reached.

Q8. What are the applications of Vector Quantization in speech processing?

Answer: Speech compression/coding (transmitting codebook indices instead of full feature vectors, reducing bit rate); speaker recognition (comparing input speech's quantization distortion against speaker-specific codebooks); and enabling discrete HMM-based speech recognition (converting continuous feature vectors into discrete symbol sequences required by discrete HMMs).

Chapter 17

Pattern Matching Approaches

Real-life scenario: When you unlock your phone using face recognition, the phone doesn't demand a pixel-perfect match to the exact photo it stored during setup (your face changes slightly every time — lighting, expression, angle). Instead, it finds the *closest match* within some tolerance. Speech recognition faces the exact same challenge: no two utterances of the same word are ever identical, so we need smart pattern-matching techniques that can tolerate natural variation while still picking the correct word.

17.1 What is Pattern Matching in Speech Recognition?

Pattern matching is a fundamental process in speech recognition that helps identify an **unknown speech input** by comparing its acoustic feature patterns with **stored reference patterns or trained models**. Since speech is highly variable — due to differences in speakers, speaking rate, pronunciation, noise, and linguistic context — pattern matching provides a systematic way to handle this variability and determine the most likely word or phoneme.

In speech recognition, the input is typically represented as a **sequence of acoustic feature vectors**, and the system selects the word or phoneme whose model provides the **best overall match**. Effective pattern matching is therefore essential for achieving accurate and reliable speech recognition.

17.2 Categories of Pattern Matching Approaches

1. **Template-based Pattern Matching:** Distance-based methods, **Dynamic Time Warping (DTW)**.
2. **Statistical/Probabilistic Modeling:** **Hidden Markov Model (HMM)**, **Gaussian Mixture Model (GMM)**.
3. **Model-based/Knowledge-based Recognition:** Phonetic/linguistic models, knowledge-based models.

17.2.1 Template Matching

- A **reference template** is stored for each word or pattern (e.g., recorded during a training/enrollment phase — similar to how you “enroll” your fingerprint on a phone).

- The unknown utterance is compared **directly** with each stored template.
- The template giving the **smallest distance** (or largest similarity) determines the recognized word.
- **Simple but sensitive** to speaker and speaking-rate variations, since it does not inherently handle timing differences between the template and the test utterance (this is where DTW helps, by non-linearly “warping” the time axis to find the best alignment before measuring distance — a bit like stretching or squeezing an audio clip in an editing app until it lines up with a reference track).

17.3 Distance Measures

A key component of speech pattern matching is measuring the **dissimilarity** between two feature vectors. The distance measure should ideally reflect the **perceptual or subjective differences** between speech signals — traditional measures like plain Euclidean distance or squared error may not adequately capture the perceptual importance of different speech features. Specialized spectral distortion measures used in speech processing include:

- **Log Spectral Distance**
- **Cepstral Distance**
- **Weighted Cepstral Distance**
- **Likelihood Ratio Distortion**
- **Weighted Likelihood Ratio Distortion**
- **Weighted Slope Metric**

17.4 Why Does Simple Matching Fail for Speech?

- Two people can pronounce the same word with very different acoustic characteristics (different vocal-tract sizes, accents).
- The same speaker can say the same word at different speeds on different occasions.
- Some phonemes naturally occupy more frames (last longer) than others.
- Noise and channel conditions can alter feature vectors unpredictably.

Because of all these sources of variability, simple, rigid template matching is insufficient. **Temporal alignment** (handling variable speaking rate — solved by DTW or, implicitly, by the HMM’s state-transition structure) and **probabilistic modeling** (handling acoustic variability — solved by HMM/GMM) are both needed for robust, real-world speech recognition.

Chapter 18

Hidden Markov Model (HMM) Based Speech Recognition

Real-life scenario: Think about predicting tomorrow’s weather. You can’t directly “see” the atmospheric pressure system (the hidden cause), but you can observe today’s weather (rain, sun, clouds) and use it to guess what’s likely tomorrow. HMMs work on exactly this idea: there’s a hidden, unobservable process (which part of a word is currently being spoken) that we can’t see directly, but it produces observable evidence (the acoustic sound) that we *can* measure, and from that evidence we try to figure out what the hidden process was doing.

18.1 What is a Hidden Markov Model?

A **Hidden Markov Model (HMM)** represents speech as a sequence of **hidden states** and **observable acoustic feature vectors**. The hidden states represent different temporal parts of a speech unit, such as the beginning, middle, and end of a phoneme. The model describes:

- How speech moves from one state to another, using **transition probabilities**.
- How likely a particular acoustic feature is to occur in each state, using **emission probabilities**.

The states are called “**hidden**” because they cannot be directly observed — only the corresponding acoustic features (observations) generated by those states can be observed.

- **Hidden States** ($q_1, q_2, q_3, \dots$): represent the underlying (unobservable) process; connections between states represent transition probabilities from one state to the next.
- **Observations** ($x_1, x_2, x_3, \dots$): the visible acoustic feature data generated at each time step; connections from hidden states to observations represent emission probabilities.

18.2 Parameters of an HMM

An HMM is formally represented by $\lambda = (A, B, \pi)$ and consists of the following parameters:

- **Observation sequence:** $X = \{x_1, x_2, \dots, x_t, \dots, x_T\}$, where x_t is the observed acoustic feature at time t .
- **States:** $S = \{S_1, \dots, S_N\}$, representing the hidden temporal conditions of the speech signal.
- **Initial state probability (π):** $\pi_i = P(q_1 = S_i)$ — the probability that the model starts in state S_i .
- **Transition probability (A):** $a_{ij} = P(q_{t+1} = S_j \mid q_t = S_i)$ — the probability of moving from state S_i to state S_j .
- **Observation emission probability (B):** $b_j(x_t) = P(x_t \mid q_t = S_j)$ — the probability of observing x_t when the system is in state S_j .

18.3 Left-to-Right (Bakis) HMM for Speech

Speech recognition typically relies on **left-to-right (Bakis) HMMs** to capture how speech naturally unfolds over time: states progress from the beginning to the middle to the end of a speech unit, with **backward transitions generally disallowed** in the basic configuration (this matches the fact that speech, once produced, does not “go backward” in time — you can’t un-say the beginning of a word).

Left-to-Right (Bakis) HMM with 3 States

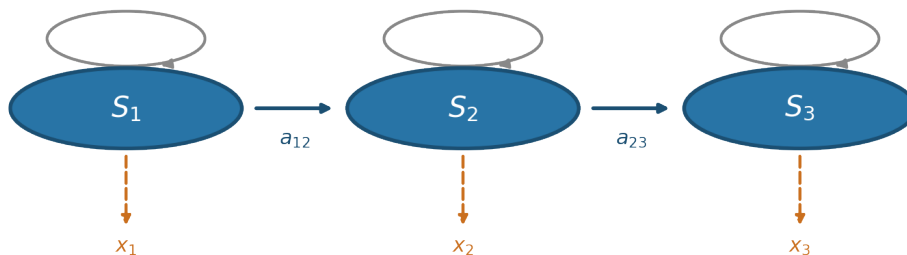


Figure 18.1: A left-to-right HMM: states only move forward or stay put, just like speech itself

- A standard **phoneme model** typically uses **three emitting states**, representing the onset, steady-state, and offset of an acoustic region.
- **Word-level models** are constructed by **concatenating** individual phoneme HMMs — a word HMM is a composite HMM built by chaining together smaller sub-word units (phonemes), the same way you’d build a word out of LEGO-brick phonemes.

Example: the phonetic transcription of the English word “cat” is /k/ /æ/ /t/. Each phoneme has its own trained HMM (a box in the model diagram). Connecting these individual phoneme HMMs in a linear sequence (/k/-HMM → /æ/-HMM → /t/-HMM) forms the complete word-level HMM used to recognize “cat.”

18.4 The Three Classic Problems of HMM

Real-life scenario: Imagine Google Translate’s speech feature. It must (1) score how likely a given phrase model matches what you said (Evaluation), (2) figure out the actual best sequence of sounds you spoke (Decoding), and (3) have been trained beforehand on thousands of hours of speech to know its parameters in the first place (Learning). These three needs map exactly onto the three classic HMM problems below.

The Hidden Markov Model framework is built around **three main problems**, each with its own standard solution algorithm:

Problem	Question Asked	Solving Algorithm
1. Evaluation — $P(X \lambda)$	Given an observation sequence and a trained model, what is the probability that this model generated this sequence?	Forward Algorithm
2. Decoding (state path)	Given an observation sequence and a model, what is the single most likely sequence of hidden states that produced those observations?	Viterbi Algorithm
3. Learning — Estimate λ	Given a set of observation sequences, how do we estimate the model’s parameters (A, B, π) to best fit the data?	Baum-Welch Algorithm (Forward-Backward)

18.5 The Forward Algorithm

Real-life scenario: Imagine you’re playing a guessing game where you need to figure out the total probability of a friend arriving home safely, given they could have taken any of dozens of possible bus + walking route combinations. Instead of tracing every single possible route individually (which would take forever), you cleverly compute running totals at each intersection, reusing previous work. That is exactly the trick the Forward Algorithm uses.

The Forward Algorithm is a **dynamic programming method** used in HMMs to compute the **total probability** of generating a specific observation sequence given a model ($P(O | \lambda)$). It uses a **forward variable**, $\alpha_t(i)$, which represents the joint probability of the partial observation sequence up to time t **and** being in hidden state S_i at time t .

$$\text{Initialization } (t = 1) : \quad \alpha_1(i) = \pi_i b_i(o_1)$$

$$\text{Recursion } (t = 2 \text{ to } T) : \quad \alpha_t(j) = \left[\sum_{i=1}^N \alpha_{t-1}(i) a_{ij} \right] b_j(o_t)$$

$$\text{Termination: } P(O | \lambda) = \sum_{i=1}^N \alpha_T(i)$$

Intuition for each step:

- **Initialization (the starting point):** at $t = 1$, calculate the starting score for each state by multiplying the probability that state i produces the first observation ($b_i(o_1)$) with the probability of starting in state i (π_i).
- **Recursion (moving forward):** for every step from time 2 until the end T , calculate the new scores based on the old ones — sum over all states i you could have just come from ($\alpha_{t-1}(i)$), multiply each by the transition probability to your new state j (a_{ij}), sum these up, then multiply by the probability that your new state produces the current observation ($b_j(o_t)$).
- **Termination (the final total):** once you reach the final time step T , you have a final score for every possible ending state; adding all of these together gives the complete probability $P(O | \lambda)$ of that entire observation sequence occurring, summed over all possible hidden-state paths.

Worked Example — Forward Algorithm. Given a two-state HMM with hidden states S_1 and S_2 , an initial state probability vector $\pi = [0.6, 0.4]$, and a state transition probability matrix $A = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix}$, calculate the total probability $P(X | \lambda)$ of generating the specific sequence of two observations $X = (x_1, x_2)$ using the Forward Algorithm, where the emission probabilities for the first observation are $b_1(x_1) = 0.5$ and $b_2(x_1) = 0.1$, and for the second observation are $b_1(x_2) = 0.4$ and $b_2(x_2) = 0.5$.

Solution:

Step 1 — Initialization ($t = 1$):

$$\alpha_1(1) = \pi_1 \times b_1(x_1) = 0.6 \times 0.5 = 0.30$$

$$\alpha_1(2) = \pi_2 \times b_2(x_1) = 0.4 \times 0.1 = 0.04$$

Step 2 — Recursion ($t = 2$):

For State 1:

$$\begin{aligned} \alpha_2(1) &= [\alpha_1(1)a_{11} + \alpha_1(2)a_{21}] b_1(x_2) = [0.30(0.7) + 0.04(0.4)] \times 0.4 \\ &= [0.21 + 0.016] \times 0.4 = 0.226 \times 0.4 = \mathbf{0.0904} \end{aligned}$$

For State 2:

$$\begin{aligned} \alpha_2(2) &= [\alpha_1(1)a_{12} + \alpha_1(2)a_{22}] b_2(x_2) = [0.30(0.3) + 0.04(0.6)] \times 0.5 \\ &= [0.09 + 0.024] \times 0.5 = 0.114 \times 0.5 = \mathbf{0.057} \end{aligned}$$

Step 3 — Termination: sum the final scores of all ending states:

$$P(X | \lambda) = \alpha_2(1) + \alpha_2(2) = 0.0904 + 0.057 = \boxed{0.1474}$$

18.6 The Viterbi Algorithm

Real-life scenario: Google Maps doesn't show you every possible route to your destination — it shows you the *single best* one, discarding all clearly worse alternatives along the way. That's precisely what the Viterbi algorithm does for speech recognition: out of every conceivable sequence of hidden states that could explain what you said, it finds and reports only the single best (most probable) one.

The **Viterbi Algorithm** finds the **single most probable sequence of hidden states** that could have generated a given observation sequence. This single optimal path (the maximum-likelihood path) is called the **Viterbi path**. It uses dynamic programming, keeping track of the best score at each step and **discarding sub-optimal paths** at every step (unlike the Forward algorithm, which *sums* over all paths, Viterbi keeps only the single *best* path into each state).

When a user speaks an utterance, the system receives a sequence of acoustic feature frames (observations). Using concatenated phoneme and word HMMs (representing all possible spoken words), a massive network of hidden states is formed; the Viterbi algorithm efficiently navigates this network to extract the single most probable string of words/phonemes, handling temporal alignment along the way.

$$\begin{aligned} \text{Initialization } (t = 1) : \quad & \delta_1(i) = \pi_i \times b_i(o_1) \\ \text{Recursion } (t = 2 \text{ to } T) : \quad & \delta_t(j) = \max_{1 \leq i \leq N} [\delta_{t-1}(i) a_{ij}] b_j(o_t) \end{aligned}$$

- **Backtracking storage:** at each step, the identity of the best previous state is stored in a tracking variable $\psi_t(j)$, so the optimal path can be reconstructed later.
- **Termination:** at the final frame, select the largest $\delta_T(j)$, then **backtrack** using ψ from T down to 1 to reconstruct the optimal state sequence.

Key difference from Forward Algorithm: the Forward algorithm uses a **sum** ($\sum$) over incoming paths (because it wants the *total* probability across all paths); the Viterbi algorithm uses a **max** (max) over incoming paths (because it wants only the *single best* path).

Worked Example — Viterbi Algorithm. Using the same two-state HMM as before ($\pi = [0.6, 0.4]$, $A = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix}$, $b_1(x_1) = 0.5$, $b_2(x_1) = 0.1$, $b_1(x_2) = 0.4$, $b_2(x_2) = 0.5$), find the most likely state sequence for $X = (x_1, x_2)$ using the Viterbi Algorithm.

Solution:

Step 1 — Initialization ($t = 1$):

$$\delta_1(1) = \pi_1 \times b_1(x_1) = 0.6 \times 0.5 = 0.30 \quad \delta_1(2) = \pi_2 \times b_2(x_1) = 0.4 \times 0.1 = 0.04$$

Step 2 — Recursion ($t = 2$), using max instead of sum:

State 1:

$$\begin{aligned} \delta_2(1) &= \max(0.30 \times 0.7, 0.04 \times 0.4) \times 0.4 = \max(0.21, 0.016) \times 0.4 \\ &= 0.21 \times 0.4 = \mathbf{0.084} \end{aligned}$$

(best predecessor for state 1 at $t = 2$ is **State 1**, since $0.21 > 0.016$)

State 2:

$$\begin{aligned}\delta_2(2) &= \max(0.30 \times 0.3, 0.04 \times 0.6) \times 0.5 = \max(0.09, 0.024) \times 0.5 \\ &= 0.09 \times 0.5 = \mathbf{0.045}\end{aligned}$$

(best predecessor for state 2 at $t = 2$ is also **State 1**, since $0.09 > 0.024$)

Step 3 — Termination and Backtracking: the best final score is $\max(0.084, 0.045) = 0.084$, pointing to **State 1** at $t = 2$. Tracing back, the predecessor of State 1 (at $t = 2$) was State 1 (at $t = 1$).

Result: the most likely sequence of hidden states that produced the observations (x_1, x_2) is $S_1 \rightarrow S_1$.

Note: while the Forward Algorithm calculates the total combined probability of an entire observation sequence summed across *all* possible paths (here, 0.1474), the Viterbi Algorithm isolates only the single most likely sequence of hidden states (here, $S_1 \rightarrow S_1$, with score 0.084) — these two numbers are generally different, and both are useful for different purposes (Forward for scoring/comparing models, Viterbi for actual decoding of what was said).

18.7 The Baum-Welch Algorithm (Training)

The **Baum-Welch Algorithm** solves the third HMM problem: **training** — estimating the HMM parameters (A, B, π) from a set of acoustic training sequences.

- It uses the **Forward-Backward** procedure to compute **state occupancy probabilities** (the probability of being in a particular state at a particular time, given the entire observation sequence).
- Parameters are **re-estimated iteratively** to increase the likelihood of the training data under the model.
- Baum-Welch is a specific instance of the general **Expectation-Maximization (EM)** algorithm: the “Expectation” step computes expected state occupancies using the current parameters; the “Maximization” step updates the parameters to maximize the likelihood given those expected occupancies; these two steps alternate until convergence.
- In classical ASR systems, many HMM states use **Gaussian** or **Gaussian Mixture Model (GMM)** emission distributions (rather than simple discrete probabilities) to model continuous-valued acoustic feature vectors.

18.8 Continuous and Discrete HMM

- **Discrete HMM:** if the observations are discrete, $b_{q_t}(x_t)$ is a **probability mass function**. Discrete HMMs require **Vector Quantization** (Chapter 16) to first convert continuous feature vectors into discrete symbols before modeling.
- **Continuous HMM:** if observations are continuous feature vectors, $b_{q_t}(x_t)$ is generally a **probability density**, often modeled using a **Gaussian** or **Gaussian Mixture**

distribution:

$$b_j(x_t) = \mathcal{N}(x_t; \mu_j, \Sigma_j)$$

GMM-HMMs provide flexible, multimodal emission distributions, allowing a single state to capture several different “clusters” of acoustic realizations of the same underlying sound.

18.9 Chapter 17-18 : Practice / Exam Questions

Q1. Explain the significance of pattern matching in speech recognition. List and briefly describe the major pattern-matching approaches.

Answer: Pattern matching identifies an unknown speech input by comparing its acoustic features against stored reference patterns/models, providing a systematic way to handle speech's inherent variability (speaker, rate, pronunciation, noise) and select the most likely word/phoneme. Major approaches: (1) **Template-based matching** (distance-based, DTW) — direct comparison against stored exemplar templates; (2) **Statistical/Probabilistic modeling** (HMM, GMM) — models the statistical distribution of acoustic features, robust to natural variability; (3) **Model-based/knowledge-based recognition** — uses explicit phonetic/linguistic rules and knowledge.

Q2. Why does simple (template) matching fail for speech recognition? What additional mechanisms are needed?

Answer: Simple matching fails because: different speakers pronounce the same word with different acoustic characteristics; the same speaker's speaking rate varies across utterances; different phonemes naturally occupy different numbers of frames; and noise/channel conditions distort feature vectors unpredictably. To handle this, **temporal alignment** (e.g., Dynamic Time Warping, or the natural state-transition structure of an HMM) is needed to handle speaking-rate variability, and **probabilistic modeling** (HMM/GMM) is needed to handle acoustic variability in a statistically principled way, rather than requiring an exact match.

Q3. Why are specialized spectral distortion measures (e.g., cepstral distance, likelihood ratio distortion) preferred over simple Euclidean distance in speech pattern matching?

Answer: Simple Euclidean distance (or squared error) treats every dimension/frequency component equally and does not necessarily correspond to how humans perceive differences between two sounds. Specialized spectral distortion measures (log spectral distance, cepstral distance, weighted cepstral distance, likelihood ratio distortion, etc.) are designed to better reflect *perceptual* or *subjective* differences between speech signals — weighting spectral regions or cepstral coefficients according to their actual importance for distinguishing sounds — leading to more accurate and perceptually meaningful pattern-matching decisions.

Q4. Define Hidden Markov Model. Why are the states called "hidden"? Write the formal representation of an HMM and explain each parameter.

Answer: An HMM represents speech as a sequence of hidden states (unobservable) generating observable acoustic feature vectors, with transition probabilities governing movement between states and emission probabilities governing what observations each state produces. States are "hidden" because we cannot directly observe which state the system is in at any time — we only observe the acoustic features that state produces. An HMM is formally represented as $\lambda = (A, B, \pi)$: A = state

transition probability matrix ($a_{ij} = P(q_{t+1} = S_j | q_t = S_i)$); B = observation emission probabilities ($b_j(x_t) = P(x_t | q_t = S_j)$); π = initial state probability distribution ($\pi_i = P(q_1 = S_i)$).

Q5. What is a left-to-right (Bakis) HMM? Why is it particularly suited for modeling speech? How are word-level HMMs constructed from phoneme HMMs?

Answer: A left-to-right (Bakis) HMM only allows transitions that move forward (or stay in the same state) — backward transitions to earlier states are disallowed. It is well suited for speech because speech unfolds strictly forward in time (a phoneme's onset always precedes its steady-state, which always precedes its offset) — matching this natural, one-directional temporal structure improves both modeling accuracy and computational efficiency. A standard phoneme HMM typically has three emitting states (onset, steady-state, offset); word-level HMMs are built by **concatenating** the phoneme HMMs corresponding to each phoneme in the word's pronunciation, in sequence (e.g., "cat" = /k/-HMM + /æ/-HMM + /t/-HMM chained together).

Q6. State and explain the three classical problems of HMM, along with the algorithm used to solve each.

Answer: (1) **Evaluation:** given an observation sequence and a trained model, find $P(X|\lambda)$, the probability the model generated this sequence — solved by the **Forward Algorithm**. (2) **Decoding:** given an observation sequence and a model, find the single most likely hidden-state sequence that produced it — solved by the **Viterbi Algorithm**. (3) **Learning/Training:** given training observation sequences, estimate the model parameters (A, B, π) that best fit the data — solved by the **Baum-Welch Algorithm** (an instance of Expectation-Maximization).

Q7. Explain the Forward Algorithm with its mathematical formulation (initialization, recursion, termination).

Answer: The Forward Algorithm computes $P(O|\lambda)$ using a forward variable $\alpha_t(i)$ = joint probability of the partial observation sequence up to time t and being in state i at time t . **Initialization** ($t = 1$): $\alpha_1(i) = \pi_i b_i(o_1)$. **Recursion** ($t = 2$ to T): $\alpha_t(j) = \left[\sum_{i=1}^N \alpha_{t-1}(i) a_{ij} \right] b_j(o_t)$ — sum over all possible previous states, weighted by transition probability, then multiply by the current emission probability. **Termination:** $P(O|\lambda) = \sum_{i=1}^N \alpha_T(i)$ — sum the final-time-step scores across all ending states.

Q8. Explain the Viterbi Algorithm and how it differs from the Forward Algorithm. What is the "Viterbi path"?

Answer: The Viterbi Algorithm finds the single most probable hidden-state sequence given an observation sequence, using $\delta_t(j) = \max_{1 \leq i \leq N} [\delta_{t-1}(i) a_{ij}] b_j(o_t)$, tracking the best predecessor at each step via a backpointer $\psi_t(j)$, then backtracking from the best final state to recover the full path. It differs from the Forward Algorithm in that Viterbi uses a **max** (keeping only the single best incoming path) at each recursion

step, whereas the Forward Algorithm uses a **sum** (combining the probabilities of *all* incoming paths). The **Viterbi path** is the resulting single, optimal (maximum-likelihood) sequence of hidden states.

Q9. Explain the Baum-Welch algorithm. Why is it described as an instance of Expectation-Maximization (EM)?

Answer: The Baum-Welch algorithm estimates (trains) the HMM parameters (A, B, π) from acoustic training data, using the Forward-Backward procedure to compute state-occupancy probabilities and iteratively re-estimating the parameters to increase the training-data likelihood. It is an instance of EM because it alternates between an **Expectation (E) step** — computing expected state-occupancy/transition counts using the *current* parameter estimates — and a **Maximization (M) step** — updating the parameters to maximize the likelihood given those expected counts — repeating until the parameters converge (the likelihood stops increasing significantly).

Q10. Differentiate between discrete HMM and continuous HMM. What role does Vector Quantization play for discrete HMMs? What is a GMM-HMM?

Answer: In a **discrete HMM**, the emission probability $b_{q_t}(x_t)$ is a probability *mass* function over a finite set of discrete symbols; since raw speech feature vectors are continuous, **Vector Quantization** must first be applied to convert each continuous feature vector into a discrete codebook index/symbol before it can be used by a discrete HMM. In a **continuous HMM**, $b_{q_t}(x_t)$ is a probability *density* function directly over the continuous feature space, commonly modeled as a single Gaussian or, more flexibly, a **Gaussian Mixture Model (GMM)** — giving rise to the classical **GMM-HMM** architecture, which allows each hidden state to represent several different acoustic sub-clusters (multimodal emissions) of the same underlying sound, offering much greater modeling flexibility than a discrete/VQ-based approach.

Q11. In an HMM-based speech recognizer, explain why the Viterbi algorithm (not the Forward algorithm) is used during actual decoding/recognition of spoken input.

Answer: During recognition, the goal is to determine *what was actually said* — i.e., to recover the single best-matching word/phoneme sequence (state path) corresponding to the observed acoustic features, which is exactly the Decoding problem solved by Viterbi. The Forward algorithm, in contrast, only computes the *total* probability that a *given* model could have produced the observed sequence (useful for scoring/comparing candidate models, e.g., picking which word model best explains the data as a whole), but it does not itself recover a specific hidden-state sequence — it sums over all paths rather than identifying one specific best path. Since recognition requires an actual output (the recognized state/word sequence), Viterbi's max-based, backtrackable computation is what is actually used in the decoder.

Q12. A two-state HMM has $\pi = [0.5, 0.5]$, $A = \begin{pmatrix} 0.6 & 0.4 \\ 0.3 & 0.7 \end{pmatrix}$, and emission proba-

bilities $b_1(o_1) = 0.2$, $b_2(o_1) = 0.3$, $b_1(o_2) = 0.4$, $b_2(o_2) = 0.1$. **Using the Forward Algorithm, compute $P(O|\lambda)$ for the observation sequence $O = (o_1, o_2)$.**

Answer:

Initialization: $\alpha_1(1) = \pi_1 b_1(o_1) = 0.5 \times 0.2 = 0.10$; $\alpha_1(2) = \pi_2 b_2(o_1) = 0.5 \times 0.3 = 0.15$.

Recursion ($t=2$), State 1:

$$\begin{aligned}\alpha_2(1) &= [\alpha_1(1)a_{11} + \alpha_1(2)a_{21}] b_1(o_2) \\ &= [0.10(0.6) + 0.15(0.3)] \times 0.4 = [0.06 + 0.045] \times 0.4 = 0.105 \times 0.4 = 0.042\end{aligned}$$

Recursion ($t=2$), State 2:

$$\begin{aligned}\alpha_2(2) &= [\alpha_1(1)a_{12} + \alpha_1(2)a_{22}] b_2(o_2) \\ &= [0.10(0.4) + 0.15(0.7)] \times 0.1 = [0.04 + 0.105] \times 0.1 = 0.145 \times 0.1 = 0.0145\end{aligned}$$

Termination: $P(O|\lambda) = \alpha_2(1) + \alpha_2(2) = 0.042 + 0.0145 = \boxed{0.0565}$.